Sarawak Tun Openg University of Technology (SOTECH)

IT PROJECT 2 REPORT

CHAPTERS 1–6

Web-Based Preorder and Freshness Management System for Home-Based Entrepreneurs

Christopher Hansen

Student ID: 2402081406

Supervisor

Ms. Nur Elana Binti Mohd Saferinor

2026

ABSTRACT

Home-based food entrepreneurs use messaging applications and food-delivery platforms to reach customers. However, the systems reviewed in this project offer limited support for structured advance orders, ingredient expiry records and independent business branding. Food-poisoning reports in Malaysia and the literature on perishable-food management also motivate practical monitoring for small food businesses. This project addresses these needs through a dedicated ordering system with sales information and menu recommendations.

This project developed a web-based preorder and freshness management system using an iterative Agile approach, informed by the literature review and feedback from Wrap & Bowl Kuch in Kuching, Sarawak. The system combines date-based preordering, ingredient monitoring based on recorded shelf life and storage duration, a personalised single-vendor storefront, sales analytics and menu recommendations. Recorded stock must cover the selected delivery date. Bank transfers, customer conversations and any courier bookings take place outside the website.

Automated functional tests and browser-based checks verified ordering calculations, stock deductions, access controls and reporting behaviour in the tested scenarios. Client requirements guided the implemented workflow, while formal client acceptance and student usability results remain pending. The implementation demonstrates how ordering and business records can work together for a home-based food business. Date-based monitoring supports stock decisions but does not establish physical food safety. Further user evaluation is needed to assess recommendation effort and any effect on purchase conversion.

TABLE OF CONTENTS

CHAPTER 1 1
1.1 INTRODUCTION 1
1.2 PROBLEM STATEMENTS 2
1.3 OBJECTIVES 3
1.4 SCOPE 3
1.5 REVENUE MODEL 4
1.6 METHODOLOGY 5
1.7 GANTT CHART 5
1.8 SOFTWARE TOOLS 5
1.9 CONCLUSION 6
CHAPTER 2 7
2.1 INTRODUCTION 7
2.2 BACKGROUND STUDY 8
2.3 METHODOLOGY 14
2.4 TECHNOLOGY 16
2.5 IMPLEMENTATION IMPLICATIONS 17
CHAPTER 3 18
3.1 INTRODUCTION 18
3.2 ITERATIVE DEVELOPMENT AND REQUIREMENT REFINEMENT 18
3.3 USE CASE DIAGRAMS 19
3.4 USE CASE SPECIFICATIONS 21
3.5 ACTIVITY DIAGRAMS 29
3.6 SEQUENCE DIAGRAMS 33
3.7 CLASS DIAGRAM 38
3.8 ENTITY–RELATIONSHIP DIAGRAMS 40
3.9 REQUIREMENT TRACEABILITY AND DESIGN LIMITS 50
3.10 CONCLUSION 50
CHAPTER 4 51
4.1 INTRODUCTION 51
4.2 CUSTOMER INTERFACES 51
4.3 OWNER INTERFACES 59
4.4 IMPLEMENTATION DECISIONS AND DEVELOPMENT CHANGES 69
4.5 SUMMARY 74
CHAPTER 5 75
5.1 INTRODUCTION 75
5.2 TESTING APPROACH 75
5.3 FUNCTIONAL TEST RESULTS 76

5.4 USER EVALUATION....81
5.5 DISCUSSION AGAINST THE OBJECTIVES....82
5.6 SUMMARY....85
CHAPTER 6....86
6.1 INTRODUCTION....86
6.2 REVIEW OF THE PROJECT OBJECTIVES....86
6.3 LIMITATIONS....87
6.4 SUGGESTIONS FOR FUTURE IMPROVEMENT....88
6.5 CONCLUSION....89
REFERENCES....90
APPENDIX A — SUPERVISOR CONSULTATION RECORD....94
APPENDIX B — ORIGINALITY / SIMILARITY RECORD....94
APPENDIX C — DECLARATION OF INTEGRITY....95
APPENDIX D — APPLICATION DATA DICTIONARY....96
APPENDIX E — CLIENT FOLLOW-UP AND ACCEPTANCE FORM....109
APPENDIX F — STUDENT CUSTOMER-USABILITY QUESTIONNAIRE....111
APPENDIX G — SUPPORTING TEST EVIDENCE....113
APPENDIX H — SHORT OWNER OPERATING GUIDE....114

LIST OF FIGURES

Figure 1.1: Original IT Project 1 proposal Gantt chart (retained; not an actual completion log)..... 5
Figure 2.1: Original literature map retained from IT Project 1.....7
Figure 3.1: Customer use cases for the current storefront and account workflow.....20
Figure 3.2: Owner/staff use cases for stock, orders, reporting and configuration.....21
Figure 3.3: Checkout validation and atomic ingredient/packaging allocation.....30
Figure 3.4: Batch date evidence, validation and stock eligibility.....31
Figure 3.5: Packaging receipts, menu mappings and placement deductions.....32
Figure 3.6: Separate payment review and actual customer receipt.....33
Figure 3.7: Customer passkey registration.....34
Figure 3.8: Authentication using a saved customer passkey.....34
Figure 3.9: Authoritative pricing, scoped evidence and transactional placement.....35
Figure 3.10: Private evidence review and manual payment recording.....36
Figure 3.11: Daily/sales reporting and read-only order-slip preview.....37
Figure 3.12: Authenticated, bounded owner handover reminder processing.....38
Figure 3.13: Selected core domain classes and their associations.....39
Figure 3.14: Identity, customer profile and passkey records.....41
Figure 3.15: Catalogue, recipe, batch and inventory-log relationships.....43
Figure 3.16: Accepted order terms, private proof and ingredient allocation.....45
Figure 3.17: Current and accepted packaging requirements and stock movements.....47
Figure 3.18: Payment, amendment, expense and wastage evidence.....49
Figure 4.1: Menu browsing with category choices, prices and food cards.....52
Figure 4.2: Customer account creation with name, phone number and a passkey.....53
Figure 4.3: Separate sign-in form for kitchen staff.....54
Figure 4.4: Date-based finder with quantity, budget and menu-choice controls.....55
Figure 4.5: Basket review before entering delivery and payment details.....56
Figure 4.6: Checkout date guidance, morning time and stock-availability result.....57
Figure 4.7: Payment section with an uploaded illustrative receipt.....58
Figure 4.8: Customer order card and cancellation confirmation.....59
Figure 4.9: Editing a menu and its selling information.....60
Figure 4.10: Inventory overview with stock filters and recorded batch status.....61
Figure 4.11: Stock-batch form with date and storage information.....61
Figure 4.12: Checking ingredient requirements for a target quantity per menu.....62
Figure 4.13: Packaging balances and stock-management actions.....63
Figure 4.14: Assigning packaging pieces to one menu unit.....63
Figure 4.15: Owner order queue with filtering and receipt actions.....64
Figure 4.16: Reviewing payment details separately from food receipt.....64

Figure 4.17: Daily order summary for a date containing example orders.....65
Figure 4.18: Matching daily order slips prepared together for printing.....66
Figure 4.19: Sales overview with date controls and summary values.....66
Figure 4.20: Price and bulk-offer scenario using current cost estimates.....67
Figure 4.21: Delivery hours and area-based fees in Settings.....68
Figure 4.22: Reminder-status interface using local example configuration.....69
Figure 4.23: Main components of the web application.....70
Figure 4.24: Public summary of recorded ingredient-batch status.....73

LIST OF TABLES

Table 2.1: Summary of findings on Malaysian online food delivery consumer behaviour.....8

Table 2.2: Research areas, focus and identified gaps retained from the proposal.....11

Table 2.3: Proposal-stage feature comparison; provider features/regions may change.. 13

Table 2.4: Agile and traditional comparison, adapted from the original proposal summary (Islam & Ferworn, 2020).....14

Table 2.5: Comparison of development dimensions, adapted from the proposal's Leau et al. (2012) summary.....15

Table 3.1: Client-led design changes and their consequences.....19

Table 4.1: Main development changes and their purpose.....72

Table 5.1: Customer account and checkout checks (automated).....76

Table 5.2: Inventory and packaging checks (automated).....77

Table 5.3: Order records and reporting checks (automated).....78

Table 5.4: Menu recommendation checks (automated).....79

Table 5.5: Selected problems, corrections and retest outcomes.....79

Table 5.6: Selected security and concurrent-order checks.....80

Table 5.7: Evidence related to the project objectives.....83

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application programming interface
CSRF	Cross-site request forgery
DRF	Django REST Framework
ERD	Entity–relationship diagram
FEFO	First-expiry, first-out
FK / PK	Foreign key / primary key
FYP	Final-year project
MAE	Mean absolute error
MYR / RM	Malaysian ringgit
ORM	Object-relational mapper
RPC	Remote procedure call
SMTP	Simple Mail Transfer Protocol
UAT	User acceptance testing
UML	Unified Modeling Language
WAPE	Weighted absolute percentage error
WCAG	Web Content Accessibility Guidelines

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The home-based food industry experienced significant growth during and after the COVID-19 pandemic (Pallianysamy et al., 2025). According to Statista (2024), the online food delivery market in Malaysia is forecasted to reach USD 2.77 billion in 2024 and is projected to grow to USD 4.63 billion by 2029.

However, existing food ordering platforms may not adequately address the specific needs of home-based food entrepreneurs, particularly in supporting structured pre-order operations. Some local home-based food sellers might rely on WhatsApp for ordering — either through its structured cart feature or informal chat — or use large food delivery platforms such as GrabFood and FoodPanda. While the big platforms offer scheduled ordering, the proposal-stage comparison recorded advance booking windows of approximately 1–2 days, which may not align with the needs of home-based sellers who prepare food in batches and require confirmed orders further in advance. WhatsApp, meanwhile, has no proper mechanism for date-based preordering in either mode: in its cart feature, any date must be manually written into the product title or description, requiring the seller to read and verify each date themselves rather than having the system check availability or capacity automatically; chat-based arrangements, similarly, rely entirely on the seller manually tracking dates and capacity.

Neither WhatsApp nor large food ordering platforms such as GrabFood and FoodPanda offer any feature for monitoring ingredient freshness, leaving sellers without a structured way to track raw food expiry. This is particularly concerning given that storing large quantities of perishable goods increases the risk of microbial growth (Pallianysamy et al., 2025), a risk further worsened by Malaysia's indoor temperature range of 21°C to 37°C (Gou et al., 2018), which accelerates food spoilage. Real-life food poisoning incidents, such as the illness of 65 primary school students after consuming flat noodle soup in May 2024 ("Sixty-five Sungai Petani primary school pupils suffer food poisoning," 2024) and the deaths caused by spoiled fried noodles and eggs in Selangor, highlight the seriousness of this problem ("Food poisoning turns fatal," 2024).

In addition, GrabFood and FoodPanda place multiple competing sellers within the same marketplace environment, limiting opportunities for sellers to establish distinctive branding and direct customer relationships. WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence. Furthermore, the basic WhatsApp ordering workflow reviewed here does not provide the integrated analytics and menu-recommendation functions proposed for this system. Sellers who need to adopt a different platform for preordering, freshness monitoring, or a dedicated storefront would also benefit from integrated analytics and recommendation features, rather than relying on separate or absent tools. The absence of recommendation mechanisms may contribute to customer decision fatigue, which could negatively influence purchasing decisions and potentially increase the likelihood of purchase abandonment, while limited access to sales analytics restricts sellers' ability to monitor performance and make informed business decisions.

To address these issues, this proposal introduces a web-based food pre-ordering system that allows home-based food sellers to manage personalised storefronts and strengthen their branding efforts. The system also includes raw ingredient freshness monitoring based on storage duration, sales analytics to support better business decisions, and a menu recommendation system to assist customers in selecting suitable menu options and enhance the overall purchasing experience.

In conclusion, the proposed web-based system aims to assist home-based food sellers in managing pre-order operations more efficiently through a dedicated platform tailored to their business needs. The system further incorporates ingredient freshness monitoring to support food safety practices, while providing personalized storefront management, sales analytics, and menu recommendation features to enhance sellers' operational effectiveness and customer experience.

1.2 PROBLEM STATEMENTS

1. Lack of reliable advance preordering: The proposal-stage comparison recorded approximately 1–2-day advance windows for GrabFood and FoodPanda, which may not align with the needs of sellers preparing food in batches further ahead. WhatsApp's cart feature has no structured date field, requiring sellers to manually write and verify dates themselves rather than having the system check availability automatically. Chat-based ordering has no structure at all, leaving sellers to track every order manually.

2. Lack of ingredient freshness monitoring: The consumer ordering interfaces compared in this proposal do not provide the proposed kitchen-level ingredient shelf-life and storage records, limiting sellers' ability to proactively manage food safety.

3. Lack of dedicated single-vendor storefront: GrabFood and FoodPanda place competing sellers in the same multivendor environment, limiting branding opportunities. WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence.

4. Limited business and decision support tailored to small operators: The basic WhatsApp ordering workflow reviewed here does not provide these integrated analytics or menu-recommendation functions. Sellers who need to adopt a different platform for preordering, freshness monitoring, or a dedicated storefront would also benefit from integrated analytics and recommendation features, rather than relying on separate or absent tools.

1.3 OBJECTIVES

1. To design and develop a web application that supports flexible date-based food preordering and scheduling for home-based food entrepreneurs.

2. To implement a food stock freshness monitoring feature based on ingredient shelf life and storage duration.

3. To develop a personalised single-vendor storefront module that allows sellers to strengthen branding without direct competitor distraction.

4. To implement a sales analytics dashboard that supports data-driven business decision-making for home-based food operators.

5. To develop a menu recommendation system that reduces customer decision fatigue and improves purchase conversion rates.

1.4 SCOPE

1.4.1 Customer Scope

- Customer can view the seller's storefront and product listings with category filters
- Customer can select available preorder dates when placing orders
- Customer can view food recommendations and add items to cart

1.4.2 Food Entrepreneur Scope

- Seller can log in as admin and manage product listings
- Seller can manage preorder availability settings
- Seller can monitor ingredient freshness status based on remaining shelf life duration
- Seller can view sales analytics to support business decisions
- Seller can customise their storefront including store name, story, contact information and menu photographs

1.4.3 System Scope

- The system checks future preorder dates and kitchen availability, and distinguishes confirmed bookings from explicit owner-review requests
- The system tracks and displays ingredient freshness status based on remaining shelf life
- The system shows recommended and all product listings filterable by category
- The system displays relevant features based on user role — entrepreneur or customer

1.4.4 IMPLEMENTED SCOPE CLARIFICATION

The case study is Wrap & Bowl Kuch in Kuching. Client feedback refined the system to use name and phone details with a passkey, morning delivery or collection, area-based fees, payment receipt uploads and ingredient/packaging stock deductions. Detailed cooking-task planning is no longer required. Chapters 3–6 describe these changes; the five objectives remain unchanged.

1.5 REVENUE MODEL

The proposed solution is designed to help home-based food entrepreneurs manage preordering operations through a structured and fixed-cost platform with record-based ingredient-date monitoring and personalised branding. The proposed commercial model has two revenue streams; automated seller subscription billing is outside the implemented system:

1. Upfront payment A one-time fee is charged for initial system access and setup.

2. Recurring fee Monthly maintenance and hosting fees are charged to sustain system operation.

1.6 METHODOLOGY

This project follows an Agile methodology due to its suitability for small-scale iterative development. Development is organised into iterative cycles where each cycle involves requirement gathering, design, implementation and testing of specific features. The process begins with identifying home-based food entrepreneurs' needs and customer pain points. Each feature — including product preordering, ingredient freshness monitoring, storefront management, sales analytics and menu recommendation — is then designed, implemented and tested iteratively. The system is refined based on feedback and testing outcomes throughout development.

1.7 GANTT CHART

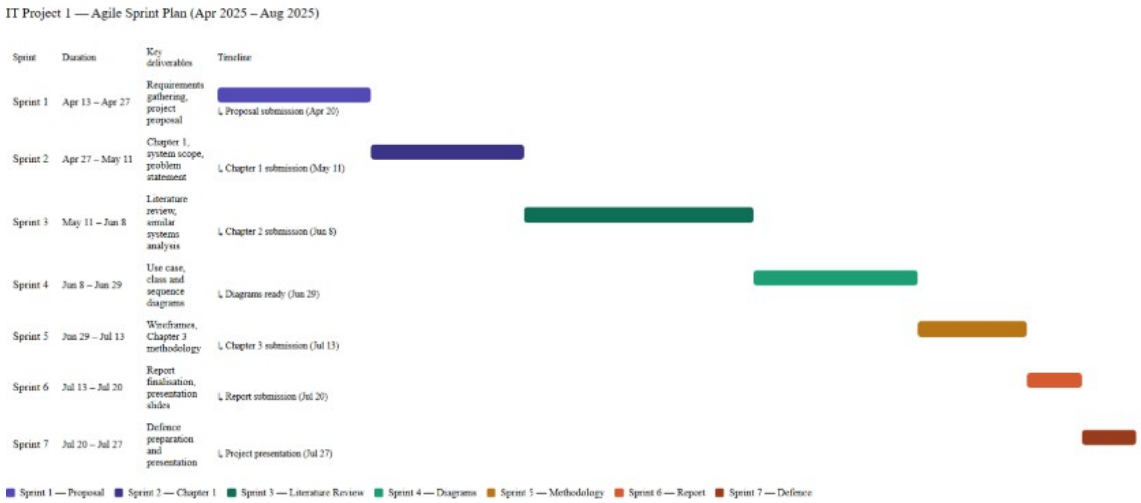


Figure 1.1: Original IT Project 1 proposal Gantt chart (retained; not an actual completion log).

1.8 SOFTWARE TOOLS

1. React — Frontend framework for building interactive user interfaces

2. Django — Backend framework supporting rapid development with built-in ORM and MVT architecture

3. PostgreSQL — Relational database suited for structured interconnected data entities

4. Visual Studio Code — Code editor with extensive Python and JavaScript extension support

5. GitHub — Version control platform for tracking code changes and managing development history

1.9 CONCLUSION

The proposed system aims to help home-based food entrepreneurs manage preorder operations more efficiently through a dedicated fixed-cost web platform tailored to their business needs.

By providing flexible date-based preordering, ingredient freshness monitoring, personalised storefront management, sales analytics and menu recommendation features, the system addresses key operational gaps faced by home-based food entrepreneurs. Operating through a dedicated platform allows sellers to maintain their own branded storefront and build direct customer relationships independent of large multivendor marketplace environments.

The project follows an Agile methodology to support iterative feature development and flexibility throughout the development process. Therefore, the proposed system supports the growth of local home-based food SMEs while contributing to food safety awareness and improved customer purchasing experience.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

This chapter consists of three parts including background study, methodology, and technology.

The background study discusses the evolution of the food ordering industry, digital transformation in Malaysia's food and beverage (F&B) sector, and the research interest landscape related to preorder and freshness management systems. In addition, similar existing systems and current technological trends relevant to the proposed system will also be reviewed and discussed.

The methodology section discusses the comparison between Agile and Traditional software development approaches to determine the most suitable methodology for the proposed system.

Meanwhile, the technology section describes the framework and database technologies considered for the project. A comparison between Django and Express.js will be carried out to determine the most suitable framework for the proposed system. Moreover, a comparison between Firebase and PostgreSQL will also be conducted to identify the most suitable database solution for the proposed system.

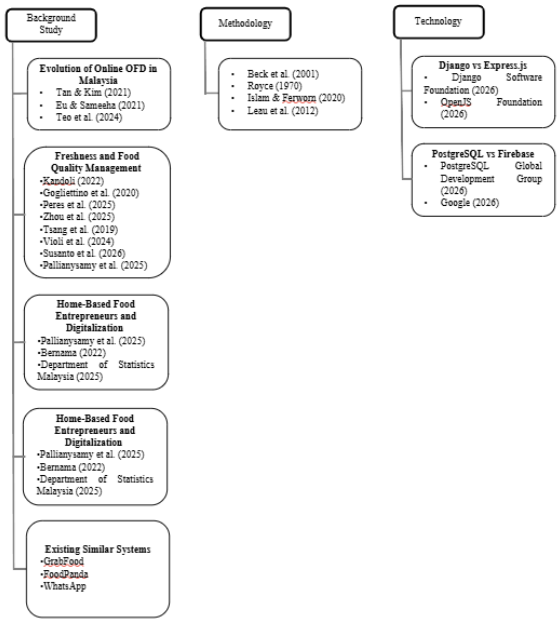


Figure 2.1: Original literature map retained from IT Project 1.

2.2 BACKGROUND STUDY

2.2.1 Evolution of Online Food Ordering Industry in Malaysia

The Online Food Ordering industry boomed in Malaysia in 2017, driven by smartphone adoption and internet accessibility (Tan & Kim, 2021). The COVID-19 pandemic further accelerated this growth as consumers shifted toward online food ordering amid the shutdown of non-essential services and dining restrictions (Eu & Sameeha, 2021). Within this growing market, research has identified food quality and health considerations as significant factors influencing Malaysian consumer behavior in OFD platforms (Teo et al., 2024).

Therefore, understanding consumer expectations regarding food quality and freshness within Malaysia's digital food ecosystem is important in identifying the relevance of the proposed web-based preorder and freshness management system. Table 2.1 presents an analysis of key findings and their implications for the Malaysian OFD landscape.

Table 2.1: Summary of findings on Malaysian online food delivery consumer behaviour.

APA citation	Key finding	Relevance
Tan & Kim (2021)	Food quality, service approach and perceived price were linked to satisfaction.	Supports attention to food quality and service.
Eu & Sameeha (2021)	76.9% perceived limited healthy options; 85.9% expressed willingness to purchase healthy food.	Highlights a gap in perceived choice/quality.
Teo et al. (2024)	Health, quality, service and value influence continuance intention.	Supports integrating quality and service information.

Across these studies, food quality and healthy food availability consistently emerge as critical factors influencing Malaysian consumers' OFD usage and satisfaction. However, existing platforms are perceived as inadequately addressing these concerns. Given that freshness is identified as a core component of food quality (Tan & Kim, 2021), this suggests a gap for more practical freshness management features within digital food ordering systems.

2.2.2 Freshness and Food Quality Management in Digital Food Systems

Food storage is one method used to preserve food for future consumption (Kandoli, 2022). However, certain perishable food products with high nutrient content

remain vulnerable to microbial contamination and pathogen growth if storage conditions are not properly managed (Gogliettino et al., 2020). The presence and growth of these microorganisms influence the freshness status and quality of raw food, thereby requiring proper freshness management before food is served to consumers.

Food quality management has become a critical aspect of the food industry, particularly in ensuring food safety and freshness throughout the supply chain. Recent studies have explored the integration of Industry 4.0 technologies such as sensor systems, blockchain, and artificial intelligence to improve food monitoring, process automation, and traceability mechanisms (Peres et al., 2025). Furthermore, TTIs and FIs were explored in food shelf-life monitoring (Zhou et al., 2025).

However, these technologies are generally implemented within large-scale food supply chains and industrial environments where structured operational processes, dedicated quality control divisions, and larger financial resources are available. Such implementations commonly emphasize end-to-end supply chain monitoring rather than operational-level freshness management for small-scale food services or home-based food businesses.

Another study explored the integration of IoT, fuzzy logic, and zero-order quality degradation models into an Internet of Things-based Shelf Life Management System (Tsang et al., 2019). Nevertheless, the implementation complexity and infrastructure requirements of such systems may reduce their suitability for home-based food microbusinesses.

Similarly, Violi et al. (2024) proposed a mathematical inventory-routing model for perishable products. The model assists warehouses in determining delivery priorities to minimize spoilage while addressing stochastic demand using historical sales data and Conditional Value-at- Risk (CVaR) risk analysis. However, the computational complexity and integration requirements of such approaches may limit their applicability in small-scale academic projects and home-based food business operations. Furthermore, these systems may require IoT infrastructure to monitor physical inventory movement, potentially increasing implementation costs.

Susanto et al. (2026) also proposed a Quality Management System framework for food SMEs consisting of Quality Planning, Quality Assurance, and Quality Control and Improvement stages. However, the implementation scope of such systems may exceed the operational requirements of small-scale home-based food businesses.

Moreover, Pallianysamy et al. (2025) highlighted the need for monitoring and inspection solutions for home-based food operations from a food safety perspective. Nevertheless, implementing advanced monitoring technologies such as cameras, sensors, or IoT devices may introduce additional infrastructure and maintenance costs that are less suitable for lightweight small-scale systems.

Therefore, a research gap exists in the development of lightweight and cost-effective freshness management approaches that can be integrated into smaller-scale digital food systems such as online preorder and home-based food ordering platforms.

2.2.3 Home-Based Food Entrepreneurs and Digitalization

Home-based food businesses in Malaysia represent a growing segment particularly following COVID-19 (Pallianysamy et al., 2025). This growth coincides with significant expansion in Malaysia's digital food market — ecommerce transactions increased by 21.8 percent from 2020 to RM1.1 trillion in 2021 (Bernama, 2022), with revenue for the first nine months of 2025 reaching RM937.5 billion (Department of Statistics Malaysia, 2025).

However, as identified in Section 2.2.5, existing platforms present several limitations for home-based food operators. GrabFood and FoodPanda were recorded in the proposal comparison as offering approximately 1–2-day advance windows, which is insufficient for sellers who prepare food in batches and require confirmed orders further ahead of time. WhatsApp's Cart feature has no structured date field, requiring sellers to manually write and verify dates themselves rather than having the system check availability automatically, while informal chat-based ordering has no structure at all, making it difficult for sellers to reliably track orders and avoid missing them amid casual conversation. None of these platforms provide ingredient freshness monitoring features.

Furthermore, existing platforms lack dedicated personalised storefronts — GrabFood and FoodPanda offer limited personalisation within competitive multivendor environments, while WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence. While dedicated single-vendor ordering systems exist for large established food brands such as McDonald's, these systems are developed for large-scale commercial operations and are not accessible to informal home-based food micro-entrepreneurs as ready-to-use platforms.

Home-based entrepreneurs whose operational needs — flexible preordering, freshness monitoring, or a dedicated single-vendor storefront — are not met by GrabFood, FoodPanda, or WhatsApp would therefore need to adopt a different platform altogether; once on such a platform, they would also benefit from integrated sales analytics and recommendation features rather than having to rely on separate or absent tools. While GrabFood and FoodPanda do provide sales analytics through vendor dashboards (Table 2.3), and WhatsApp offers none at all, neither pathway currently combines these analytics and recommendation capabilities with the preorder, freshness, and storefront features that the proposed system addresses together as a single integrated solution.

2.2.4 Research Interest & Academic Relevance Landscape

Tan & Kim (2021) examined consumer satisfaction in Malaysian OFD, focusing on factors such as food quality and pricing. However, this study focuses on consumer satisfaction rather than operational tools for food businesses.

In addition, research on healthy food awareness in OFD platforms indicates that Malaysian consumers value healthier food options and food quality visibility (Eu & Sameeha, 2021). However, these studies mainly focus on healthy food availability perception and food choice behavior, rather than real-time freshness monitoring or freshness-status tracking within online food ordering systems.

Moreover, recent studies exploring digital technologies in food monitoring suggest increasing adoption of Artificial Intelligence (AI), Internet of Things (IoT), and smart sensors for food tracking and waste reduction (Zhou et al., 2025). However, these solutions are often complex and costly, making them less suitable for small-scale food businesses. Therefore, there is a need for a more lightweight and cost-effective approach to freshness management, such as a system based on shelf-life tracking and inventory entry time. Table 2.2 presents an analysis of existing research areas and their identified gaps relevant to the proposed system.

Table 2.2: Research areas, focus and identified gaps retained from the proposal.

Research area	Study focus	Gap relevant to this project	Citation
Consumer satisfaction	Quality, price and service	Operational preorder/freshness tools	Tan & Kim (2021)
Healthy food availability	Perceptions of variety and price	Practical ingredient-date visibility	Eu & Sameeha (2021)

Research area	Study focus	Gap relevant to this project	Citation
Digital food monitoring	AI/IoT and larger supply chains	Lightweight tools for small operators	Zhou et al. (2025)

Overall, the existing literature tends to focus either on consumer behavior, healthy food availability perception, or advanced technological solutions, with limited attention given to affordable, practical freshness management systems tailored for home-based food entrepreneurs. This identifies a clear research gap that the proposed web-based preorder and freshness management system aims to address.

2.2.5 Existing Similar Systems

The following platform descriptions and Table 2.3 preserve the proposal-stage comparison. Ordering windows and commercial features can vary by region, account and date; they were not revalidated as current provider promises for this implementation review. The comparison is not a universal claim that no other software supports these functions.

This section compares existing systems to identify features currently available and limitations that the proposed system aims to address.

2.2.5.1 GrabFood

GrabFood is the most popular food ordering platform among university students (Lok et al., 2024). It was founded in 2018 which provided online food delivery solutions in Southeast Asia (Grab, 2018). The platform primarily focuses on registered restaurant vendors and large-scale food businesses and was recorded in the proposal comparison as allowing advance preordering up to two days. Moreover it is a multivendor platform, therefore, they provide less freedom in personalized content. However, publicly available platform features indicate limited support for freshness-status monitoring and inventory- oriented analytics.

2.2.5.2 FoodPanda

FoodPanda was launched in 2012 before expanding across Southeast Asia (Foodpanda, 2012). It allows advance preordering up to 1 day. Same with GrabFood, the platform primarily focuses on registered restaurant vendors and large-scale food businesses. It allows limited preordering and does not provide personalized storefront. It also does not offer food freshness monitoring features, and provides limited sales

analytics through its vendor dashboard. Recommendation capabilities are relatively limited and primarily focused on vendor visibility rather than personalized freshness-oriented recommendations.

2.2.5.3 WhatsApp

WhatsApp may be used as an informal preorder communication platform among small- scale food businesses and microentrepreneurs in Southeast Asia. However, the preorder process is largely unstructured and dependent on manual chat management, which may increase the risk of missed orders and communication inefficiencies. In addition, WhatsApp does not provide dedicated features such as freshness monitoring, analytics dashboards, or personalized preorder management.

This section reviews several existing platforms related to online food ordering and preorder management to identify their functionalities, limitations, and relevance to the proposed system. The comparison focuses on preorder flexibility, personalization, freshness monitoring, sales analytics, and recommendation capabilities. Through this comparison, the limitations of current systems can be identified to justify the need for the proposed system.

Table 2.3: Proposal-stage feature comparison; provider features/regions may change.

Feature	**GrabFood**	**foodpanda**	**WhatsApp**	**McDonald's**	**Proposed system**
Advance ordering	Up to 2 days	Up to 1 day	Manual date in conversation	Limited scheduling	Flexible seller rules
Single-vendor storefront	Marketplace	Marketplace	Basic catalogue/chat	Brand-owned	Dedicated kitchen
Ingredient-date monitoring	Not in compared interface	Not in compared interface	Manual records	Not in compared interface	Shelf-life records
Sales analytics	Vendor analytics	Vendor analytics	Not in basic ordering flow	Not evaluated	Integrated reports
Recommendations	Platform-based	Category-based	Not in compared flow	Not evaluated	Explainable ranking

Note. McDonald's is included as a reference example of an existing single-vendor dedicated ordering system. It is not a practical alternative for home-based food entrepreneurs, as such systems are developed for large-scale commercial operations and are not accessible as ready-to-use platforms for informal micro-entrepreneurs.

2.3 METHODOLOGY

Agile methodology is an iterative software development approach where developers can always revisit earlier stages of the Software Development Life Cycle after feedback. According to Beck et al. (2001), the Agile Manifesto emphasizes four core values: individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. Individuals and interactions over processes and tools means the methodology capitalizes more on human resources rather than advanced technologies or tools. Working software over comprehensive documentation means the methodology makes use of early prototypes for testing purposes rather than planning everything from start through documentation. Customer collaboration over contract negotiation means the method prefers continuous customer feedback rather than a fixed negotiation contract at the beginning. Responding to change over following a plan means the methodology prioritises adapting to new information and feedback rather than rigidly adhering to an initial plan.

A traditional model is a software development process where each stage is rigidly planned from the beginning. According to Royce (1970) it consists of system requirements, software requirements, analysis, program design, coding, testing, operation. He, however, said that the model is so simple that it's very risky for larger and more costly software projects.

Agile methodology is an incremental software development process while traditional methodology is a more formal or plan-driven software development approach (Islam & Ferworn, 2020). The comparisons between the two are described in the table below.

Table 2.4: Agile and traditional comparison, adapted from the original proposal summary (Islam & Ferworn, 2020).

Dimension	Agile (proposal summary)	Traditional (proposal summary)
Approach	Adaptive	Predictive
Success emphasis	Business value	Conformance to plan
Typical project context	Small / evolving	Larger / stable
Management emphasis	Decentralised	Centralised
Change	Adaptability	Controlled changes

Dimension	Agile (proposal summary)	Traditional (proposal summary)
Culture	Collaboration	Formal coordination
Documentation	Selected supporting records	Extensive records
Focus	People and iteration	Process and sequence
Cycles	Numerous	Limited
Domain	Exploratory	Predictable
Team	Small / adaptable	Larger / structured
Upfront planning	Incremental	Comprehensive
Return	Incremental delivery	Later completion

This table summarises the proposal's comparison rather than establishing universal rules about documentation, team size or engineering skill. The present project uses iterative development with written requirements, diagrams and automated tests in every relevant increment.

Agile methodology is selected for this project due to its suitability for small-scale projects, support for iterative feature development, and flexibility to accommodate requirement changes throughout the development process. These characteristics align with the nature of this academic project, where features may be refined or extended based on feedback and testing outcomes. Leau et al. (2012) provide a comparative analysis between Agile and traditional methodologies regarding rework costs, testing protocols, and project scalability, as outlined in Table 2.5.

Table 2.5: Comparison of development dimensions, adapted from the proposal's Leau et al. (2012) summary.

Dimension	Agile	Traditional
Requirements	Iteratively refined	Defined before implementation
Rework	Earlier feedback can reduce later rework	Late changes can be expensive
Direction	Changeable within iterations	More fixed baseline
Testing	Each iteration	Historically later stages
Customer involvement	Frequent	Often concentrated at milestones
Developer skills	Interpersonal and business knowledge	Technical and coordination skills still required

Dimension	Agile	Traditional
Project scale in proposal	Low to medium	Larger plan-driven context

This table summarises the proposal's comparison rather than establishing universal rules about documentation, team size or engineering skill. The present project uses iterative development with written requirements, diagrams and automated tests in every relevant increment.

2.4 TECHNOLOGY

2.4.1 Django vs Express.js

According to the Django Software Foundation (2026), Django is a high-level Python web framework designed to support the rapid development of backend code. The framework is widely adopted for its Model-View-Template (MVT) architecture, which separates the application's core data, logic, and user interface. Within this architecture, a Model is defined as a Python class object that maps directly to a database table, providing an object-relational interface to database queries (Django Software Foundation, 2026). The View serves as a function or class that accepts web requests and returns the corresponding web responses. Finally, the Django architecture incorporates Templates, which consist of HTML code enhanced with template tags to dynamically display the data retrieved from the database to the end user (Django Software Foundation, 2026).

According to the OpenJS Foundation (2026), Express.js is a minimalist web framework designed to provide a robust set of features for web and application development. The framework relies primarily on two core functionalities: routing and middleware. Routing defines how application endpoints or URIs respond to specific incoming client requests, while middleware functionality allows developers to access and manipulate the request (req) and response (res) objects during the application's cycle (OpenJS Foundation, 2026).

Django was selected over Express.js due to prior development experience with the framework, which reduces learning overhead in a time-constrained academic project. Additionally, Django's MVT architecture enforces a clear project structure that supports code maintainability and organisation throughout development.

2.4.2 PostgreSQL vs Firebase

According to PostgreSQL Global Development Group (2026), PostgreSQL is an open- source object-relational database that has been developed for more than 35 years, making it a reputable database. It has many features that help developers in application building, administrators that protect data integrity, and support datasets of varying sizes. Furthermore, many features that are supported by SQL standard are also supported by PostgreSQL making it compliant with the SQL standard.

The Google (2026) comparison concerns Cloud Firestore, Firebase's document-oriented database, rather than every service under the Firebase platform. Meanwhile, it stores data in documents where each document contains key-value pairs. A document object is a type of JSON object that supports extra data types and has document size limits. Collection is the container that holds instances of the same data structure.

PostgreSQL is selected as its relational data model better suits structured data with interconnected entities, improving readability and maintainability compared to Firebase's document-based storage which may result in deeply nested data structures that are harder to navigate and maintain. Furthermore, Firebase's NoSQL structure is not natively supported by Django's ORM, which would require bypassing Django's built-in database management tools and increasing implementation complexity.

2.5 IMPLEMENTATION IMPLICATIONS

The literature supports a lightweight system that brings preorder dates, recorded ingredient life, a dedicated storefront and decision support together. The implementation was refined through client feedback without changing the objectives. Chapters 4–6 distinguish the functions tested from food-safety and business outcomes that still need evaluation.

CHAPTER 3

METHODOLOGY AND SYSTEM DESIGN

3.1 INTRODUCTION

This chapter presents the system design for Wrap & Bowl Kuch, Ayu Shafinaz's home-based food business in Kuching, Sarawak. It explains the iterative development process, customer and owner use cases, application interactions and database relationships. The design supports the objectives in Section 1.3 while reflecting the client's menu, morning delivery arrangements, stock records and payment-verification needs. The requirements were informed by the supplied menu poster, founder introduction and informal feature feedback.

The principal refinement is a simpler operating workflow. Scheduling means choosing a permitted future delivery or collection date and time, rather than requiring the owner to define cooking tasks or confirm a preparation plan. Customer access uses a name/phone profile with a passkey. The owner retains separate staff authentication. Ingredient and packaging allocations occur when an order is accepted. Payment verification and actual food receipt remain distinct events. Earlier scheduling and payment components remain in the codebase for compatibility, but they are not the active client checkout design.

3.2 ITERATIVE DEVELOPMENT AND REQUIREMENT REFINEMENT

Development followed small requirement, implementation, verification and feedback cycles. The original problem, literature rationale and objectives provided the baseline. Client feedback subsequently prioritised name-and-phone contact, packaging stock, payment-proof upload, area fees, daily order summaries, printing and a received/not-received presentation. Additional iterations addressed phone normalisation, accepted-order snapshots, stock shortages, passkey recovery limitations and consistent interfaces.

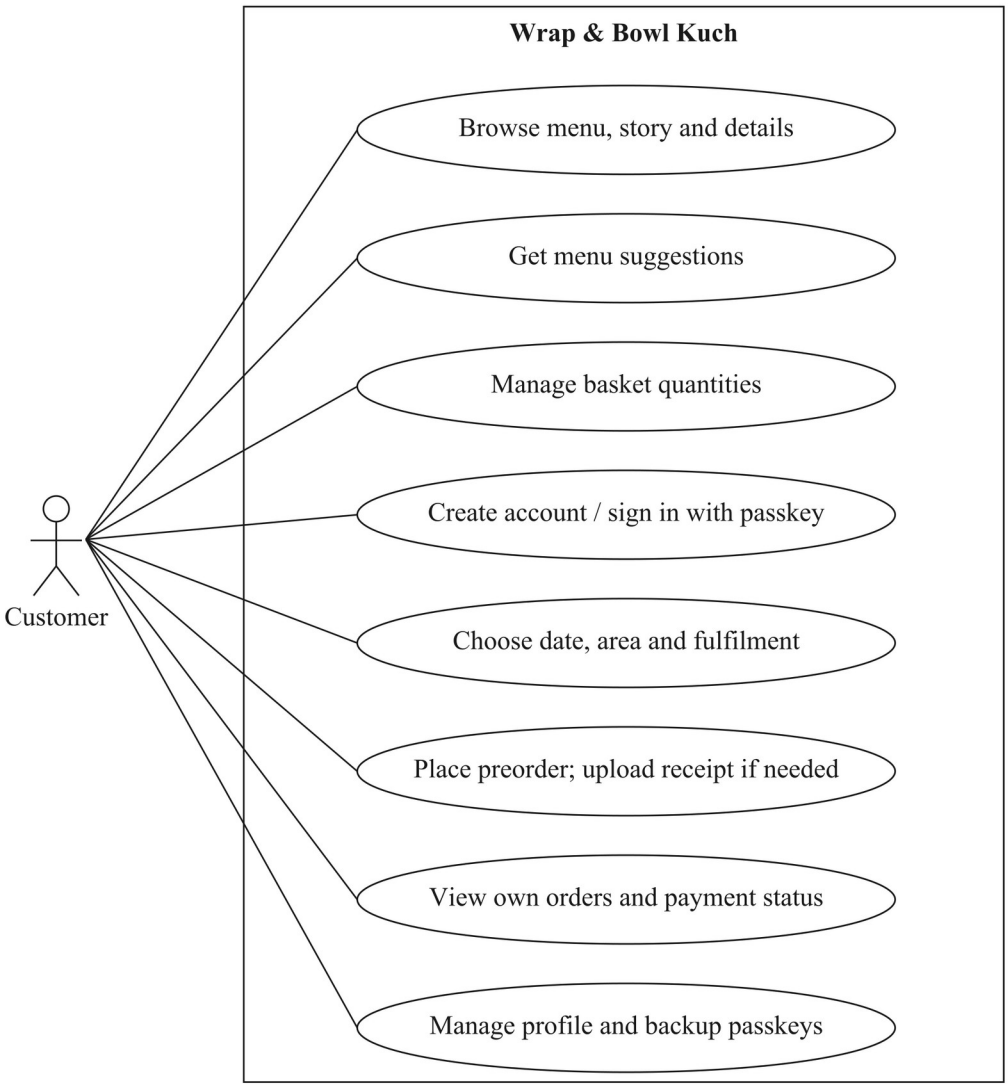
Each iteration reviewed a working feature and checked its effect on orders, stock and account access. Informal client meetings guided requirements changes. Chapter 4 explains the resulting implementation, while Chapter 5 separates functional testing from formal user evaluation.

Table 3.1: Client-led design changes and their consequences

Requirement change	Current design consequence	Preserved constraint
Reduce registration effort	Name and phone profile with native passkey registration/sign-in	A phone number is contact data, not proof of identity
No cooking Planner in daily use	Future handover date/time, offered days and cutoffs; no cooking reservation	A date must still be permitted and within the supported horizon
Track packaging as well as ingredients	Packaging quantities, menu assignments and per-order allocations	A shortage must prevent the whole placement
Upload payment evidence	Private sanitised receipt image associated with the quoted selection	An upload never means the bank payment has been verified
Morning deliveries and area fees	Editable minute-based hours and delivery-area price mapping	The server calculates the accepted fee
Simplify order handling	Received / Not received, with cancellation and audited corrections	Food receipt does not record money received

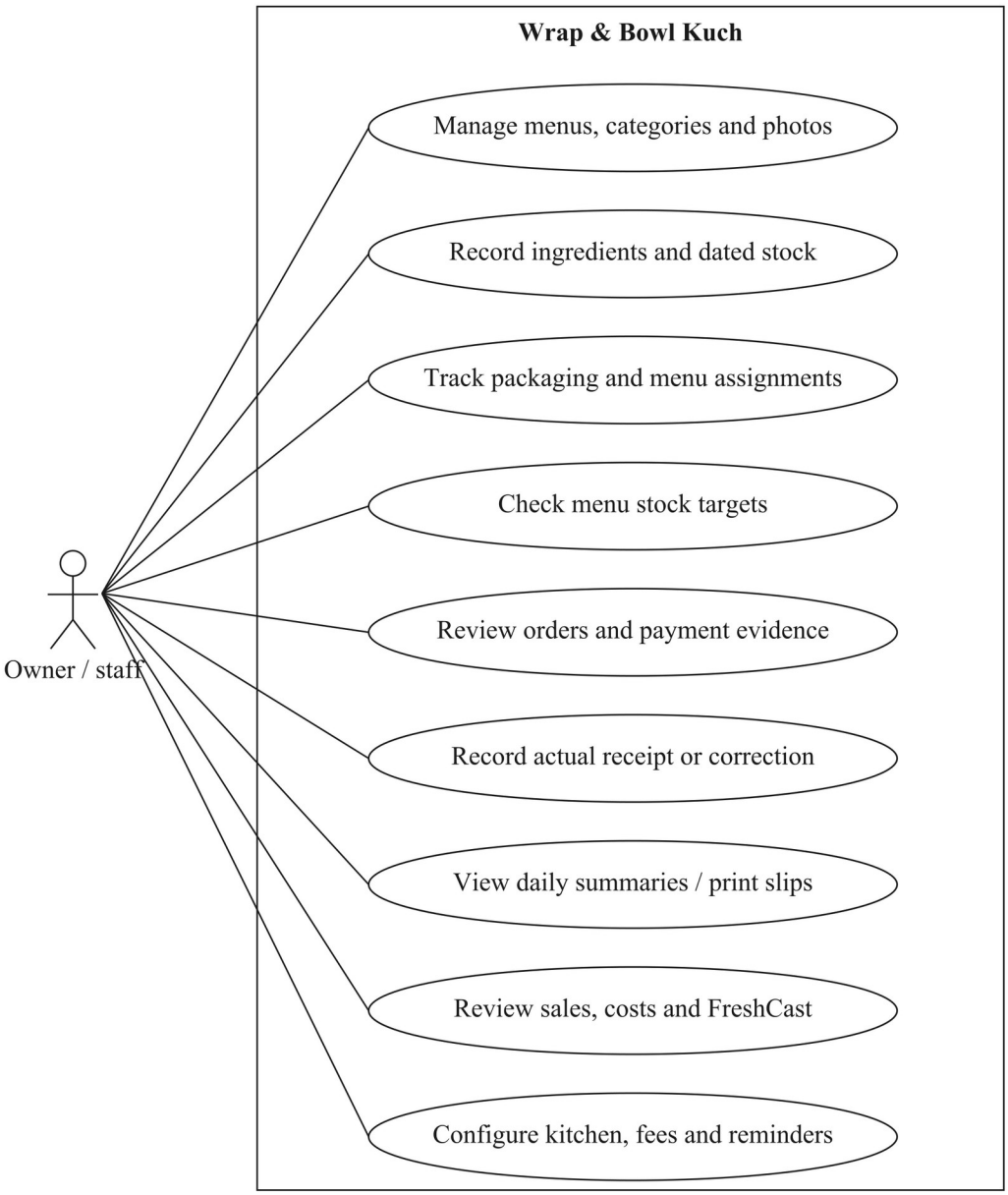
3.3 USE CASE DIAGRAMS

The customer diagram represents customer actions, including public browsing and actions that need an account. Public browsing does not require an account; checkout and private history do. The owner diagram represents authenticated staff. A bank or courier is not drawn as an automatically connected application actor because the current website does not call either provider's operational API. The customer uses a separate banking application, and the kitchen arranges the rider externally.



Browsing is public. Ordering requires an account. Bank transfer occurs outside the website.

Figure 3.1: Customer use cases for the current storefront and account workflow.



Staff password sign-in is required. A reminder or uploaded receipt never confirms delivery or payment.

Figure 3.2: Owner/staff use cases for stock, orders, reporting and configuration.

3.4 USE CASE SPECIFICATIONS

The specifications below describe the current client workflow. Shared preconditions include a valid session, role-appropriate permissions and current saved configuration. An error must leave an order, payment or stock operation either fully recorded or unrecorded; the interface must not treat an uncertain response as a successful second submission.

3.4.1 BROWSE MENU

Use case	Current specification
ID / actor	UC-01 — Visitor/customer
Precondition	Public storefront available.
Main flow	Open the menu; search/filter categories; inspect price, contents and offered days.
Alternative / exception	Empty results or unavailable records show a recovery state.
Postcondition	No order or stock change.

3.4.2 CREATE CUSTOMER ACCOUNT

Use case	Current specification
ID / actor	UC-02 — Customer
Precondition	Browsing session; supported passkey device.
Main flow	Enter name and full phone; request registration; complete native verification.
Alternative / exception	Invalid phone, unsupported device, rejected origin/challenge or cancelled ceremony does not create false authentication.
Postcondition	Registered account preserves its existing basket; no website password/email required.

3.4.3 SIGN IN

Use case	Current specification
ID / actor	UC-03 — Customer / staff
Precondition	Registered passkey or staff credentials.
Main flow	Use the customer passkey; staff use the separate protected password form.
Alternative / exception	Wrong credential remains an error; the phone alone never opens another account.
Postcondition	Verified session and role-appropriate access.

3.4.4 MANAGE PROFILE AND PASSKEYS

Use case	Current specification
ID / actor	UC-04 — Customer
Precondition	Authenticated account.
Main flow	Review saved contact; reauthenticate for sensitive changes; add or remove a backup key.
Alternative / exception	Last-key removal and unauthorised/stale verification are rejected.
Postcondition	Profile/key change is saved without rewriting accepted order contacts.

3.4.5 GET MENU SUGGESTIONS

Use case	Current specification
ID / actor	UC-05 — Customer
Precondition	Menus configured; date finder needs a valid selection.
Main flow	Browse suggestions, or choose a date, total items, mixed/same-menu choice and optional maximum whole-order budget; inspect up to three alternatives.
Alternative / exception	No feasible match is explained; replacing a basket needs deliberate confirmation.
Postcondition	Proposed basket obeys relevant stock/pricing rules; no handover hour reserved.

3.4.6 MANAGE BASKET

Use case	Current specification
ID / actor	UC-06 — Customer
Precondition	Browsing or registered session.
Main flow	Add food, adjust quantity, remove a line and review subtotal.
Alternative / exception	Failures retain truthful current/error state; no duplicate successful add is invented.
Postcondition	Basket changes only; accepted stock is not yet consumed.

3.4.7 CHOOSE FULFILMENT AND DATE

Use case	Current specification
ID / actor	UC-07 — Customer
Precondition	Registered account and nonempty basket.
Main flow	Select delivery/collection, configured area if needed, common offered date and allowed time.
Alternative / exception	Invalid weekdays, cutoffs, time bounds or restricted area block a valid quote.
Postcondition	Current server-priced fulfilment selection; no accepted order yet.

3.4.8 UPLOAD PAYMENT PROOF

Use case	Current specification
ID / actor	UC-08 — Customer
Precondition	Manual payment available; confirmed quote and account.
Main flow	Pay externally if appropriate; select an eligible receipt image and upload it.
Alternative / exception	Nonimage, excessive size, wrong scope, unauthorised/private read or reused evidence is rejected.
Postcondition	Private scoped proof ID; payment is still unverified.

3.4.9 PLACE PREORDER

Use case	Current specification
ID / actor	UC-09 — Customer
Precondition	Account, valid fulfilment/contact, accepted price and required proof or delivery cash.
Main flow	Submit one request ID; server locks/rechecks current state and commits order plus allocations.
Alternative / exception	Any shortage, stale scope or later write error rolls back the entire placement; retry is idempotent.
Postcondition	Accepted snapshots, one ingredient/packaging deduction and saved order confirmation.

3.4.10 VIEW OWN ORDERS

Use case	Current specification
ID / actor	UC-10 — Customer
Precondition	Authenticated owner of order.
Main flow	Open My orders and inspect the saved items, total, date, fulfilment details and payment status. Contact the kitchen or use a permitted change request when needed.
Alternative / exception	Another account cannot access private history; ineligible changes are rejected.
Postcondition	Order information is displayed; any requested change is validated separately.

3.4.11 MANAGE CATALOGUE

Use case	Current specification
ID / actor	UC-11 — Owner/staff
Precondition	Staff session.
Main flow	Create/edit category/menu, price, unit contents, offered dates, image, recipe and selling status.
Alternative / exception	Invalid fields show form errors; protected historic relationships prevent unsafe deletion.
Postcondition	Current catalogue updates; accepted order names/prices remain unchanged.

3.4.12 RECEIVE OR ADJUST INGREDIENT BATCH

Use case	Current specification
ID / actor	UC-12 — Owner/staff
Precondition	Ingredient unit and actual received evidence available.
Main flow	Choose date basis, storage and handling; enter quantity; review deadline and save with a reason where needed.
Alternative / exception	Invalid evidence or stale edit version is rejected.
Postcondition	Dated batch and audit record; held/expired stock excluded.

3.4.13 CHECK A STOCK TARGET

Use case	Current specification
ID / actor	UC-13 — Owner/staff
Precondition	Current recipes and packaging configured.
Main flow	Choose a date and 1–100 units per offered menu; inspect totals, shortfalls and included menus.
Alternative / exception	Missing recipes/settings and no offered menus are reported distinctly.
Postcondition	Read-only target/worksheet; no receipt, reservation or deduction.

3.4.14 MANAGE PACKAGING

Use case	Current specification
ID / actor	UC-14 — Owner/staff
Precondition	Staff session.
Main flow	Create packaging, record received pieces/cost, assign per-menu pieces and review movements.
Alternative / exception	Invalid removal or missing supplies is rejected; removals require review.
Postcondition	Current pieces and menu mappings saved; old accepted allocations preserved.

3.4.15 RECORD VERIFIED PAYMENT

Use case	Current specification
ID / actor	UC-15 — Owner/staff
Precondition	Accepted order and bank/cash evidence.
Main flow	Inspect private proof; check external receipt; record full payment reference and outcome.
Alternative / exception	An upload alone is insufficient; duplicate/conflicting events are validated.
Postcondition	Payment event/state updated; food receipt unchanged.

3.4.16 CONFIRM ACTUAL FOOD RECEIPT

Use case	Current specification
ID / actor	UC-16 — Owner/staff
Precondition	Eligible current order(s); actual handover confirmed.
Main flow	Select one or more orders; review exact set and confirm food received.
Alternative / exception	Stale/ineligible selected order prevents an unreviewed partial update.
Postcondition	Actual receipt/audit state updated; no invented payment.

3.4.17 REVIEW DAILY SUMMARY AND PRINT

Use case	Current specification
ID / actor	UC-17 — Owner/staff
Precondition	Orders and a chosen handover date.
Main flow	Open daily menu totals; inspect orders; preview an individual slip or all matching slips for the day; explicitly print or save as PDF.
Alternative / exception	Undated records and missing slips are handled without invented dates.
Postcondition	Read-only report/preview with accepted name/address/phone.

3.4.18 REVIEW SALES AND PRICING

Use case	Current specification
ID / actor	UC-18 — Owner/staff
Precondition	Relevant recorded orders and known costs where required.
Main flow	Select period; distinguish operational value from paid/received sales; compare a price/bulk scenario.
Alternative / exception	Unknown costs remain unavailable; CSV ranges and formula text are bounded/sanitised.
Postcondition	Read-only report/export/scenario; live prices unchanged.

3.4.19 RECORD WASTE OR EXPENSE

Use case	Current specification
ID / actor	UC-19 — Owner/staff
Precondition	Batch quantity or overhead evidence.
Main flow	Review a waste deduction with reason, or record/void an eligible operating expense.
Alternative / exception	Do not double-count cooking deductions or ingredients as overhead; invalid quantities rejected.
Postcondition	Waste adjusts stock with cost evidence; expense ledger remains separate.

3.4.20 MANAGE KITCHEN SETTINGS

Use case	Current specification
ID / actor	UC-20 — Owner/staff
Precondition	Staff session.
Main flow	Save identity/story, area fees/hours, enabled collection, receiving QR and promotion section.
Alternative / exception	Invalid configuration is rejected; one section save preserves unrelated drafts.
Postcondition	New quotes use settings; accepted terms remain snapshots.

3.4.21 PROCESS OWNER REMINDER

Use case	Current specification
ID / actor	UC-21 — Scheduler / owner
Precondition	Authenticated worker and configured SMTP.
Main flow	Check due delivery/collection events and receipts; recheck under lock; attempt bounded sends.
Alternative / exception	Unauthorised calls rejected; failed sends do not save a success receipt.
Postcondition	Run metadata/accepted-send receipt only; order/payment/stock not changed.

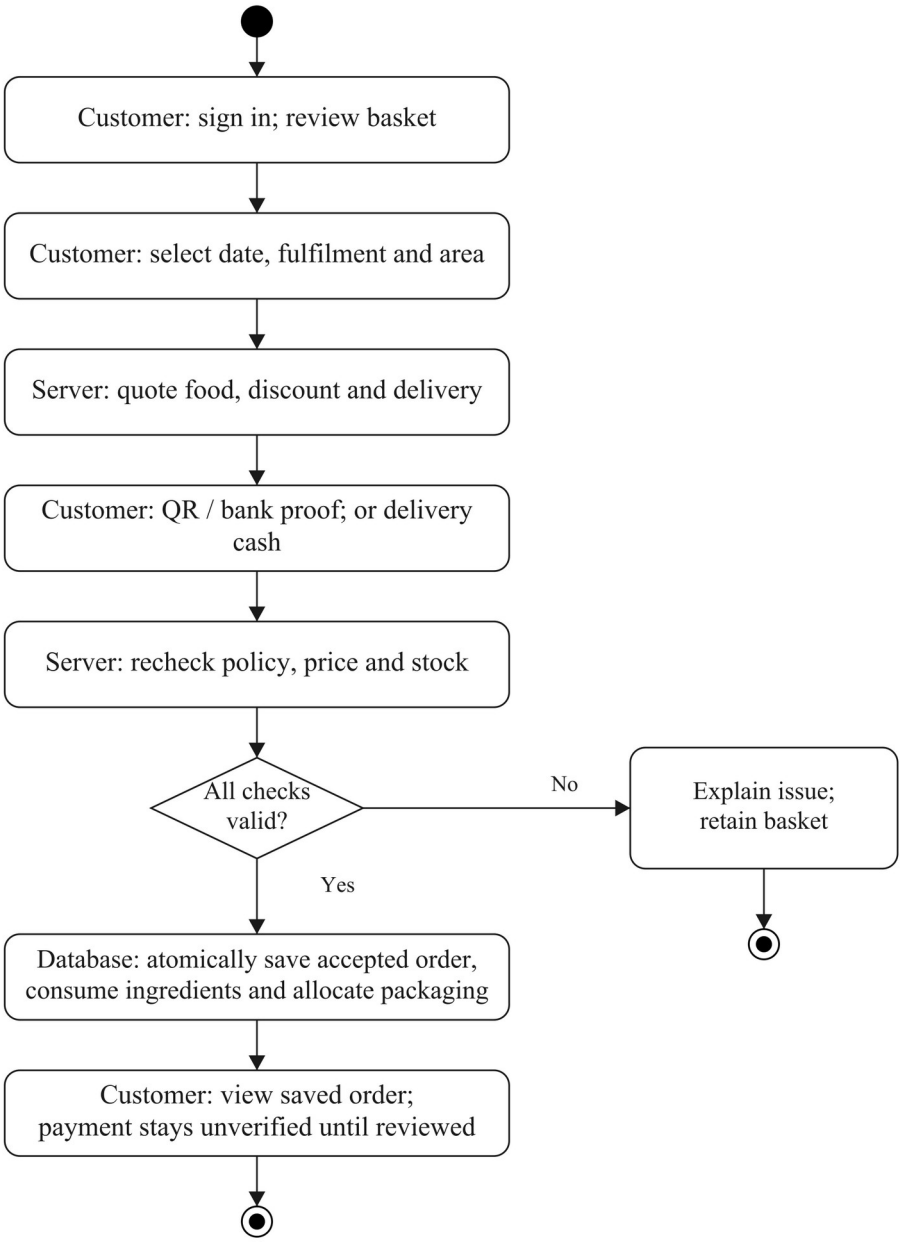
3.4.22 USE KITCHEN HELP AND FRESHCAST

Use case	Current specification
ID / actor	UC-22 — Customer / staff respectively
Precondition	Relevant public help or staff decision route available.
Main flow	Use supported help/contact guidance; staff select clearly labelled decision data/scenarios.
Alternative / exception	Service failure provides recovery; local, external and illustrative evidence are not mixed.
Postcondition	Assistance or hypothetical analysis; no automatic banking/courier/procurement action.

3.5 ACTIVITY DIAGRAMS

3.5.1 CUSTOMER CHECKOUT AND STOCK COMMITMENT

The customer selects delivery or available collection, an offered date and an allowed time. Delivery requires an area from the kitchen's configured list when that list is restricted. The server quotes food, bulk discount and delivery together. Delivery can use an available QR/bank-transfer method or cash. Collection requires a receiving QR and payment evidence. Placement repeats the relevant checks inside the order transaction and creates accepted terms before allocating stock. A failure must preserve the basket without a partial order or partial deduction.

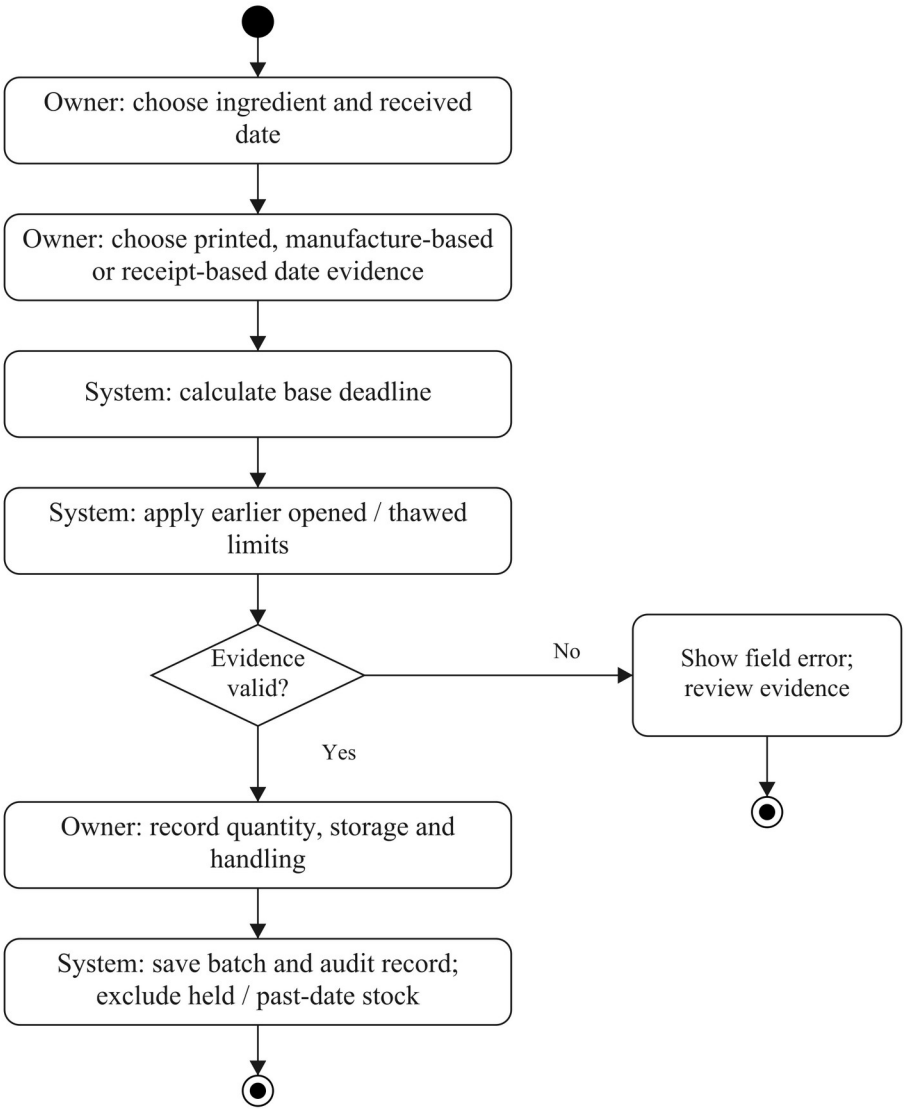


Collection requires QR proof. A stock failure rolls back the entire placement; a repeated request does not deduct twice.

Figure 3.3: Checkout validation and atomic ingredient/packaging allocation.

3.5.2 DATED INGREDIENT BATCHES

An ingredient definition supplies its inventory unit and optional documented storage guidance. A received batch records its own quantity, dates, actual storage and handling evidence. Printed-date, manufacture-plus-life and receipt-plus-storage-life modes use different anchors. Opening and thawing limits can shorten the base deadline. Uncertain handling or a known breach holds the batch. Selecting a wet-market source does not itself prove that the product has no usable supplier label or establish a safe duration.

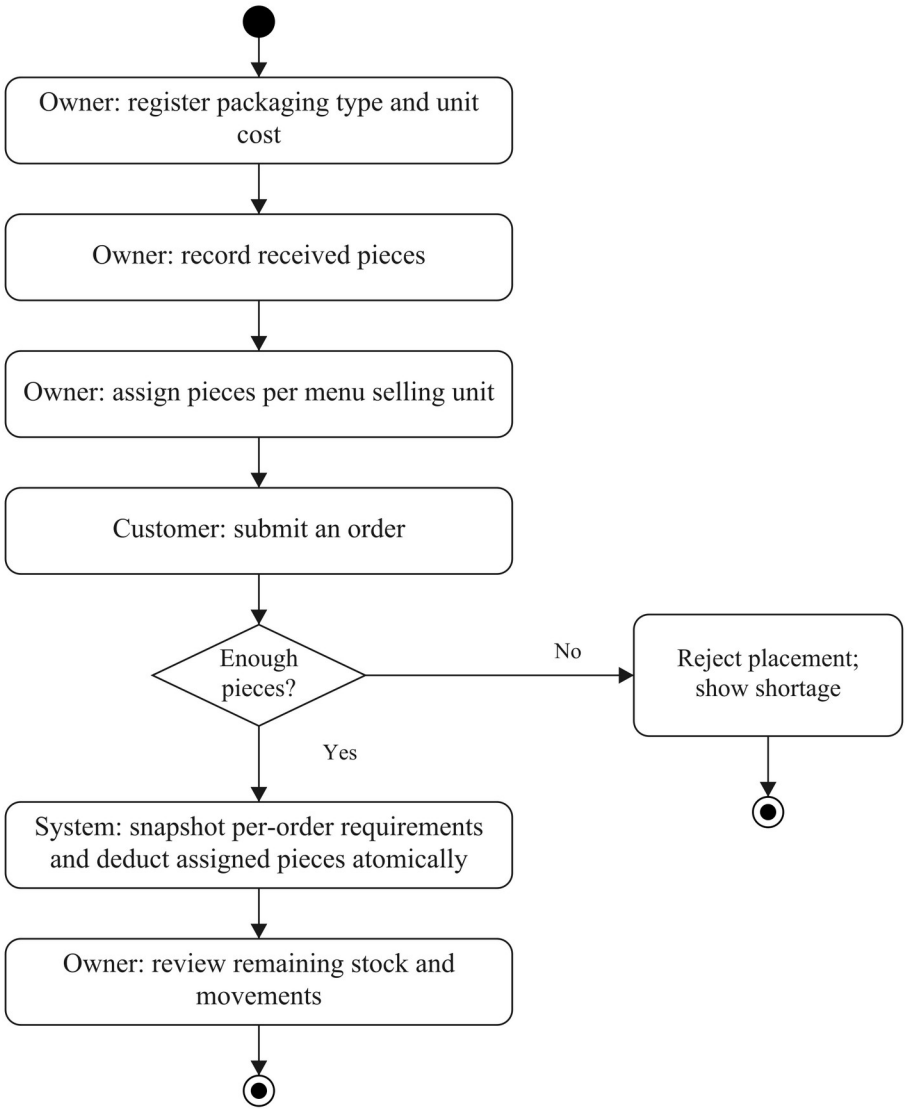


Documented shelf life is a calendar planning rule, not a physical freshness or food-safety measurement.

Figure 3.4: Batch date evidence, validation and stock eligibility.

3.5.3 PACKAGING STOCK

Packaging is tracked in whole pieces independently of edible ingredients. Each menu maps the pieces required for one selling unit. Placement multiplies these requirements by the ordered quantity and captures the accepted packaging names, quantities and costs. A later menu edit cannot rewrite that accepted allocation.

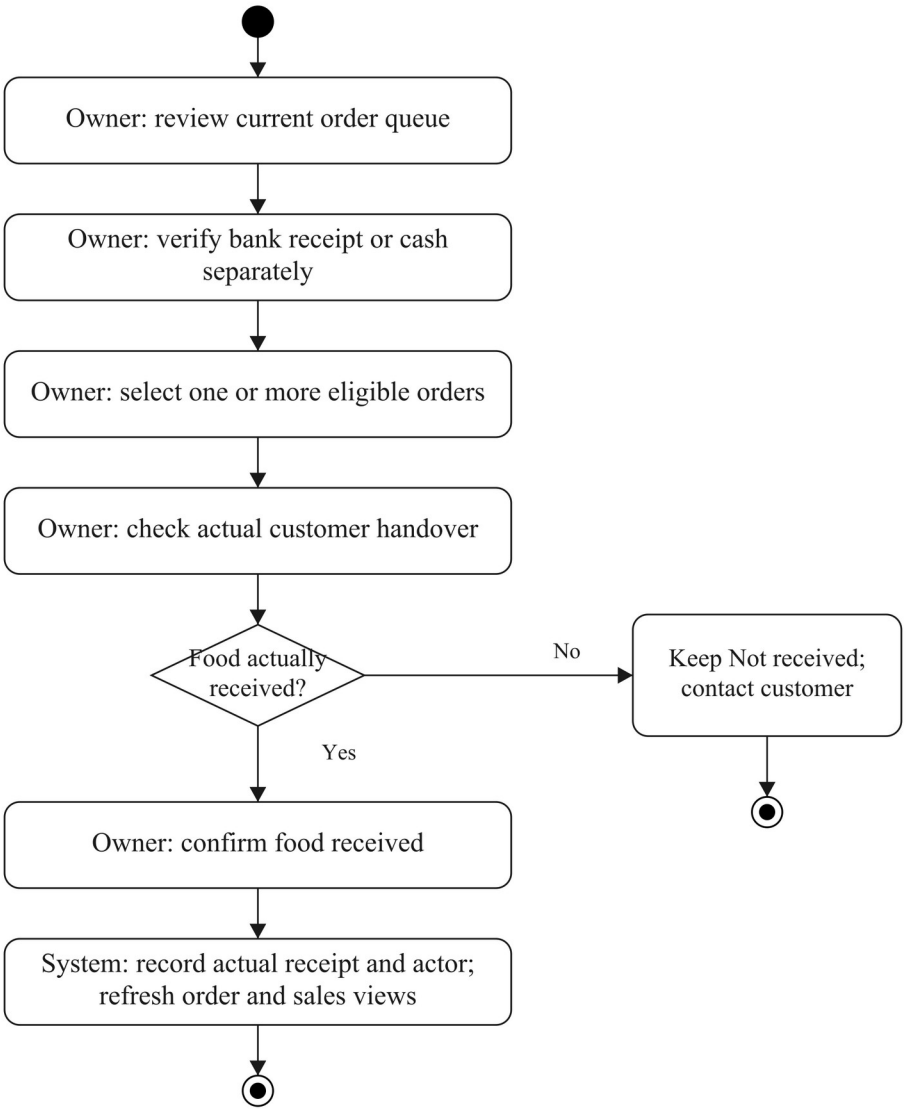


A low-stock warning does not create stock. Accepted packaging requirements survive later menu edits.

Figure 3.5: Packaging receipts, menu mappings and placement deductions.

3.5.4 ACTUAL FOOD RECEIPT

The owner confirms actual receipt only when the customer has received or collected the food. Several eligible orders can be selected together, but the selected set and current versions must be rechecked. A planned time, bank receipt, reminder or timer cannot establish that the food was received. Corrections and cancellation use their own audited operations.



The clock and email worker never mark food received. Receipt of food and receipt of money are separate facts.

Figure 3.6: Separate payment review and actual customer receipt.

3.6 SEQUENCE DIAGRAMS

3.6.1 CUSTOMER PASSKEY ACCESS

Registration and authentication use session-bound, short-lived WebAuthn challenges. The device obtains user verification and signs a response; Django verifies the relevant challenge, origin, relying party and credential data. The server stores a public key, credential identifier and account association. It does not receive the customer's fingerprint, face template, private key or device PIN. A matching phone number is not used to take over an existing account.

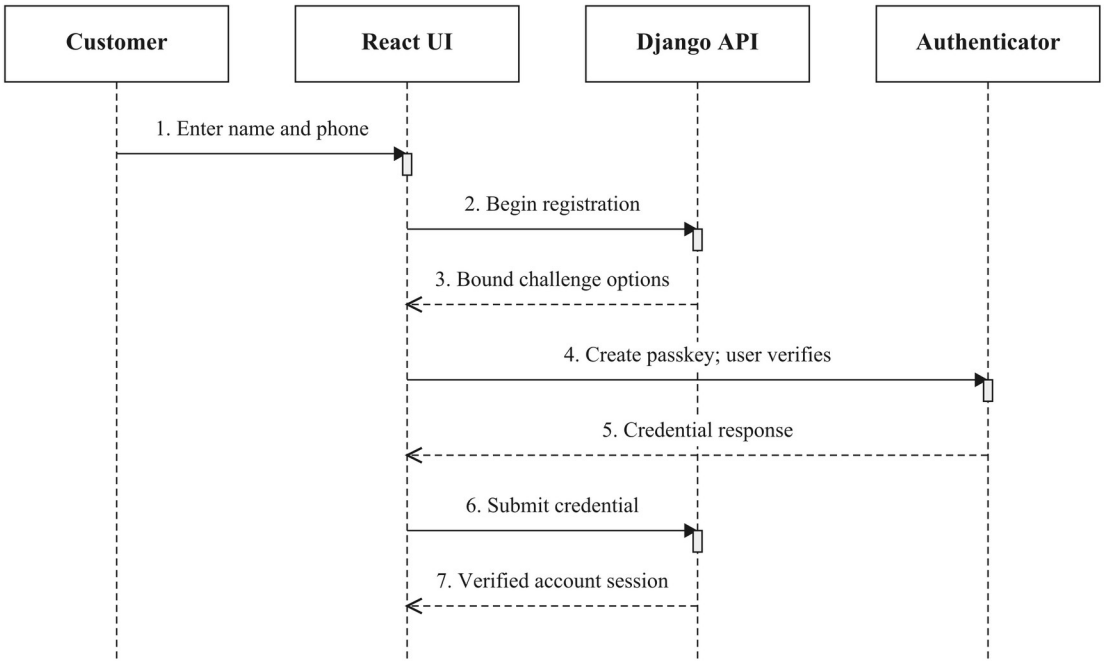


Figure 3.7: Customer passkey registration.

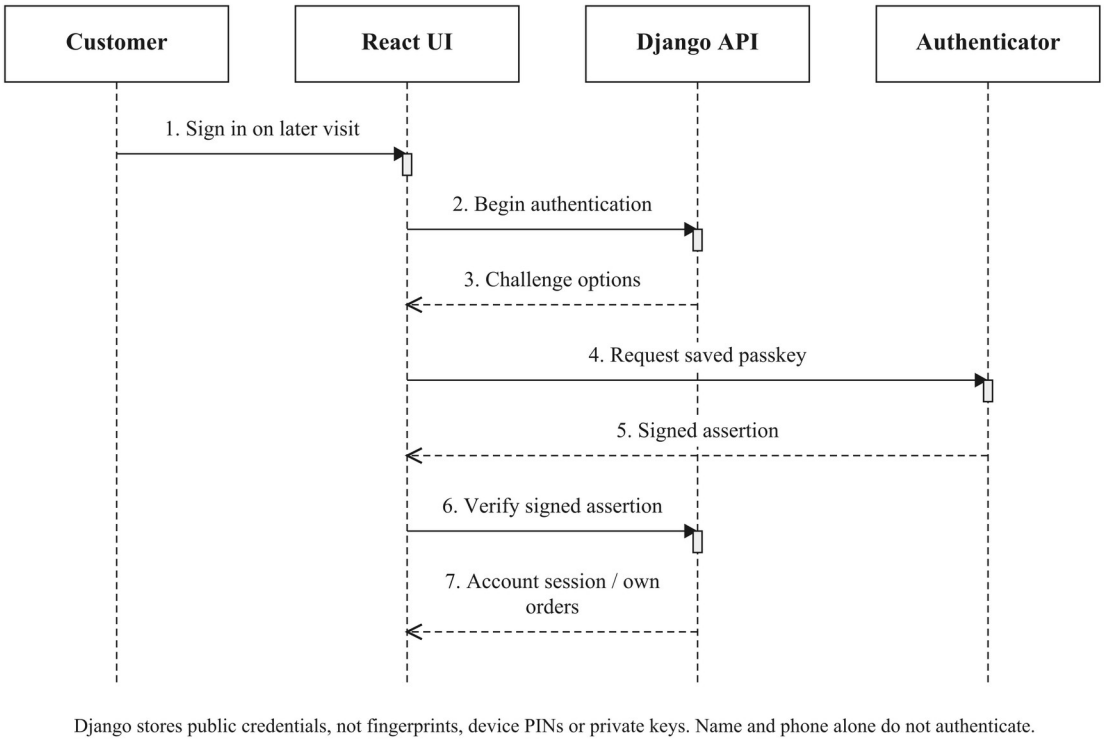


Figure 3.8: Authentication using a saved customer passkey.

3.6.2 QUOTE, PRIVATE PROOF AND ORDER PLACEMENT

The quote is advisory until placement succeeds. Its pricing uses the current basket and fulfilment selection. A receipt upload is authenticated, sanitised and bound to a price/selection scope. Changing the basket, recipient configuration or relevant

fulfilment details can require the existing receipt to be reviewed or uploaded again; it must not prompt a second bank payment automatically. Placement uses a request identifier and locks current state so replay does not allocate ingredients or packaging twice.

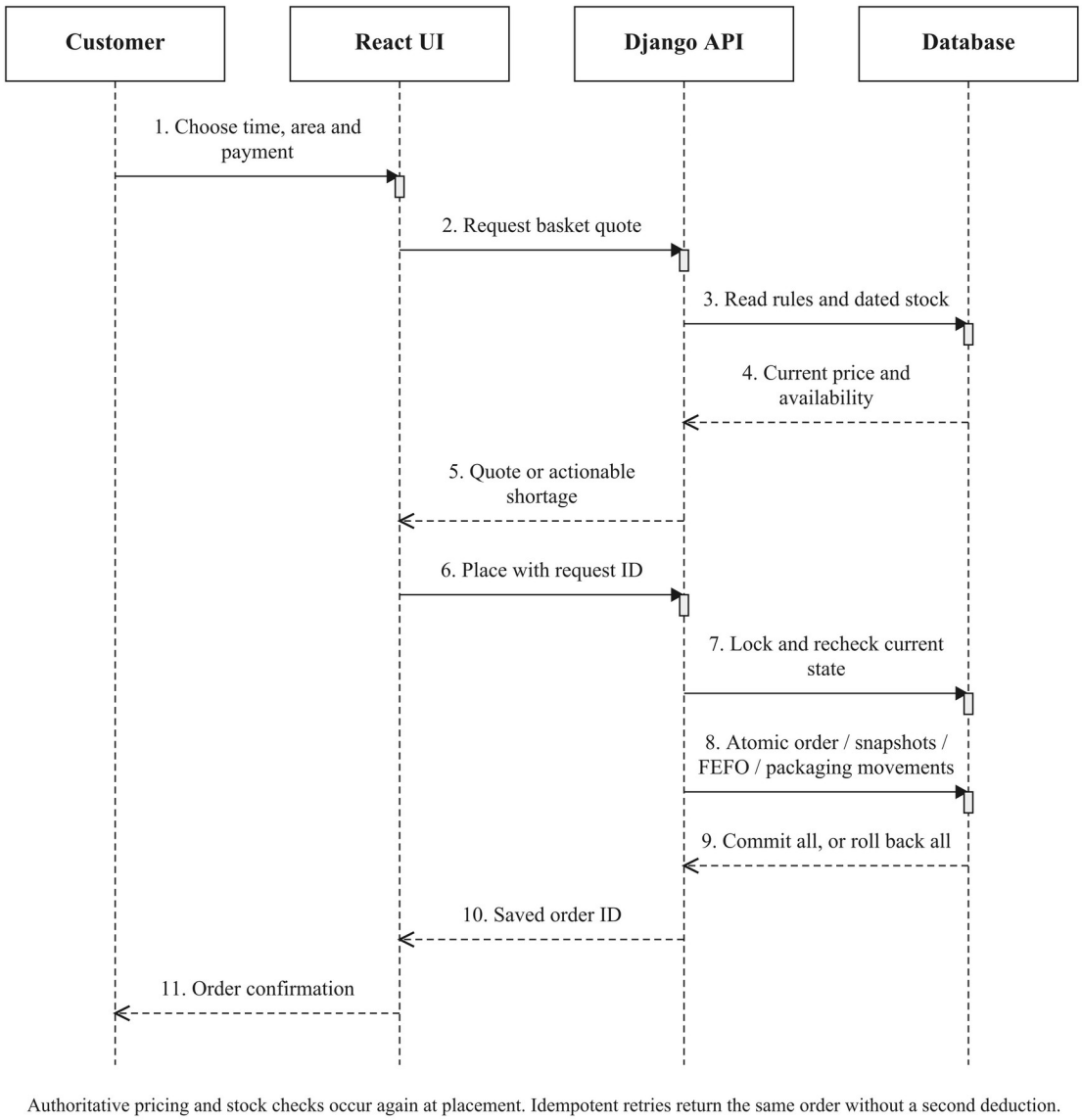


Figure 3.9: Authoritative pricing, scoped evidence and transactional placement.

3.6.3 OWNER PAYMENT VERIFICATION

The kitchen reviews the uploaded image and its bank records outside the website. Recording a completed full receipt creates a payment event and updates the payment state under server validation. A refund record describes money already returned externally; it is not a bank-transfer instruction. The payment operation does not establish physical food receipt.

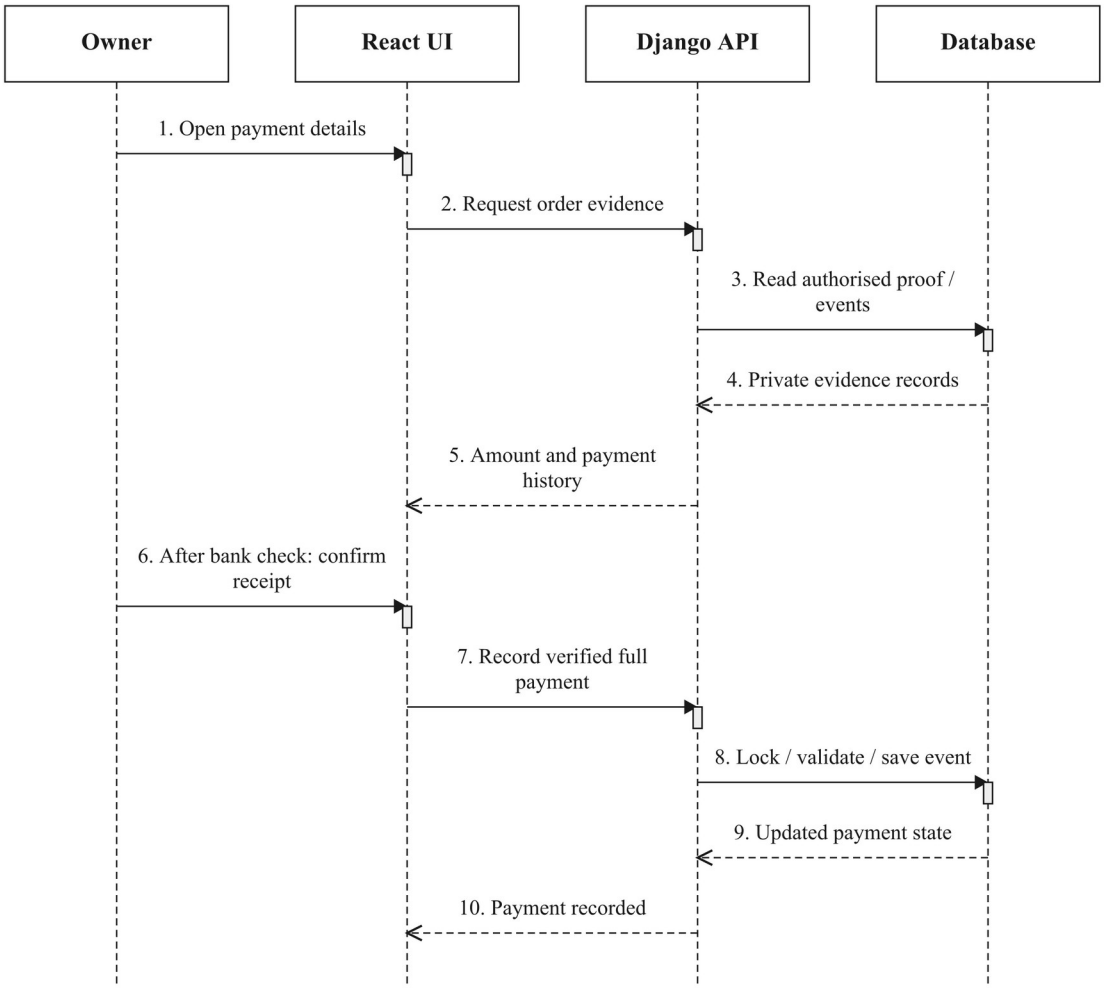


Figure 3.10: Private evidence review and manual payment recording.

3.6.4 REPORTING AND PRINT PREVIEW

Daily reports aggregate accepted line names and quantities for the selected delivery or collection date. Sales reporting applies its own paid-and-received inclusion rules. The owner can preview an individual slip or all matching slips for a day and print them through one browser print action. Each slip uses the accepted customer, phone, address and handover details. Viewing or printing does not change the order.

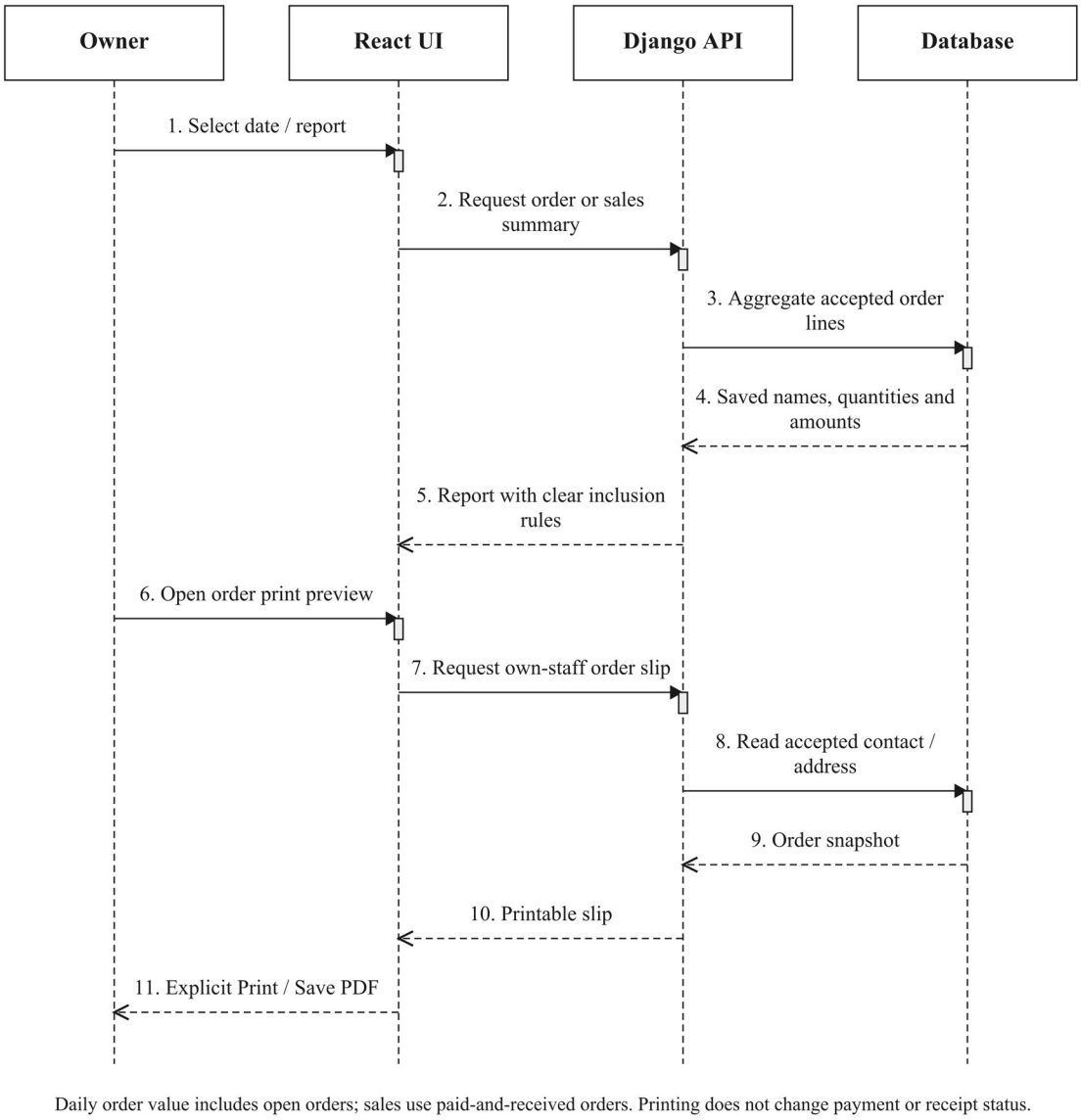


Figure 3.11: Daily/sales reporting and read-only order-slip preview.

3.6.5 OWNER HANDOVER REMINDERS

One authenticated worker checks eligible reminder events. In simple mode, the current owner queue concerns delivery/collection rather than preparation. Successful provider acceptance is followed by a saved reminder receipt; failed attempts can be retried. The worker has a bounded attempt budget. Provider acceptance is not proof of inbox receipt, and a crash between acceptance and saving the receipt can still cause a duplicate on retry.

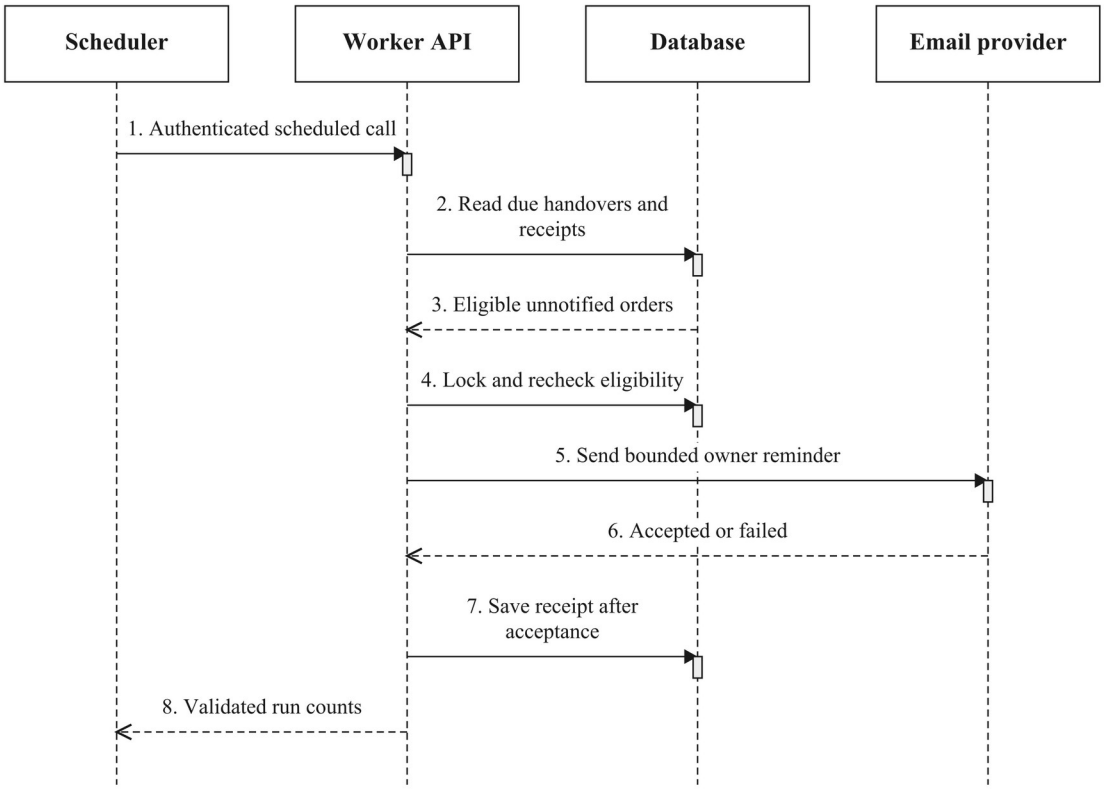
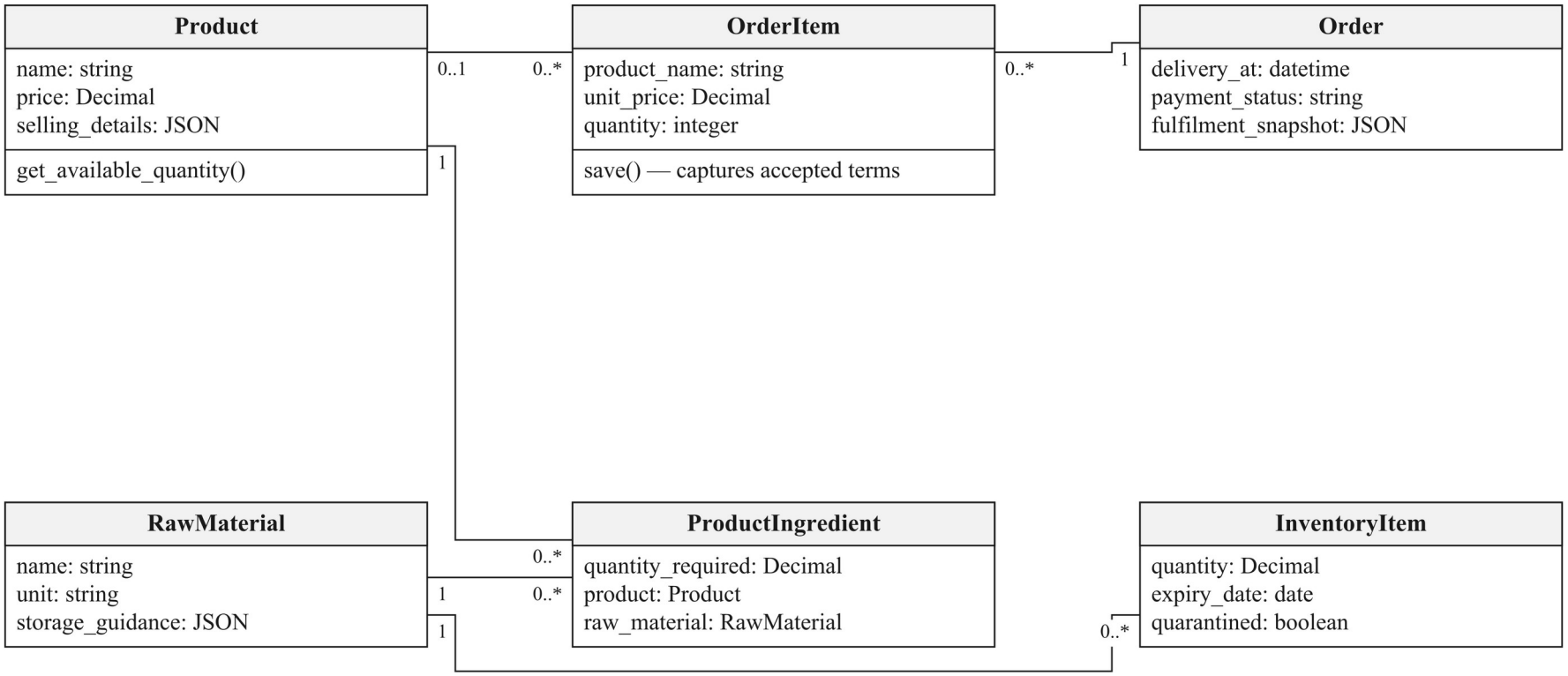


Figure 3.12: Authenticated, bounded owner handover reminder processing.

3.7 CLASS DIAGRAM

Figure 3.13 shows a selected core of the implemented Django domain, rather than inventing a separate class for every screen. Product represents the current menu, while OrderItem records accepted purchase terms. ProductIngredient connects current recipes to RawMaterial; InventoryItem records dated lots. The full application contains additional account, packaging, payment and operational models described in the ERDs and Appendix D. Methods shown correspond to the actual implementation.



Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

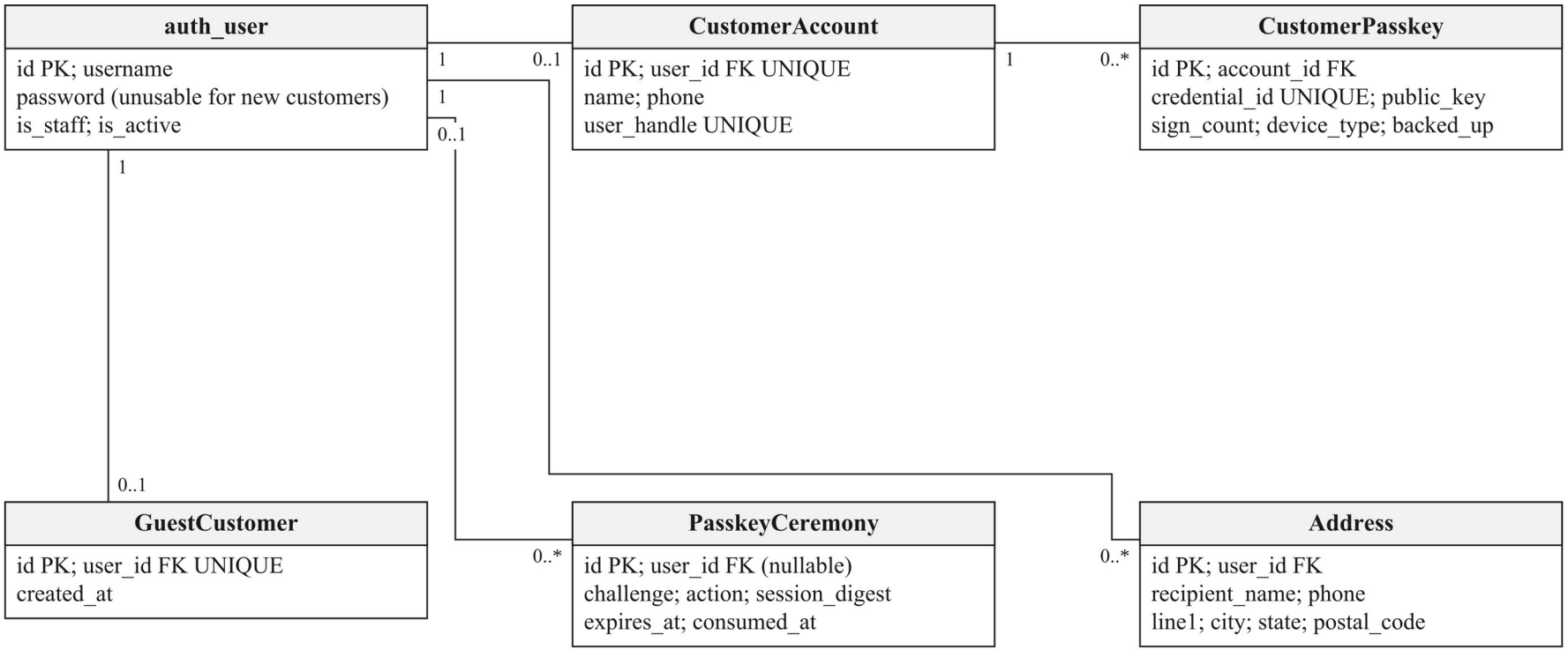
Figure 3.13: Selected core domain classes and their associations.

3.8 ENTITY–RELATIONSHIP DIAGRAMS

The ERDs show selected attributes and actual relationship columns. Multiplicities apply at the indicated entity ends. Database fields for earlier scheduling, wallet and external-payment experiments still exist for compatibility. Their presence does not imply that those methods are available in the current client checkout. Appendix D distinguishes current and supporting/legacy models.

3.8.1 IDENTITY AND ACCOUNT ACCESS

Django's auth_user remains the underlying identity table. A browsing session can use an opaque GuestCustomer identity with an unusable password. A registered customer has a CustomerAccount and one or more public CustomerPasskey credentials. PasskeyCeremony binds a short-lived action to its session and, where applicable, user. Staff continue to use the protected staff credential flow. Address is retained as a supporting/legacy address entity, while current orders preserve accepted contact/address snapshots.

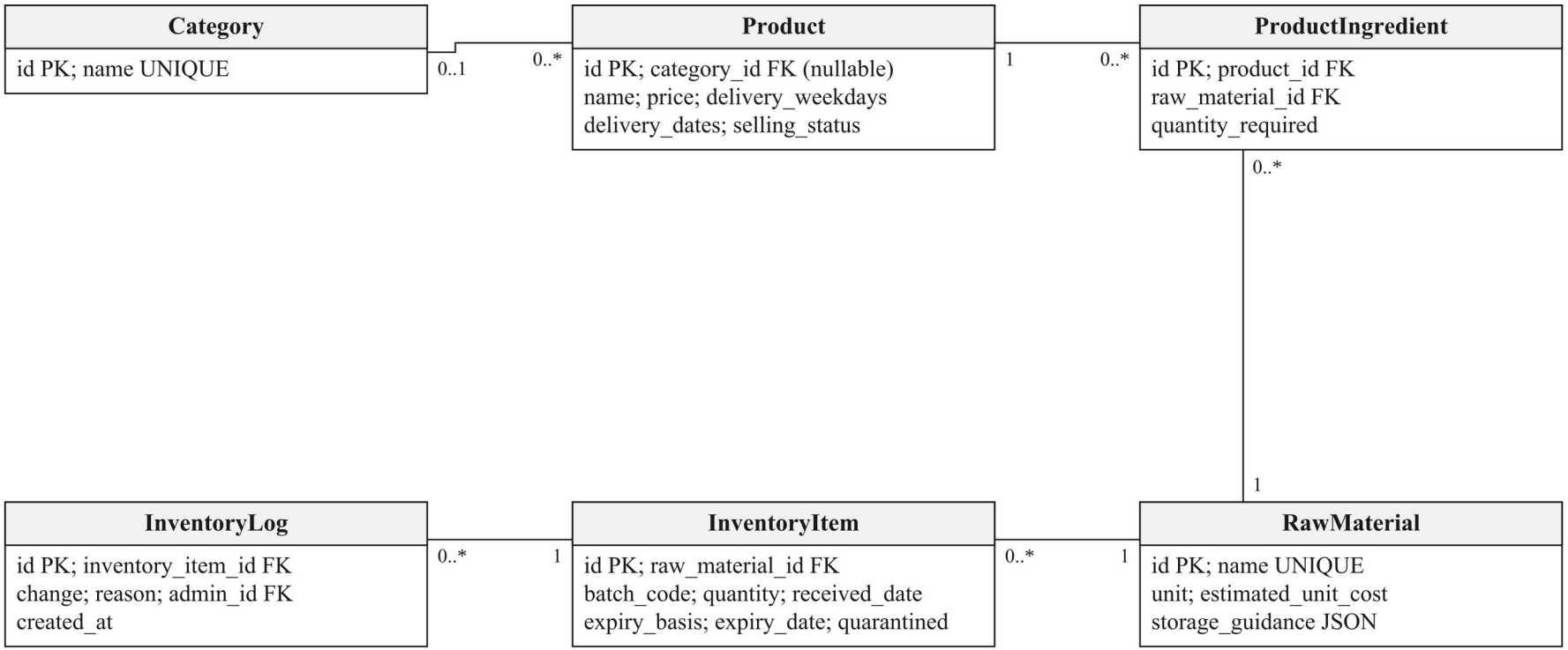


Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

Figure 3.14: Identity, customer profile and passkey records.

3.8.2 CATALOGUE, RECIPES AND INVENTORY

A category contains current menus. A recipe joins a menu and ingredient with a positive quantity in that ingredient's inventory unit. A material can have many dated batches and each batch can have many inventory log entries. Batch quantity and date eligibility are authoritative for the stock check; a displayed stock-supported menu quantity is derived, not a second manually maintained physical count.

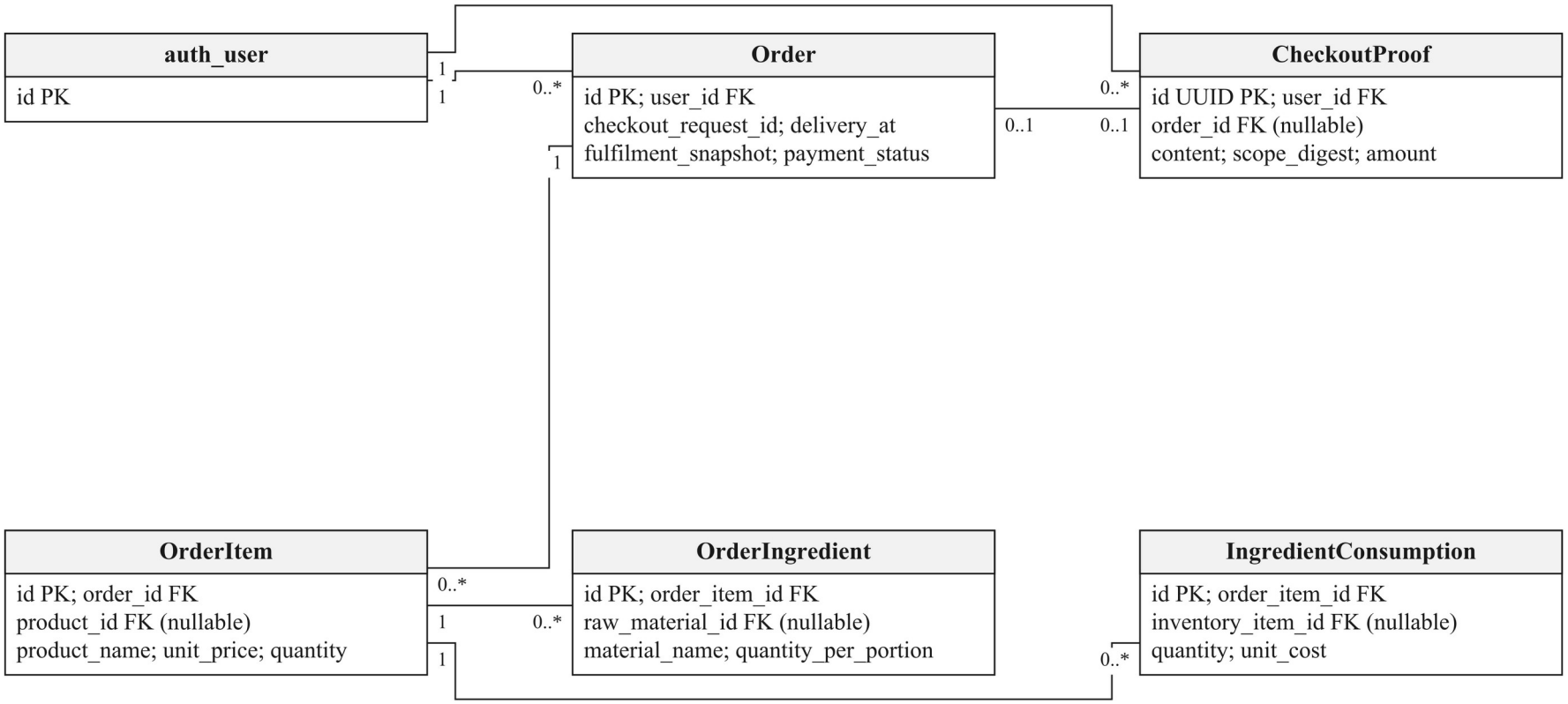


Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

Figure 3.15: Catalogue, recipe, batch and inventory-log relationships.

3.8.3 ACCEPTED ORDERS AND CONSUMPTION

OrderItem has both a nullable Product foreign key and its own product_name, unit_price and quantity. History displays the saved name and price, so a menu rename does not rewrite a previous purchase. OrderIngredient preserves the accepted recipe requirement. IngredientConsumption records actual batch allocations against order items, including quantity, cost and recorded expiry evidence. CheckoutProof is private evidence belonging to its user and, after acceptance, its associated order.

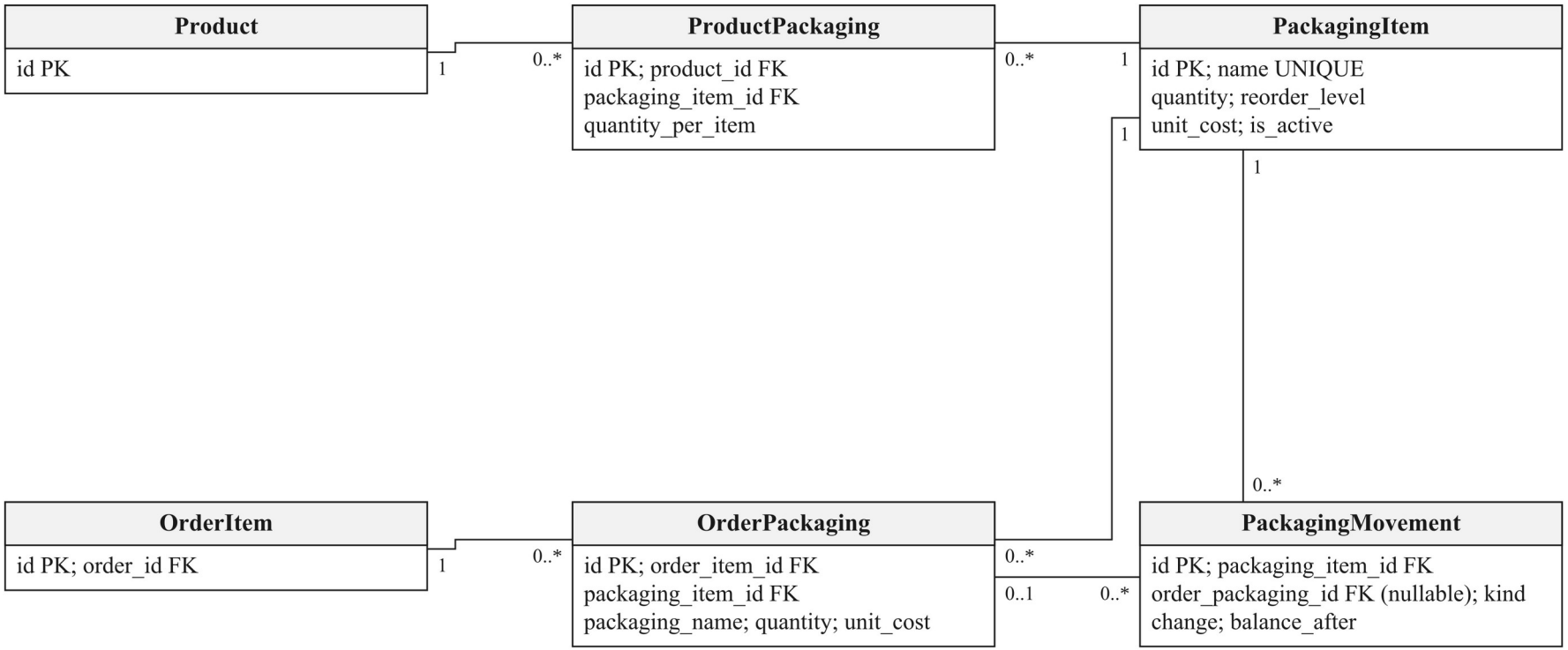


Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

Figure 3.16: Accepted order terms, private proof and ingredient allocation.

3.8.4 PACKAGING RELATIONSHIPS

ProductPackaging defines the current per-menu requirement. OrderPackaging preserves the accepted per-order-line requirement, including the packaging name and cost where known. PackagingMovement records stock changes and can refer to an accepted packaging allocation. Names and quantities are deliberately snapshotted so later catalogue edits do not alter history.

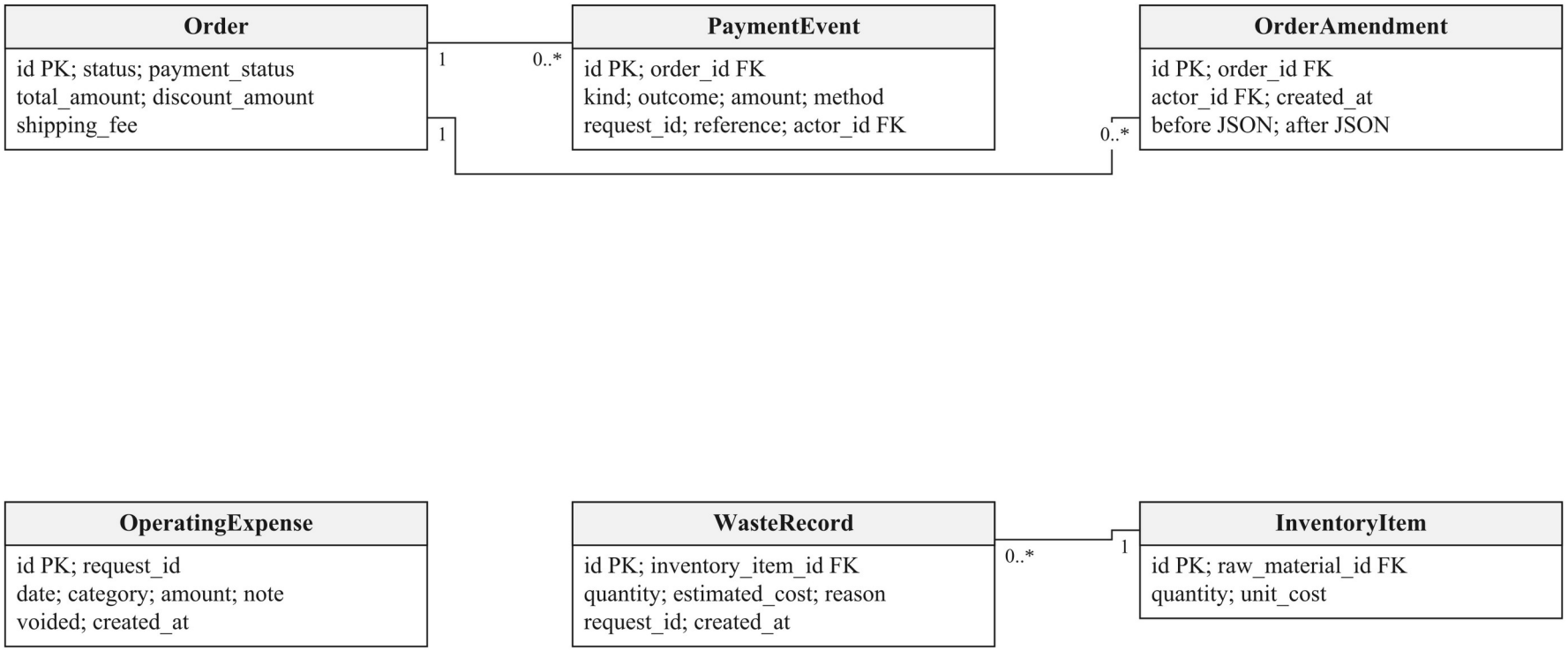


Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

Figure 3.17: Current and accepted packaging requirements and stock movements.

3.8.5 FINANCIAL AND AUDIT RECORDS

PaymentEvent stores completed/pending/failed receipt or refund evidence. OrderAmendment stores the actor and before/after documents for relevant corrections. OperatingExpense records overhead rather than ingredient purchases. WasteRecord associates discarded quantities and estimated cost with an inventory batch. These records support an audit trail; they do not replace bank reconciliation or establish net profit.



Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

Figure 3.18: Payment, amendment, expense and wastage evidence.

3.9 REQUIREMENT TRACEABILITY AND DESIGN LIMITS

The first objective is realised through permitted future handover dates, editable delivery/collection hours and date-dependent menu checks. It does not require automatic cooking-task allocation after the client removed that workflow. The second objective is implemented through recorded shelf-life evidence and storage duration; it is not a sensor-based food-safety claim. The single-vendor storefront supports the third objective. Sales and cost evidence support the fourth, and the two recommendation surfaces implement the fifth. Whether recommendations reduce decision time or increase conversion must be evaluated with users rather than inferred from successful test requests.

The design intentionally leaves transfers, rider booking and physical handover outside automatic verification. It provides clear records and contact routes around those activities. Supported browser/device access, honest stock records, correct ingredient quantities and owner configuration remain prerequisites. Chapter 5 identifies the checks performed and the empirical claims that remain unmeasured.

3.10 CONCLUSION

The revised design reflects the client's simpler ordering model while keeping the original project objectives. Accepted-order snapshots protect historical terms, and the placement transaction coordinates ingredient and packaging records. The next chapter explains the implemented interfaces, calculations and development decisions using this design.

CHAPTER 4

INTERFACE AND SYSTEM IMPLEMENTATION

4.1 INTRODUCTION

This chapter describes the implementation of the Web-Based Preorder and Freshness Management System for Wrap & Bowl Kuch. Customers use the website to browse food, choose an order date and submit a preorder. The owner uses the management pages to maintain menus, ingredients and packaging, review payments, and manage orders and sales.

Client feedback helped to refine the original design. The client preferred a simple morning ordering process, with fewer account details and no detailed preparation planner. The implemented workflow therefore uses name and phone details with a passkey, delivery fees for selected areas, payment-receipt uploads and a simplified food-receipt status.

The interface figures were captured on 9 November 2026 from the tested application running locally with example orders and stock and the client's supplied menu photos. They illustrate the implemented screens rather than the client's actual sales or inventory balances. Desktop captures use a laptop-sized browser viewport. Payment receipts are illustrative and do not represent bank transfers.

4.2 CUSTOMER INTERFACES

4.2.1 BUSINESS INFORMATION AND MENU

The public pages introduce Wrap & Bowl Kuch, a home-based food business in Kuching, Sarawak. The business story uses the introduction supplied by Ayu Shafinaz, who described how her interest in cooking developed into a small business. The owner can edit the introduction, contact details and business story through Settings.

The menu contains salad bowls, Wanpaku sandwiches and a rice bowl. Customers can search by name or choose a category. Each card shows the food name, selling unit and price. For example, one sandwich box is one selling unit even though the sandwich is cut into two halves. This distinction helps customers understand what they are buying.

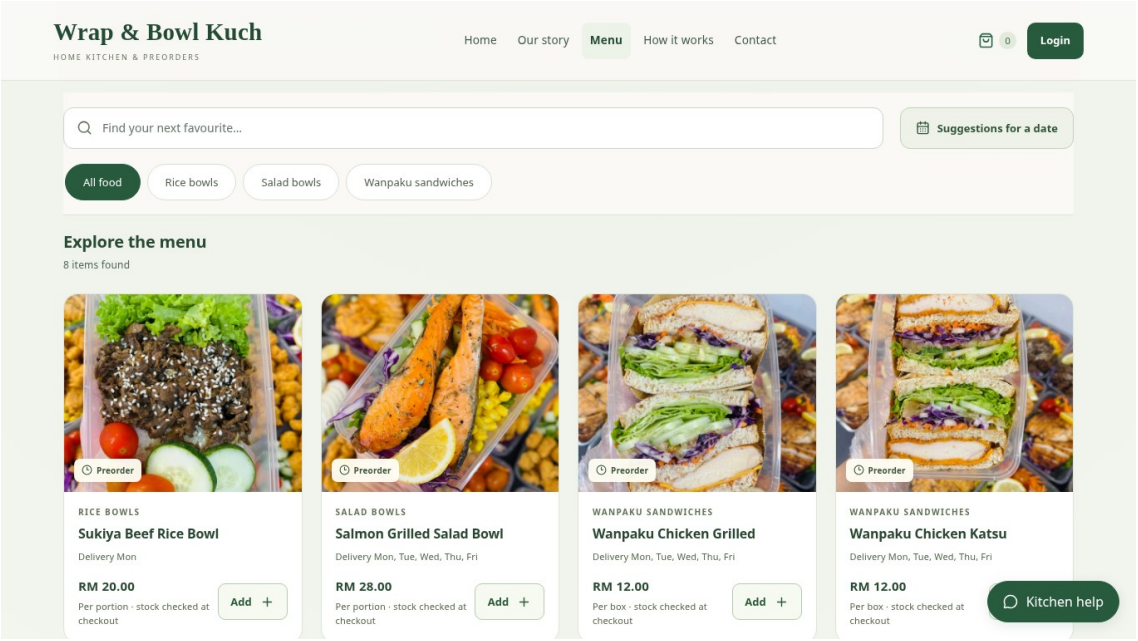


Figure 4.1: Menu browsing with category choices, prices and food cards.

The client catalogue contains eight menus. Seven are offered from Monday to Friday, while Sukiya Beef Rice Bowl is offered on Monday only. Offered days and prices are editable. The menu uses the client's supplied food photographs. The owner can replace a menu image by uploading a file or entering an image URL.

When an item has been added, its quantity control allows the customer to increase or reduce the basket quantity. A short success message confirms the action. The basket shortcut remains available while the basket contains food, allowing customers to continue browsing before reviewing their order.

4.2.2 CUSTOMER ACCOUNT AND SIGN-IN

Customers can browse, add food and review their basket without registering. Selecting Continue to checkout opens the Login modal over the basket. New customers can choose Register, enter their name and full phone number, and create a passkey through a supported device. Registration keeps the guest basket. Returning customers use their saved passkey, which loads that account's basket and orders.

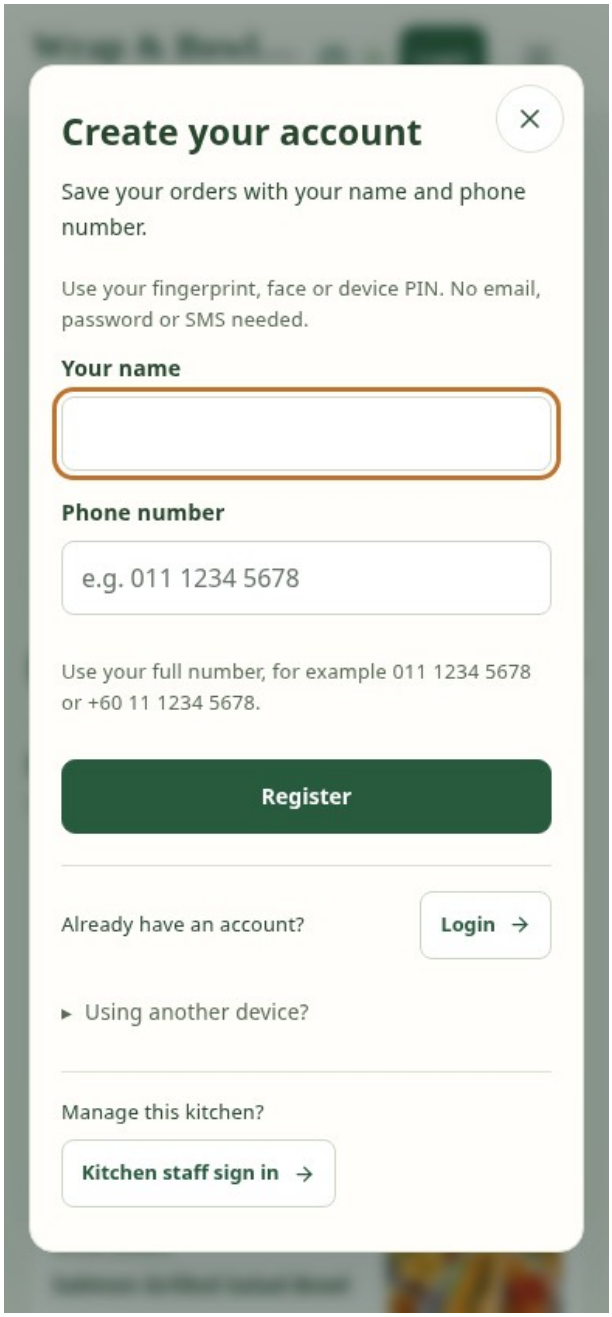


Figure 4.2: Customer account creation with name, phone number and a passkey.

The phone number is used for order coordination. It is not verified by SMS and does not replace the passkey as proof of account ownership. Returning customers need access to their saved passkey. They can add a backup passkey through Account Settings. This is a practical limitation when changing devices or losing access to a passkey manager.

Kitchen staff use a separate sign-in form with their username or email and password. Staff and customer forms use a consistent modal layout, with a route back to the customer account form. Staff password recovery is available in the same modal. Customer accounts cannot access the owner's management pages.

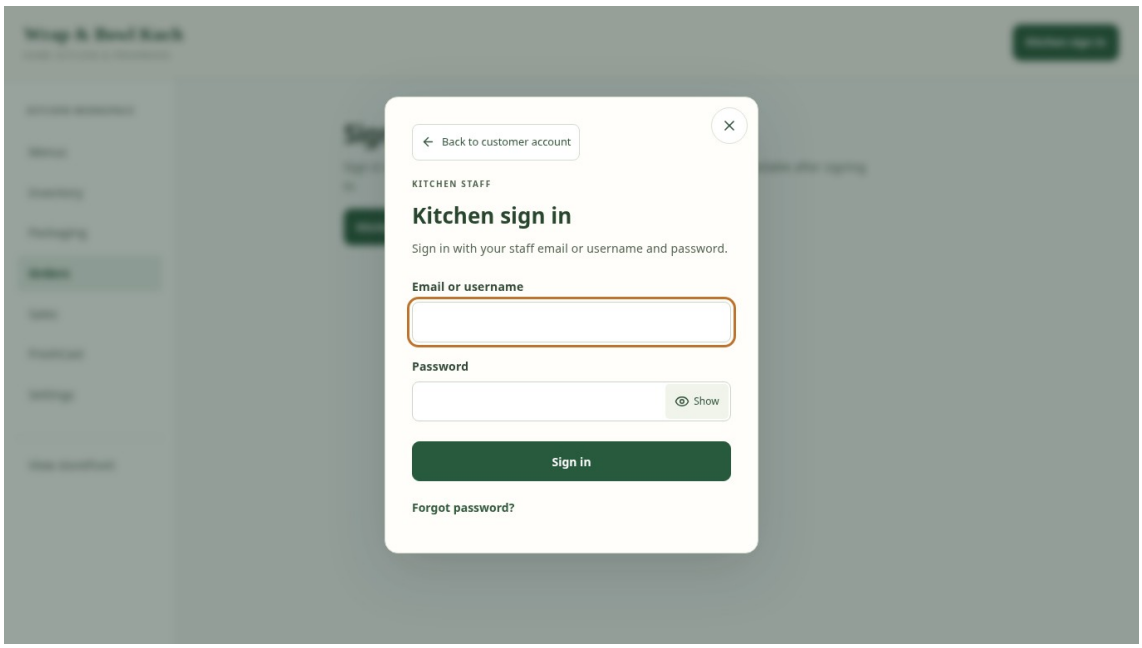


Figure 4.3: Separate sign-in form for kitchen staff.

4.2.3 MENU RECOMMENDATIONS

The menu provides two ways to discover food. Recommended for you gives up to three suggestions using recorded purchase preferences, categories and recent popularity. The customer does not need to enter a date or budget to use this browsing feature. The suggestions are alternatives, rather than a combined order.

Suggestions for a date considers the customer's intended date, total number of items and optional whole-order budget. Customers can allow a mix of menus or request one menu only. In the one-menu mode, each result is an alternative containing the requested quantity of that menu. For example, three RM12 portions have a food total of RM36; RM36 is not the price of each portion.

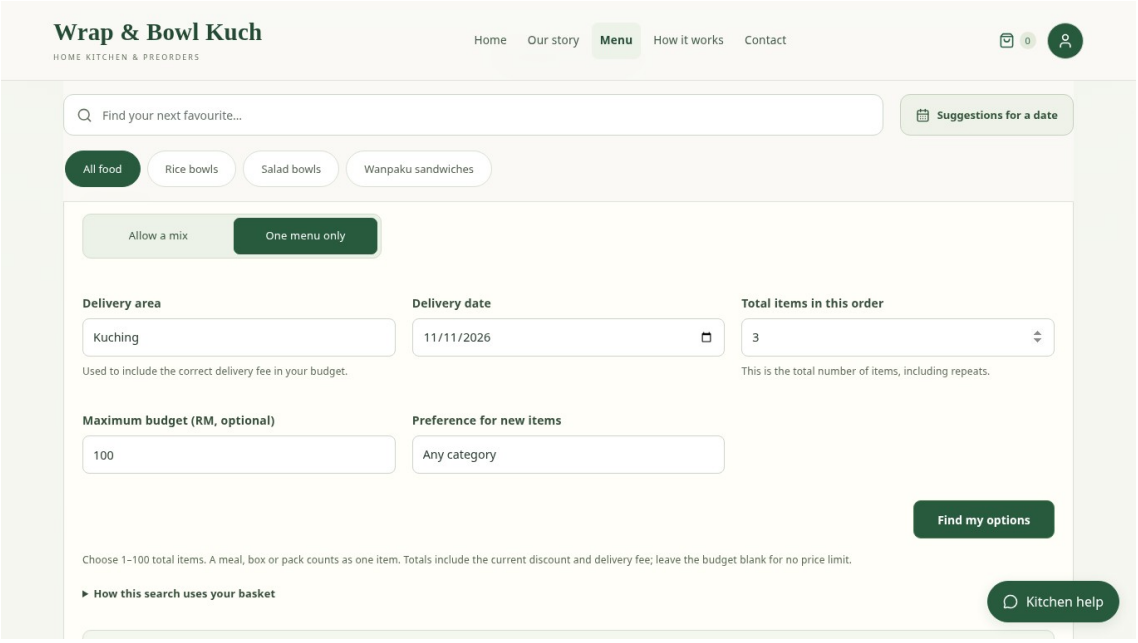


Figure 4.4: Date-based finder with quantity, budget and menu-choice controls.

The optional budget is a maximum for the complete order, including the applicable food discount and delivery charge. A total equal to the budget is allowed. Leaving the budget blank removes the price limit. The finder also checks the menus' offered dates and recorded stock for the selected day.

A mixed search keeps existing basket items and suggests additions up to the requested total quantity. A menu may appear more than once. If the basket already contains the requested number of items, it may be the only complete option. One-menu searches offer repeated units of one menu and ask for confirmation before replacing the basket. Selecting a result does not reserve stock; checkout checks the complete basket again.

4.2.4 BASKET REVIEW

The basket shows selected food, quantities, item details and the food subtotal. Customers can change quantities, remove food or return to the menu. Menu details explain the contents of a box or portion and the days on which it is offered.

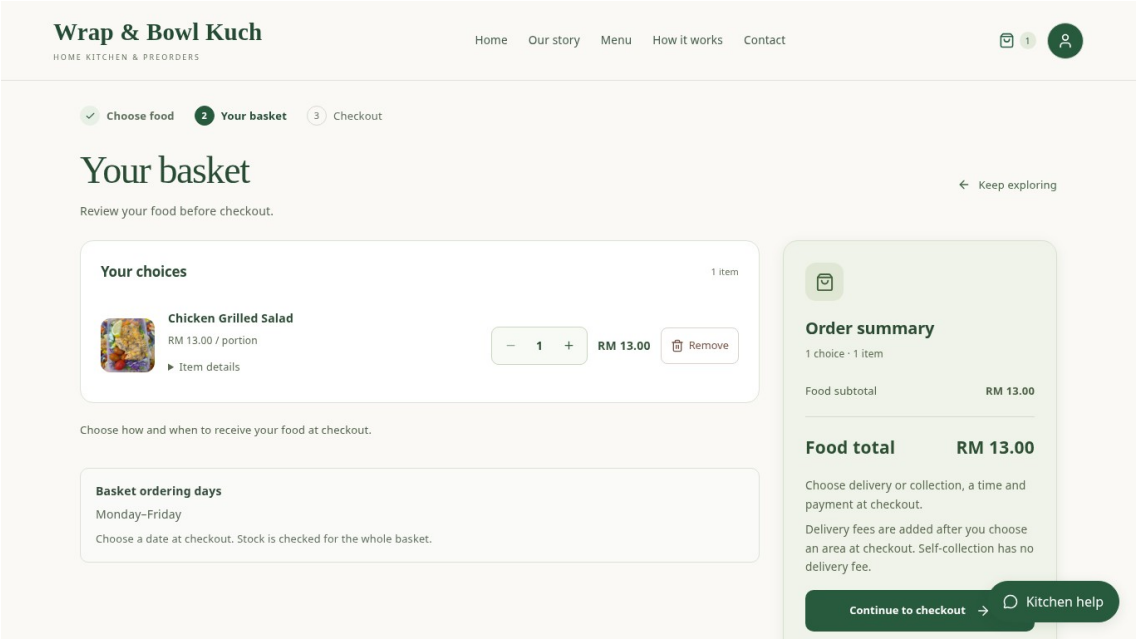


Figure 4.5: Basket review before entering delivery and payment details.

The basket also summarises the ordering days shared by its items. For example, a basket containing a Monday-only rice bowl must use Monday even if its other menus are offered throughout the working week. Delivery area and time are entered at checkout, avoiding the need to fill in the same information on two pages.

4.2.5 CHECKOUT AND PAYMENT

Checkout brings together fulfilment, contact details, the order date and payment. Delivery customers choose a saved area, which determines the delivery fee. Self-collection appears when the owner has enabled it in Settings. A collection address alone does not turn on the option.

The client requested morning delivery between 7:30 am and 11:00 am. Collection has its own configurable hours. The date controls show the basket's offered days and upcoming date choices so that customers can select a suitable day without remembering every menu's schedule. Ingredient and packaging availability are checked for the selected date.

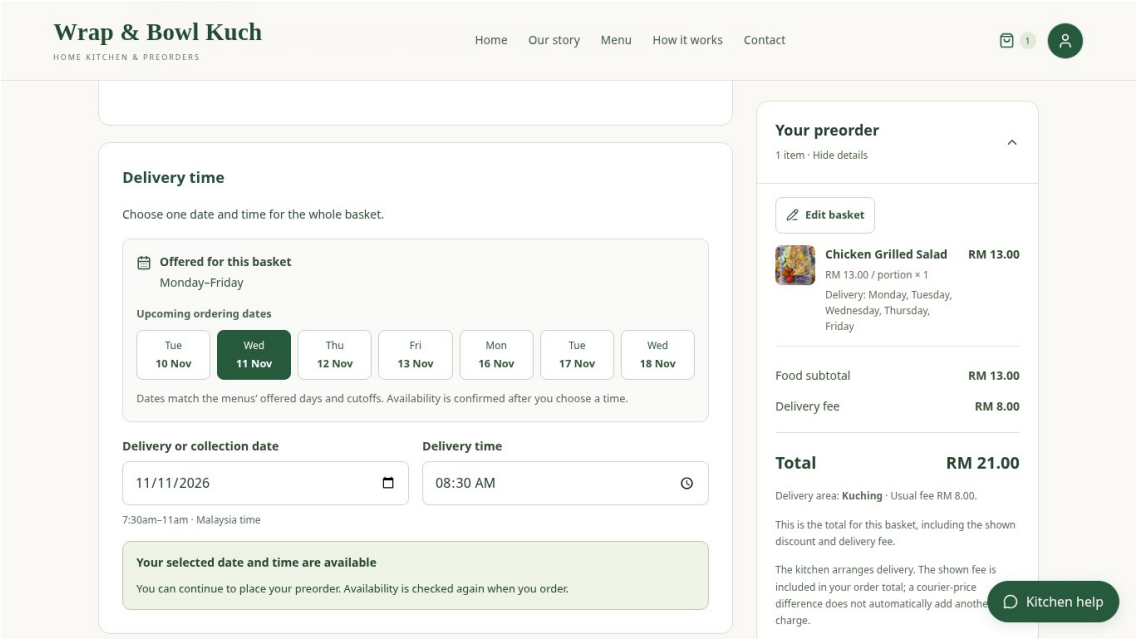


Figure 4.6: Checkout date guidance, morning time and stock-availability result.

An online preorder is accepted only when recorded ingredient and packaging stock covers the chosen date. Stock that will expire before delivery cannot be used for that order. The system does not assume that the kitchen will buy replacements later. When availability cannot be confirmed, customers see a short explanation and options to change the date, edit the basket or contact the kitchen.

Saved name and phone details are filled in when available. Customers can correct these details during checkout and choose to save them for future orders. A full delivery address is required for delivery. Optional handover instructions allow the customer to explain access or contact arrangements.

Delivery permits bank QR or bank-transfer payment with an uploaded receipt, or cash. Collection permits QR payment with a receipt. The receiving image and payment instructions are maintained by the owner. Customers make the actual transfer in their banking application, then upload a supported receipt image before placing the order.

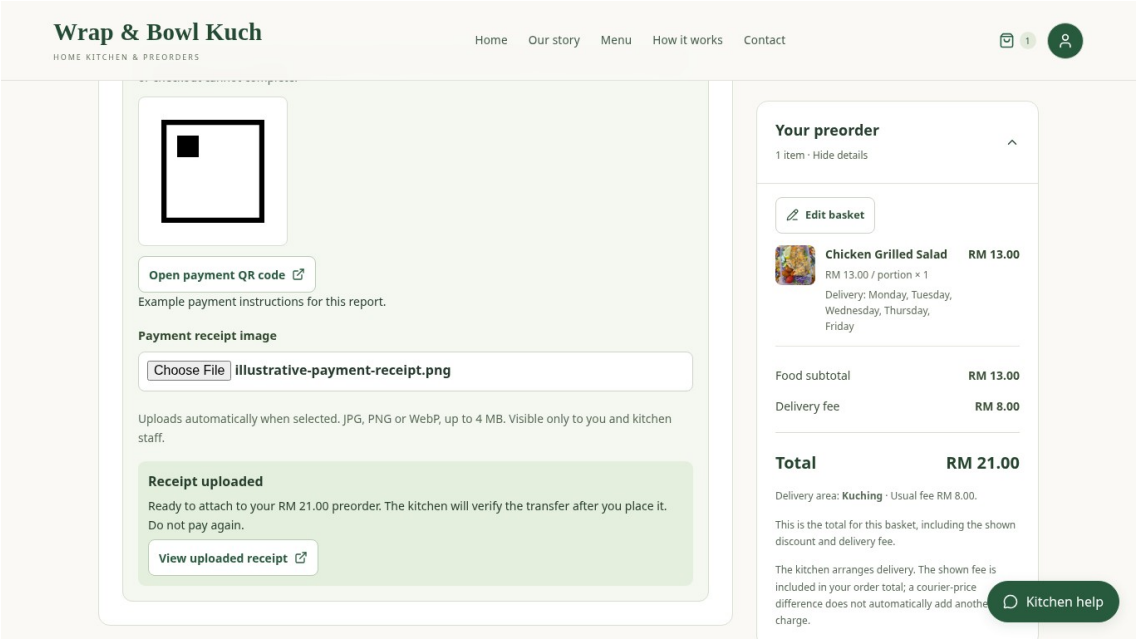


Figure 4.7: Payment section with an uploaded illustrative receipt.

Uploading a receipt does not prove that money has reached the bank account. The order remains subject to the owner's payment review. The server checks the current total, selected date and stock before accepting the order, so changing the browser display cannot change the accepted price or bypass a shortage.

Open payment QR code and View uploaded receipt use outlined links with an external-window icon. Edit basket is separated from the summary divider. These controls retain their original actions: opening evidence does not confirm a payment, and editing the basket returns to basket review.

4.2.6 MY ORDERS AND CUSTOMER SUPPORT

My Orders shows the customer's order items, accepted total, fulfilment details and payment status. Food receipt and payment are separate records: food may be marked received only after actual handover, while payment requires its own confirmation.

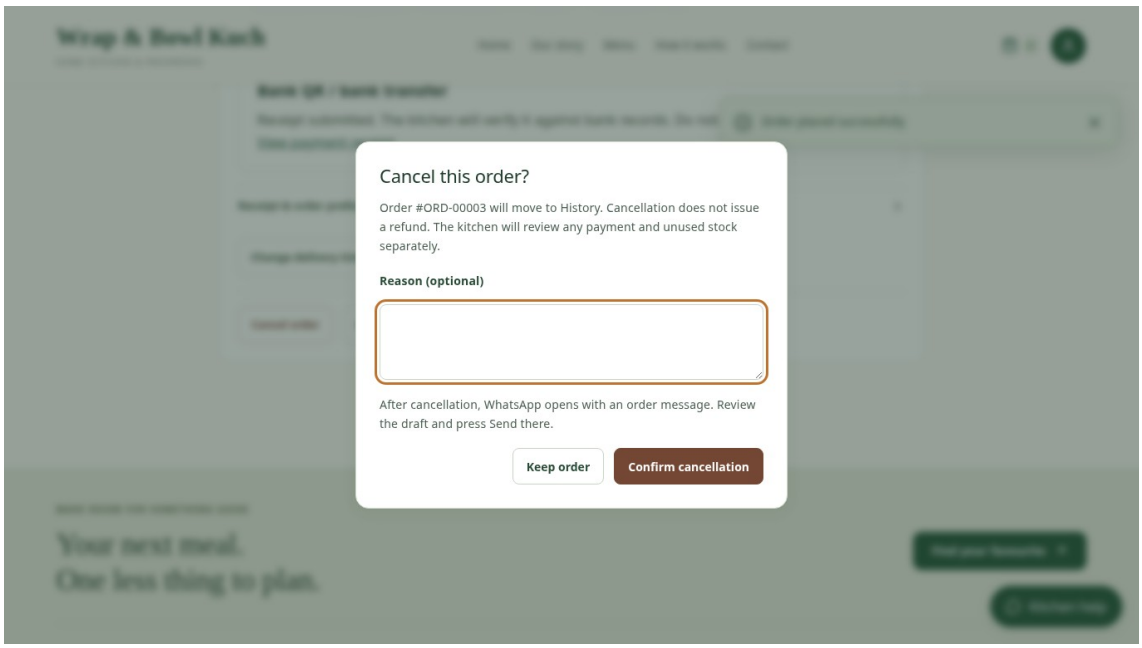


Figure 4.8: Customer order card and cancellation confirmation.

Eligible unreceived orders have a Cancel order button. Confirming cancellation updates the order card and moves it to History without a page refresh. WhatsApp then opens a message addressed to the kitchen's configured number, with the order reference and cancellation details. The customer must review and send the draft. Cancellation does not automatically refund a payment or return allocated stock; the owner reviews those separately.

Customers can also change the delivery or collection time for eligible orders. The latest permitted date depends on the ingredient batches already allocated to that order. The customer sees the date limit and can contact the kitchen about a later arrangement. A time change does not automatically replace the reserved ingredients. Kitchen Help remains available for questions about the website and ordering process.

4.3 OWNER INTERFACES

4.3.1 MENU MANAGEMENT

The owner can create, edit, pause and archive menus, manage categories, and update prices, images, selling units and offered days. Recipe lines record the quantity of each ingredient needed for one selling unit. These quantities connect a menu to inventory and are necessary for stock calculations. The initial recipes and packaging assignments are editable estimates because the client menu supplied names and prices rather than measured recipes.

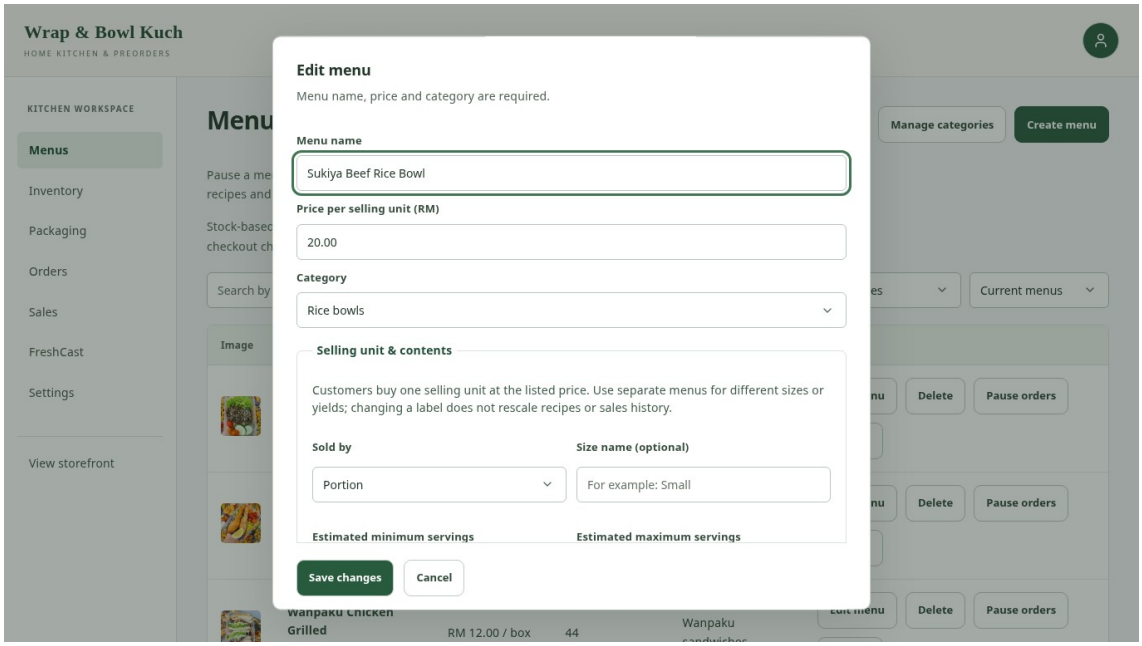


Figure 4.9: Editing a menu and its selling information.

The stock-based unit count is calculated from usable ingredient batches and assigned packaging. It is not an independent quantity that the owner must reduce after every sale. Menus can share ingredients, so their displayed counts cannot simply be added together. Checkout checks the actual combined requirement.

Menus linked to past orders retain those relationships. Pausing or archiving a menu can stop new orders while preserving earlier records. An accepted order also stores the menu name and price used at the time of purchase, so a later menu edit does not rewrite what a customer previously ordered.

4.3.2 INGREDIENTS AND INVENTORY BATCHES

The ingredient directory defines names and measurement units, such as grams, millilitres or pieces. Inventory batches record the amount received, receipt date, storage location and expiry evidence. Separate batches are useful when the same ingredient is purchased on different dates or has different deadlines.

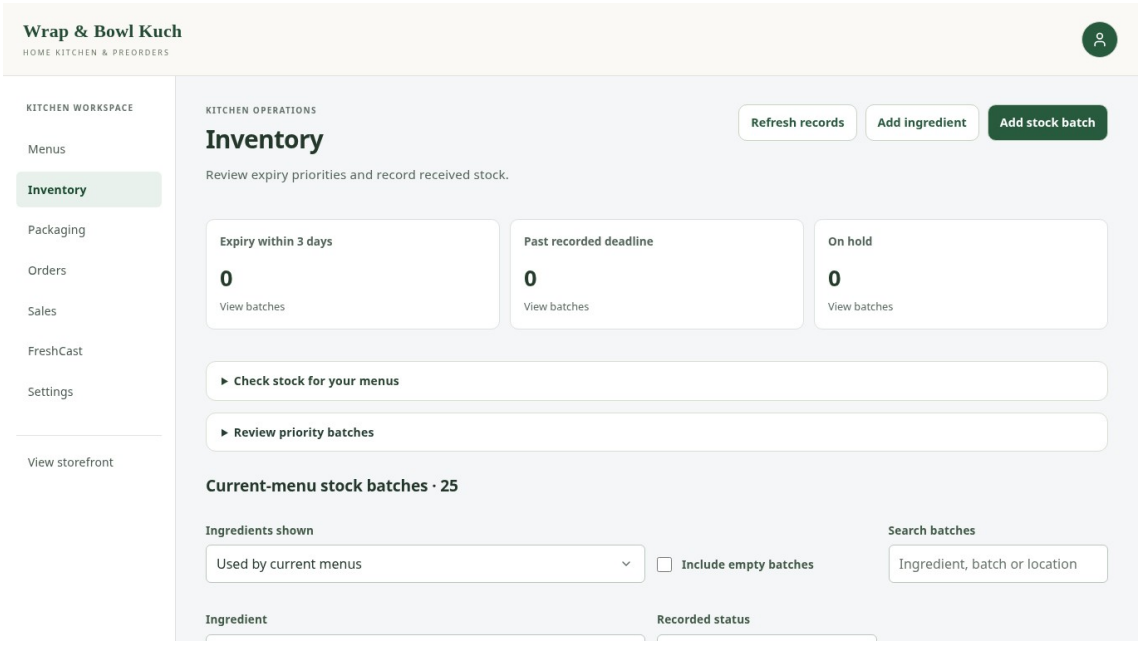


Figure 4.10: Inventory overview with stock filters and recorded batch status.

The batch form supports three date methods: a printed expiry or best-before date, manufacture date plus documented shelf life, and received date plus documented storage life. Received date means the day the batch reached the kitchen. Manufacture date means the day it was produced; the two dates are not interchangeable.

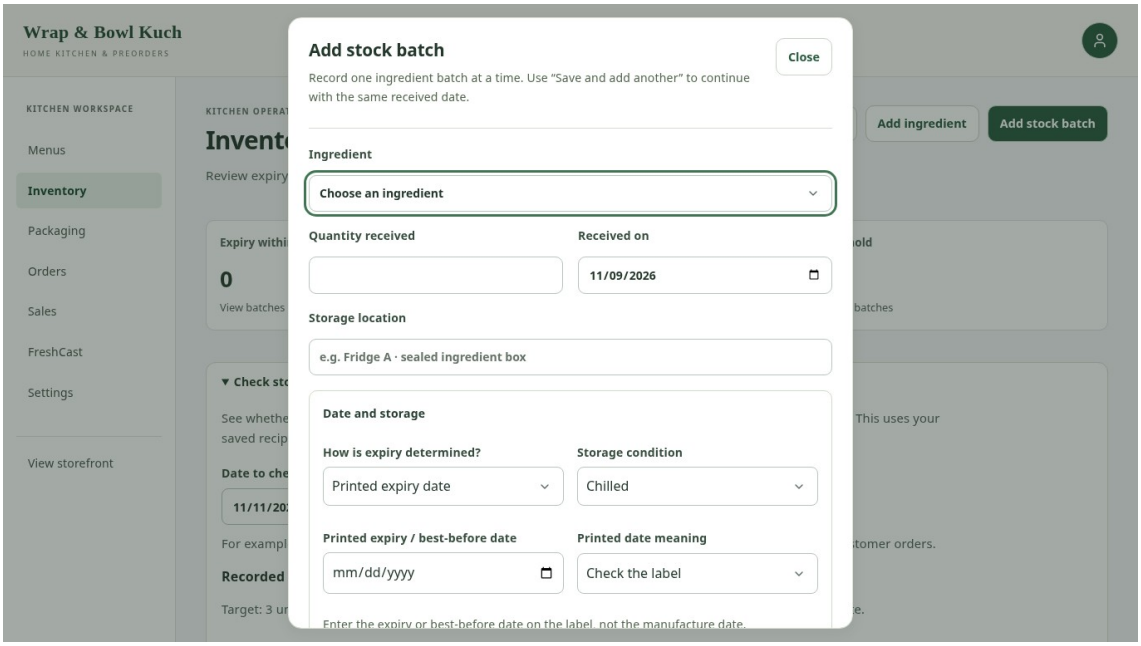


Figure 4.11: Stock-batch form with date and storage information.

For an unlabelled ingredient, the owner may enter a documented storage duration and its source. Optional reusable guidance can be saved against an ingredient, then copied into a batch and adjusted where appropriate. Creating a saved reference is not compulsory. The FoodSafety.gov link provides a general reference, but the owner must check whether the ingredient, food state and storage conditions match the guidance.

The earliest applicable deadline controls the batch. For example, a receipt-based calculation may give 30 days, but an earlier printed date still limits the batch. Opening or thawing can introduce an earlier documented limit. A batch placed on hold is excluded from cooking stock even if its date is in the future.

Recorded status follows these date rules consistently. Past-deadline batches are excluded, batches due within three days are highlighted, and held batches are identified separately. These are stock-planning indicators, not a measurement of physical freshness or a guarantee of food safety.

Save and add another lets the owner continue with the next stock batch while keeping the received date. The previous ingredient and quantity are cleared, reducing the chance of carrying them into a different batch.

4.3.3 STOCK TARGET CHECK AND PACKAGING

Check stock for your menus helps the owner review a planned quantity of every menu offered on a chosen date. If eight menus are offered and the target is three units per menu, the check covers 24 units in total. It compares the ingredient and packaging requirements with recorded usable stock and lists shortages. It does not place an order or deduct stock.

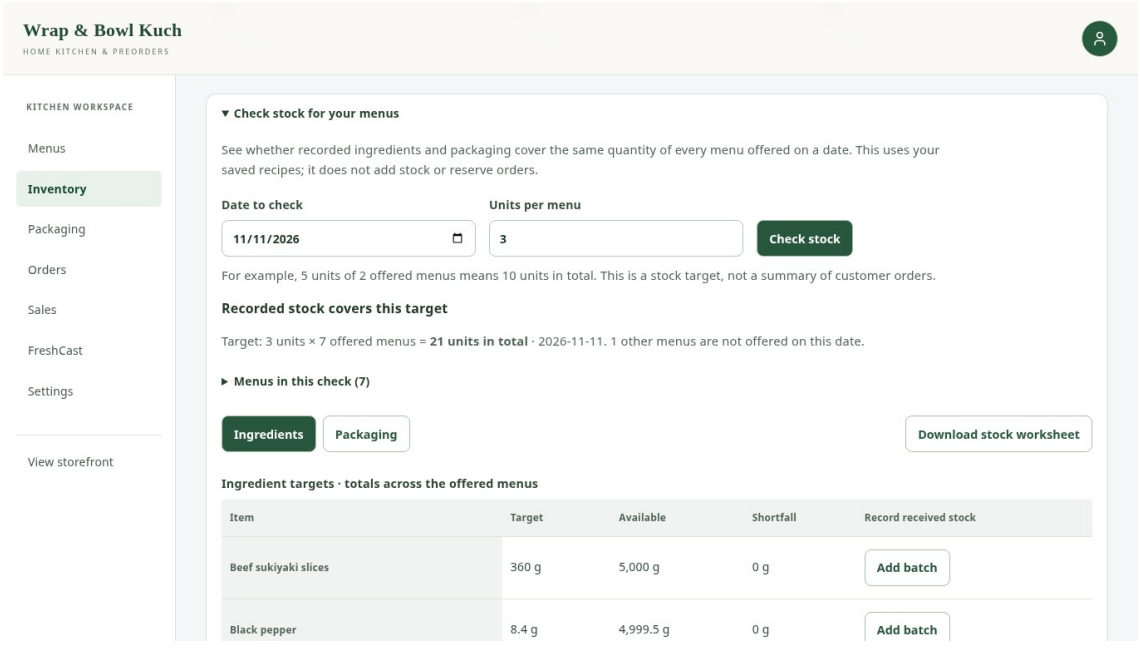


Figure 4.12: Checking ingredient requirements for a target quantity per menu.

The Packaging page tracks containers, lids, cups, cutlery, bags and labels. The owner can record received pieces, correct a balance with a reason, and review stock movements. Each menu is assigned the number of pieces required for one selling unit.

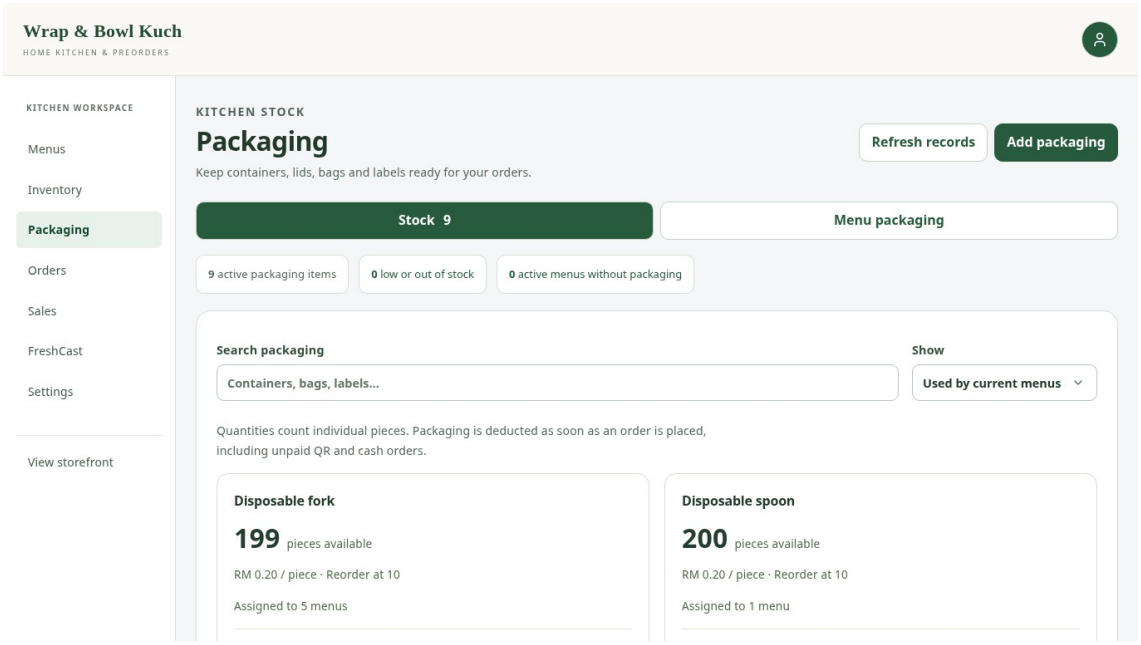


Figure 4.13: Packaging balances and stock-management actions.

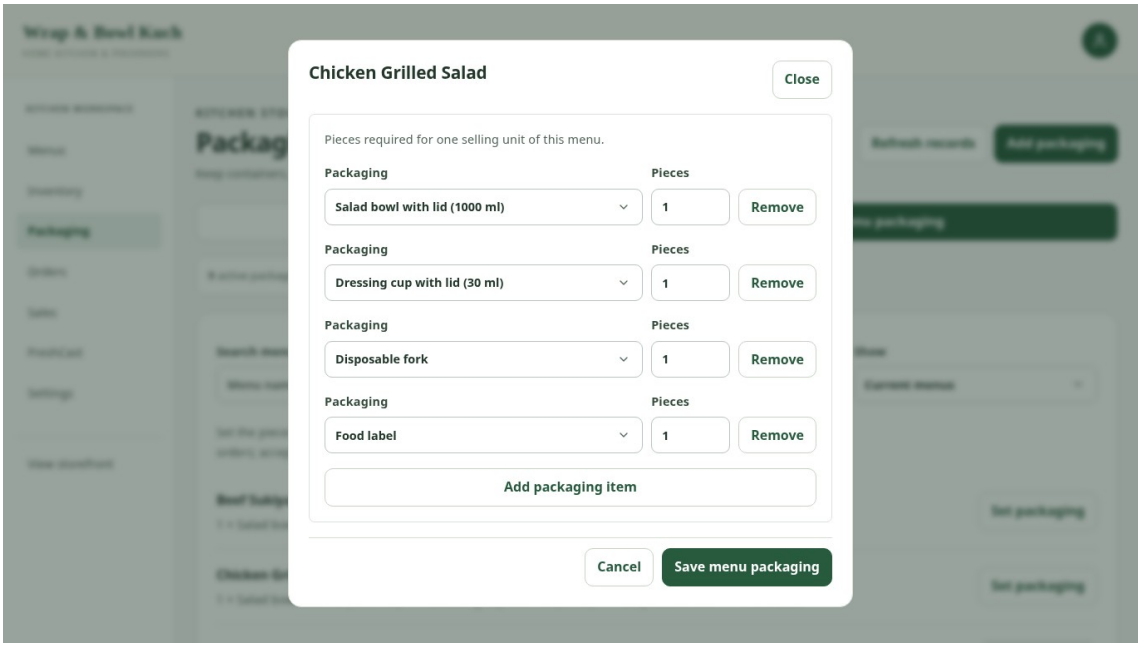


Figure 4.14: Assigning packaging pieces to one menu unit.

When an order is accepted, the system deducts both its ingredient quantities and packaging pieces. The owner therefore does not need to repeat those deductions manually. If either requirement cannot be met, the order is not partially accepted with only some stock deducted.

4.3.4 ORDERS AND PAYMENT REVIEW

The order queue groups orders by Not received, Received, Cancelled and All orders. Search, date filters and sorting help the owner find the relevant customers. Selected orders can be marked received together, but the owner must confirm that the food has actually reached every selected customer.

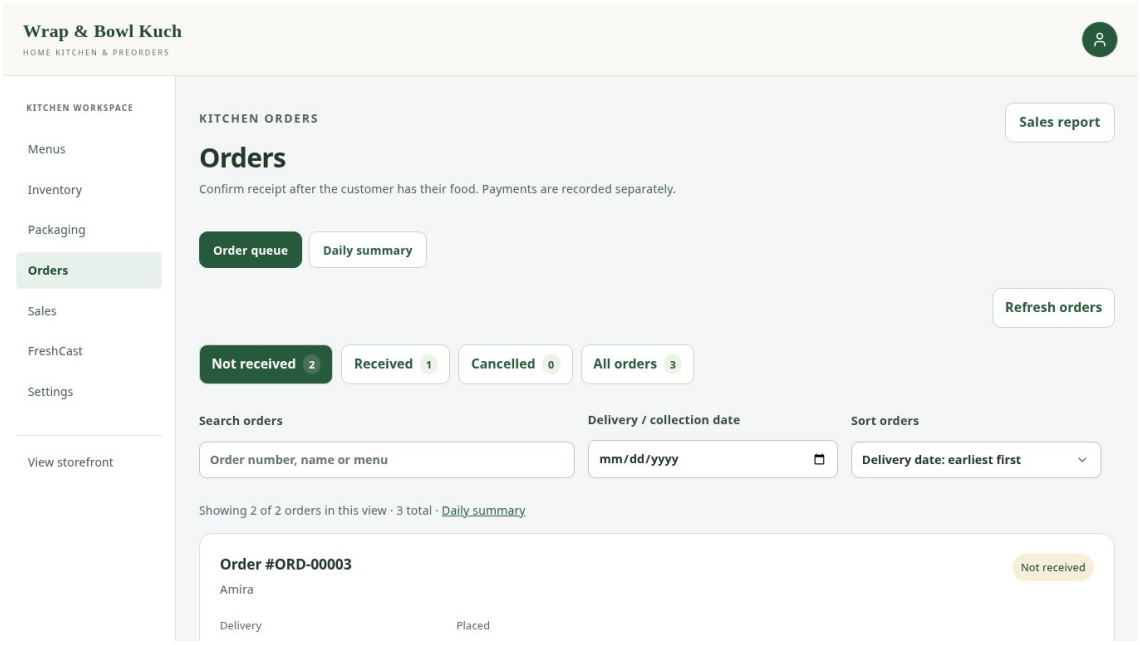


Figure 4.15: Owner order queue with filtering and receipt actions.

Payment review shows the accepted amount, uploaded evidence where available, and recorded payment events. The owner checks the bank transfer or cash collected before recording a receipt. Refund records describe money already returned outside the website; they do not send a bank transfer.

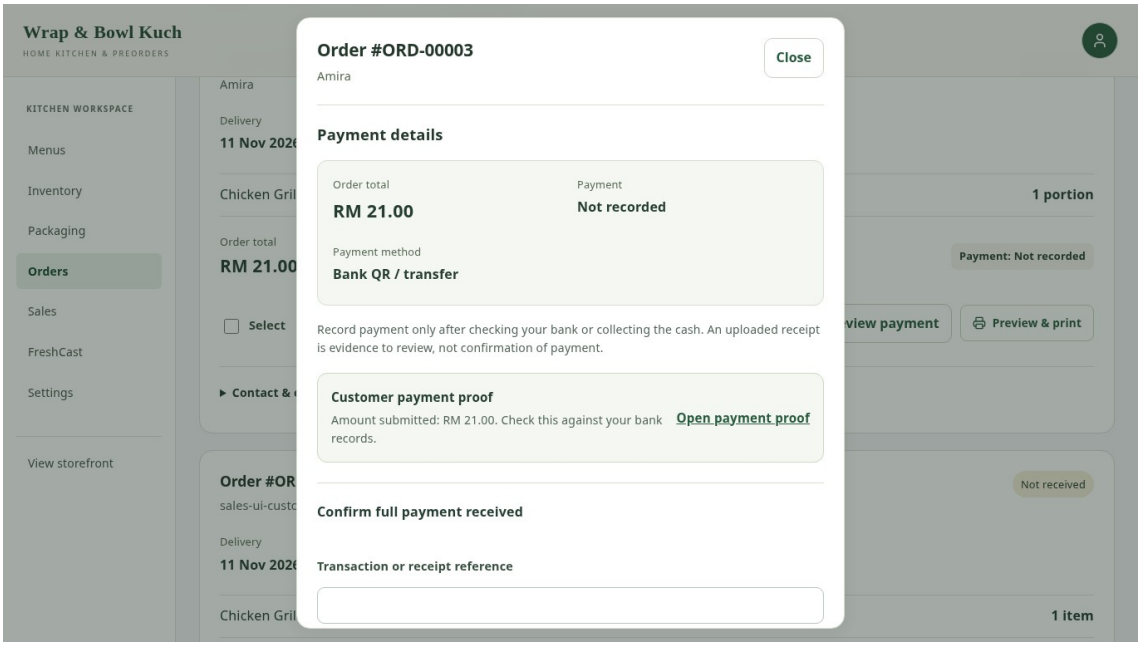


Figure 4.16: Reviewing payment details separately from food receipt.

The kitchen decides whether to use its own driver or arrange an external courier, for example when the usual driver is unavailable. Customers select delivery or collection at checkout; they do not select a courier company. The optional order tool lets the owner copy the recipient details and save a courier reference or tracking link after

arranging delivery externally. It does not book a rider, receive live delivery events or confirm that food has been received.

For a cash-on-delivery order, the owner must agree who will collect the money. If the external courier cannot collect cash for the kitchen, the owner needs to contact the customer, obtain agreement to a bank transfer and verify the transfer before dispatch. Adding a tracking link does not change the recorded checkout payment method. WhatsApp links open a conversation or prepared draft in WhatsApp; messages are not sent or stored by this website.

4.3.5 DAILY SUMMARY AND PRINTING

The daily summary groups orders by their delivery or collection date and totals the required units of each menu. It supports daily kitchen work, including orders whose payments still need review. This differs from the sales report, which applies its own payment and receipt conditions.

Order queue and Daily summary form a compact navigation group. The selected view uses the same green treatment as the other owner actions, with no full-width navigation strip. Switching views changes the displayed task; it does not change any order status.

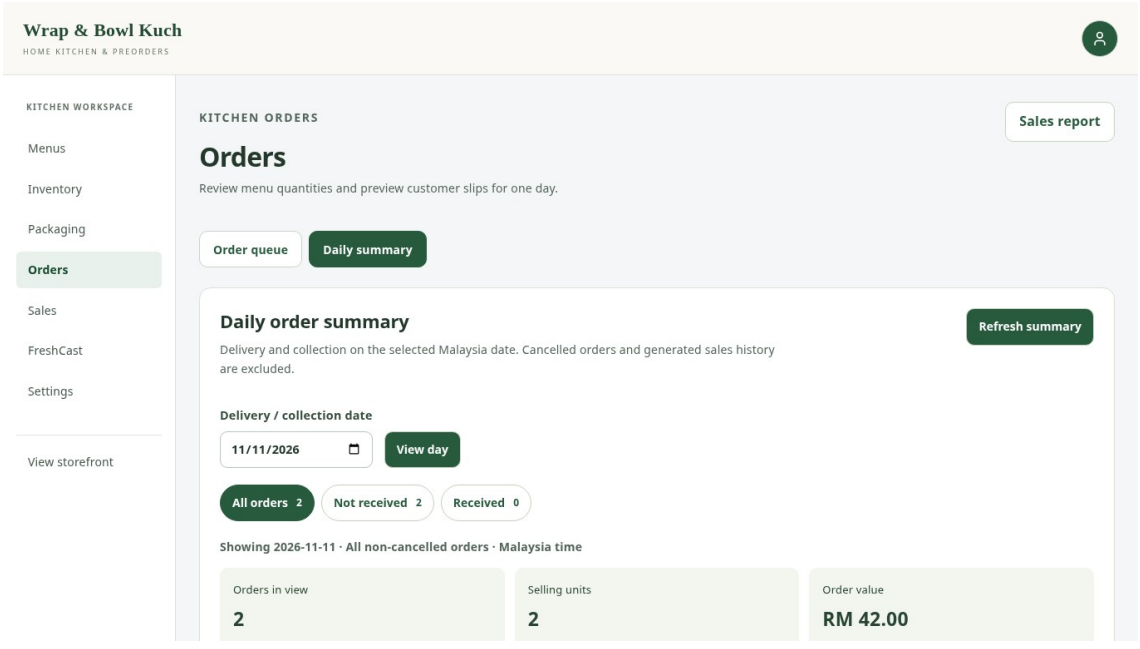


Figure 4.17: Daily order summary for a date containing example orders.

The daily summary provides Print all for the orders matching its date and receipt filter. One browser print action prepares a separate slip for each matching order. Cancelled orders are excluded. Individual order previews remain available. Each slip

contains the accepted customer name, phone, items, address where relevant and handover details.

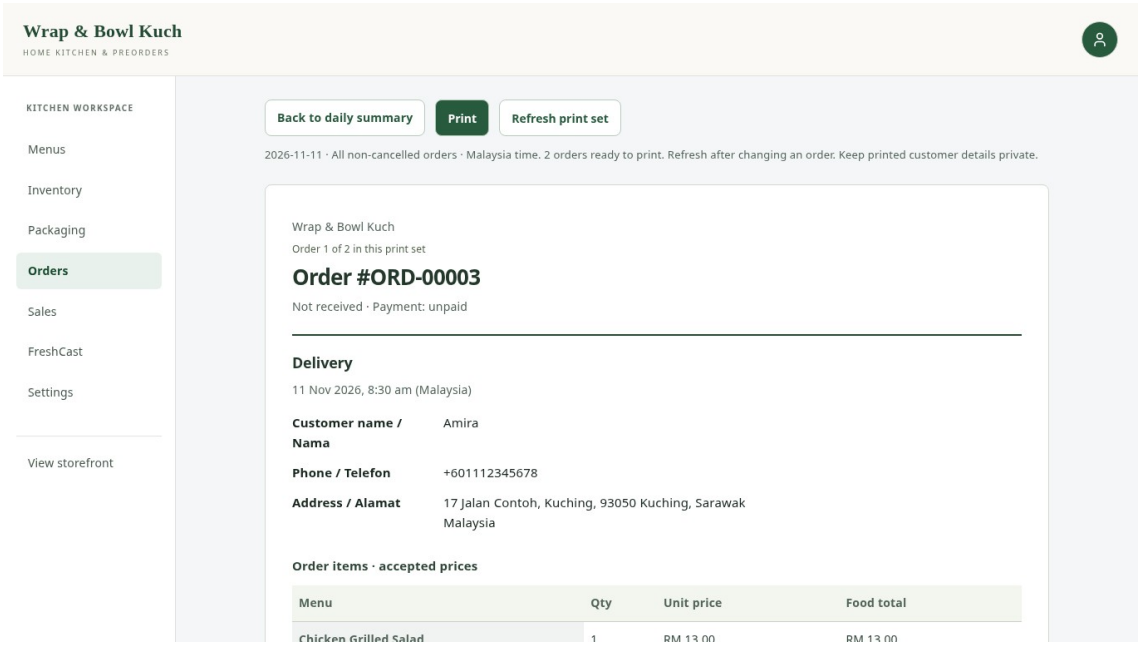


Figure 4.18: Matching daily order slips prepared together for printing.

4.3.6 SALES, COSTS AND WASTAGE

Sales overview lets the owner choose a date range and review recorded sales, quantities and delivery charges. Paid and received orders are distinguished from orders that are still pending or have incomplete payment evidence. The daily operational summary and sales totals therefore need not be identical.

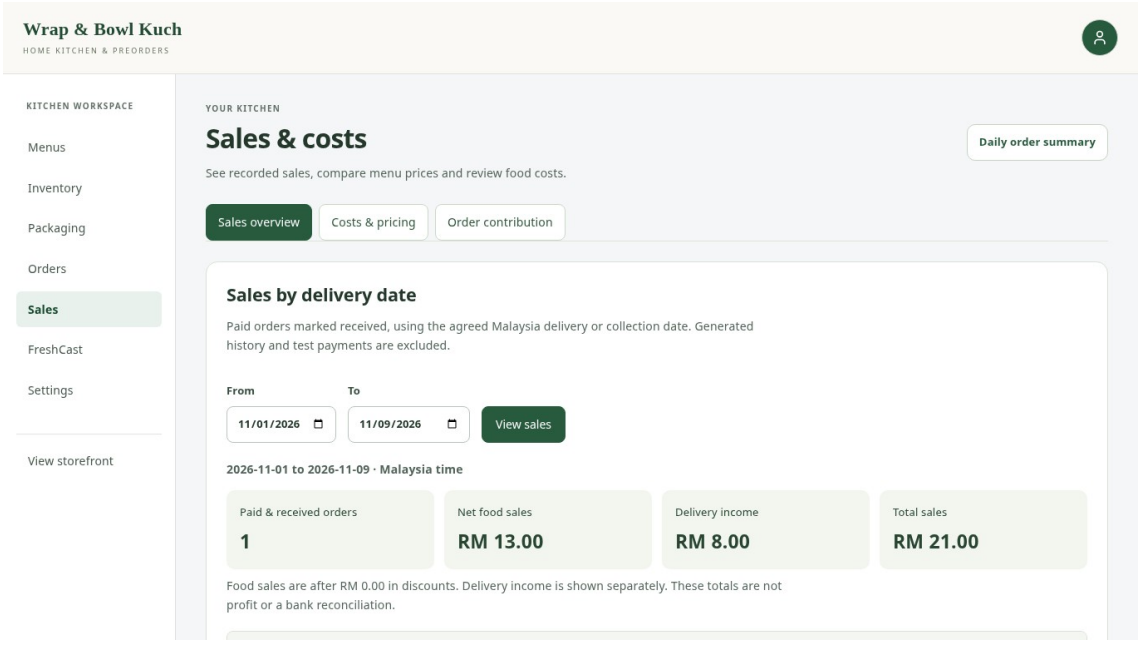


Figure 4.19: Sales overview with date controls and summary values.

Costs & pricing estimates food contribution using recipe ingredients, packaging and entered costs. The scenario form compares a proposed price or bulk discount

without changing the live menu or promotion. Ingredient costs are entered per inventory unit; for example, RM12 per kilogram is RM0.012 per gram.

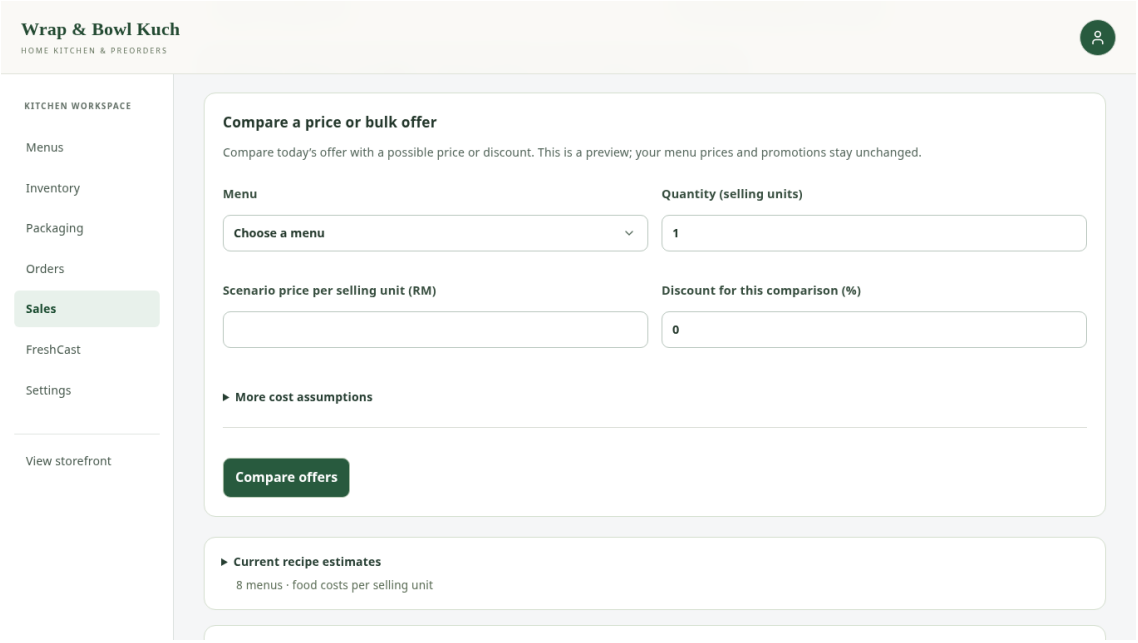


Figure 4.20: Price and bulk-offer scenario using current cost estimates.

Order contribution uses recorded costs associated with accepted orders where the required evidence is available. Packaging is included when its cost is recorded. A missing cost is not silently treated as zero. The result is food contribution rather than net profit, because labour, utilities and other expenses may remain outside that calculation.

Operating-expense records allow the owner to note costs such as delivery, utilities and marketing. The wastage form records ingredients that were discarded, deducts the selected quantity from its batch, and retains the cost information available at that time. It should not be used for ingredients already deducted for an order. These supporting records help the owner explain stock and cost changes; they are not a complete accounting system.

4.3.7 SETTINGS AND REMINDERS

Settings groups business information, collection and delivery, payments, promotions, email reminders and setup checks into separate sections. Delivery start and end times are shown together, followed by the delivery areas and fees. Each area uses a short place name and its own fee. The service-area description explains coverage in prose, while the collection location identifies where customers collect food. Courier handover is an optional order tool rather than an editable delivery setting.

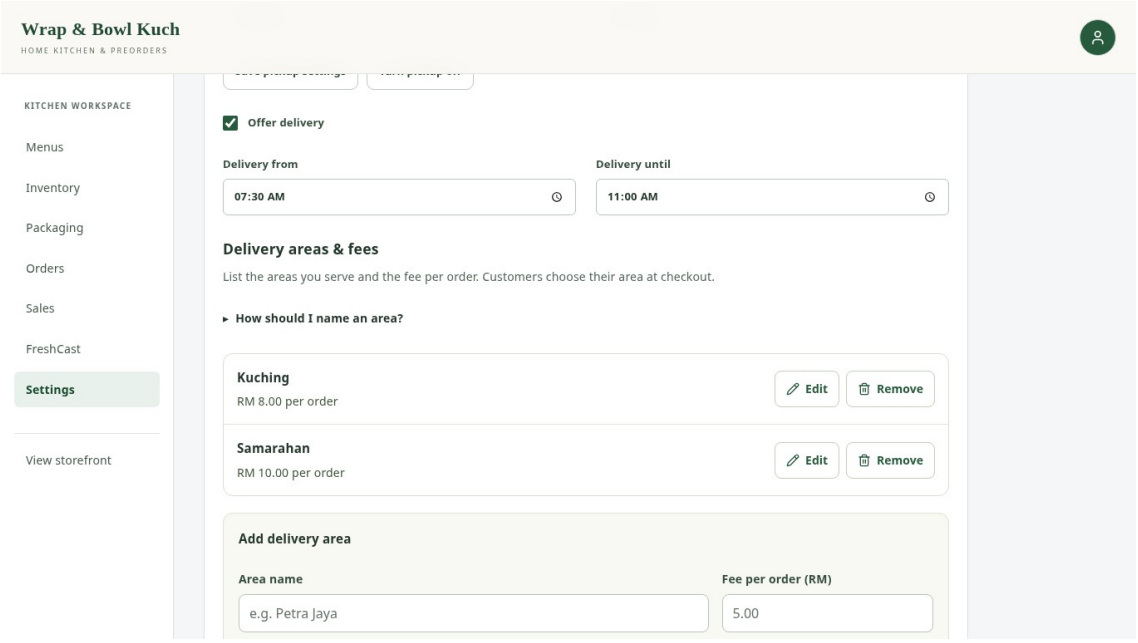


Figure 4.21: Delivery hours and area-based fees in Settings.

The owner can enable collection, set morning hours, update a receiving QR image and change a bulk promotion. Setup checks identify missing recipe, stock or cost information. They describe saved records; they do not verify a bank transfer or certify that food is safe.

The business-story section also accepts an uploaded owner photo. The owner can preview, replace or remove the image, then choose Save business story to publish the change on Our story. A failed upload keeps the previous photo. An image-link option remains available, so existing photo URLs can still be used.

The current ordering workflow uses owner delivery and collection reminders rather than preparation-planner reminders. Settings shows recent email checks and provides a separate test-email button. A successful sending attempt means the email service accepted the message; it does not prove the message reached the recipient's inbox.

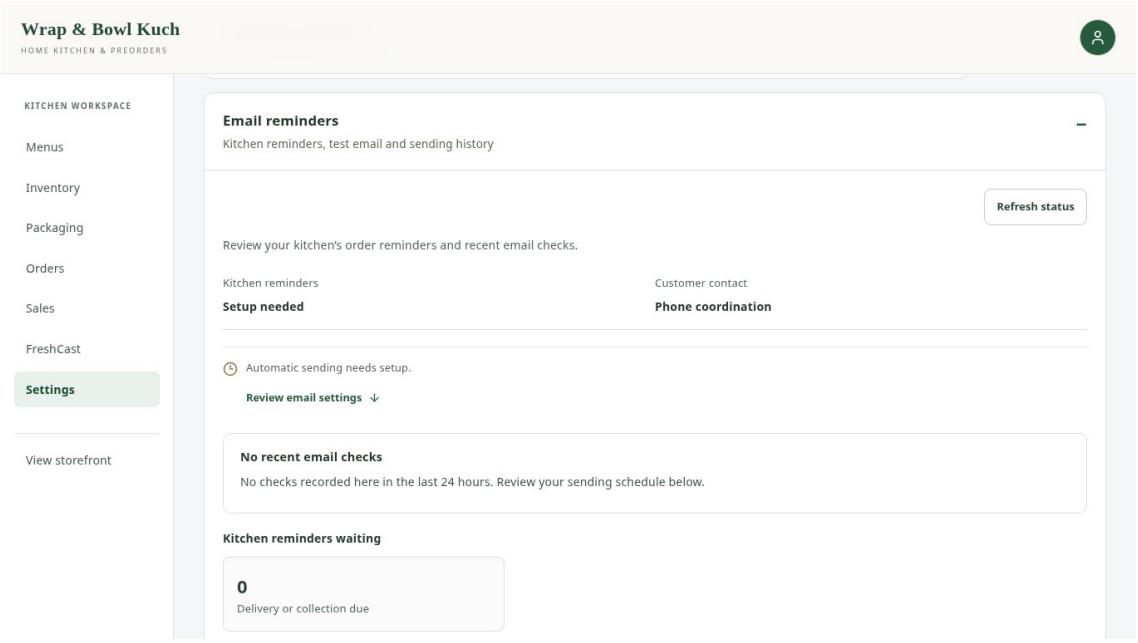


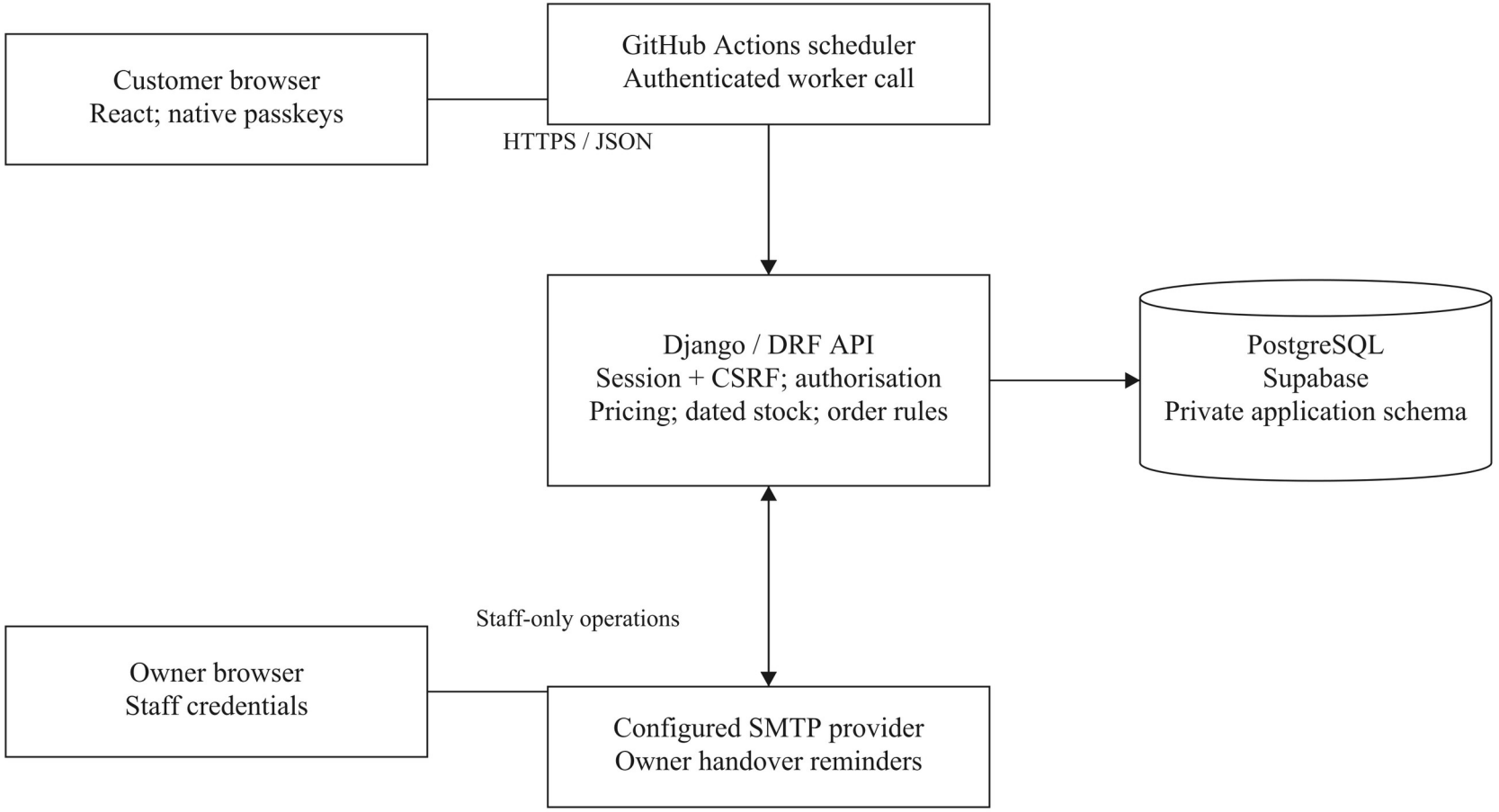
Figure 4.22: Reminder-status interface using local example configuration.

Customer accounts do not require an email address, and entering a phone number does not subscribe the customer to automated SMS or WhatsApp messages. Customer coordination therefore remains through the available contact routes. Reminder timing also depends on the configured scheduler running successfully.

4.4 IMPLEMENTATION DECISIONS AND DEVELOPMENT CHANGES

4.4.1 APPLICATION STRUCTURE AND ORDER PROCESSING

React displays the customer and owner interfaces. Django processes requests, applies permissions and performs the stock and pricing checks. PostgreSQL stores application records through Supabase, while Vercel hosts the deployed application.



QR transfers and courier bookings happen in external apps. There is no live bank or GrabExpress API connection.

Figure 4.23: Main components of the web application.

For each basket line, ingredient demand is the ordered quantity multiplied by the recipe quantity. Requirements for a shared ingredient are added together. Eligible batches must already have been received, remain within their recorded dates and not be on hold. The system allocates earlier-expiring stock first and records the quantity taken from each batch.

Ingredient deduction, packaging deduction and order creation are saved as one database transaction. If a required part fails, those changes are cancelled together. The accepted order keeps its names, prices, recipe and packaging details, allowing later menu changes without rewriting the order history.

For example, an order for two boxes that each require 100 g of one ingredient needs 200 g in total. A batch that expires before the delivery date is excluded even when its quantity is still recorded. If the eligible ingredient stock is sufficient but the required containers are unavailable, the order is rejected and neither stock type is partly deducted. This rule prevents acceptance from depending on an assumed future restock.

Prices are also calculated on the server. The food subtotal is reduced by a saved bulk discount when its threshold is met, then the applicable delivery fee is added. Collection has no delivery charge. The date-based finder and checkout use the relevant shared pricing rules to reduce inconsistent budget and payment totals.

4.4.2 INTERFACE REFINEMENTS

Development involved changes based on client needs and observed interface problems. The main changes are summarised in Table 4.1. Their purpose was to make common actions understandable while keeping stock, payment and receipt records consistent.

Each development cycle reviewed a user task, adjusted the design, implemented the change and checked its effect on related records. For example, the client’s request to print daily orders led to a shared daily filter and a Print all action; verification then checked that the selected slips matched that filter. Cancellation required both a saved status change and an immediate update to the customer’s order card. These cycles explain how feedback was translated into working behaviour rather than treating interface screenshots alone as development evidence.

Table 4.1: Main development changes and their purpose

Earlier issue or requirement	Implemented change
The client wanted fewer account details	Name and phone registration with a passkey; separate staff sign-in.
Detailed cooking planning did not suit the client's routine	Removed mandatory Planner steps from the normal ordering workflow.
Customers repeated delivery information	Kept area, date and time selection in checkout rather than repeating it in the basket.
Customers could overlook a menu's offered days	Added common ordering-day guidance and upcoming date choices.
Uploaded evidence could be mistaken for confirmed payment	Separated receipt upload, owner payment review and food-received status.
Dense forms and inconsistent controls made actions harder to find	Grouped related fields, aligned controls and used optional sections for secondary details.
The page could remain unable to scroll after closing sign-in	Corrected dialog closing behaviour and checked that scrolling returned.
The client wanted easier cancellation and follow-up.	Added confirmed cancellation with immediate order updates and a WhatsApp draft.
Printing one order at a time slowed daily preparation.	Added Print all for matching daily orders.
The owner needed to use a photo file rather than a web link.	Added portrait upload and preview, with publication after saving.

The design uses consistent labels, visible selection states, short feedback messages and errors near the relevant fields. These decisions relate to Nielsen's principles of visibility, consistency and error prevention (Nielsen, 1994). Keyboard focus, form labels and adaptable layouts also support accessibility (W3C, 2023). The green identity is retained for the business, with neutral page surfaces and restrained status colours.

4.4.3 FRESHCAST AND RECORDED-STOCK INFORMATION

FreshCast is a separate owner workspace for reviewing demand and ingredient-planning information. Current menu recipes connect selling units to ingredient quantities. The tool distinguishes analysis of local order records from the external benchmark model and its example walkthrough. The external model does not establish forecasting accuracy for the client's business, because genuine client sales have not been collected for that evaluation.

FreshCast distinguishes forecast stock use before expiry from demand that occurs later. Stock can therefore remain unused at its deadline while a later date still has a shortfall. The review screen explains these outcomes in words, with the arithmetic available under See calculation. These estimates do not deduct stock or place a purchase order.

The public storage section explains the kitchen's recorded batches. It shows batches within their dates, past their dates, on hold, or missing enough date information. An optional average describes the proportion of recorded shelf life remaining. This value is calculated from saved dates; it is not a laboratory measure of food freshness.

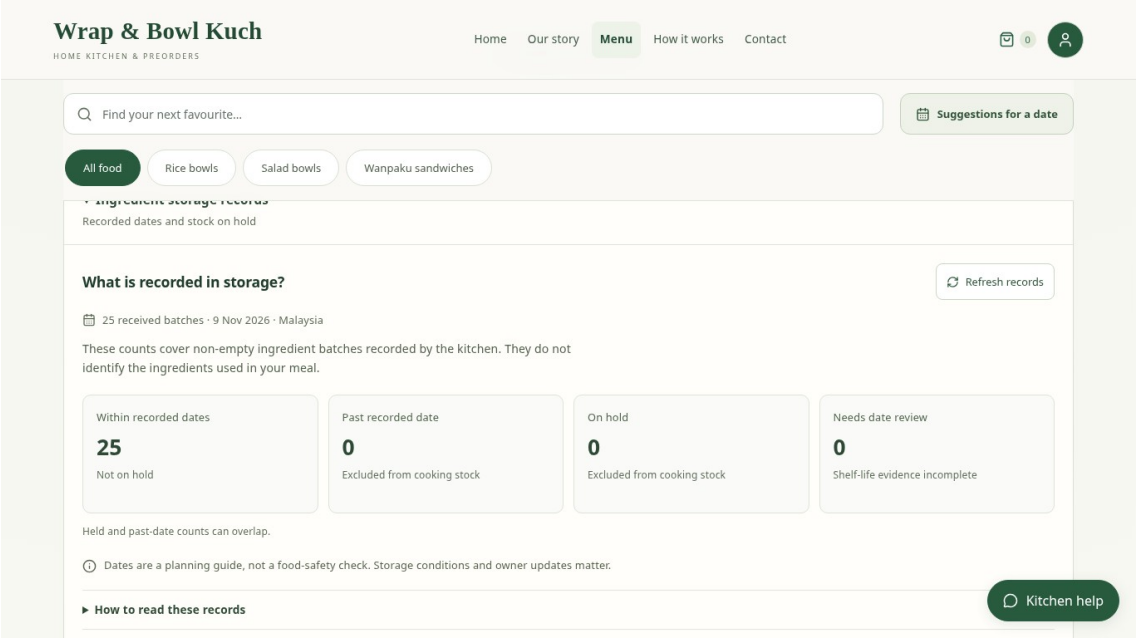


Figure 4.24: Public summary of recorded ingredient-batch status.

Stock accuracy still depends on the owner entering actual receipts, storage conditions, corrections and wastage. The system can apply consistent date rules to those records, but it cannot observe physical handling or guarantee food safety.

4.4.4 ACCESS CONTROL AND FAILURE RECOVERY

Customer and staff actions are checked on the server. A customer can access only their own private orders and payment receipts, while kitchen management actions require staff access. Session-based changes also require a valid cross-site request forgery token. Hiding an owner button in the customer interface is therefore not the access-control mechanism.

The server rechecks prices, stock and order versions when saving a change. A stale date amendment or cancellation is rejected instead of overwriting a more recent update. Checkout uses a request reference so a retry can return the accepted order

without deducting stock again. PostgreSQL transactions and row locks protect related writes and competing requests (Django Software Foundation, n.d.).

Receipt and owner-photo uploads are validated before use. Receipts remain private, while the owner image becomes public only after its settings section is saved. Rejected or failed uploads preserve the previously saved image. These choices apply the separation of public content and private evidence, together with file validation described by OWASP Foundation (n.d.).

The interface reports a failed or uncertain save and provides a way to retry or reload the saved record. It does not display a success message merely because a button was pressed. This preserves the distinction between an attempted action, a confirmed database change and a separate real-world event such as receiving money or handing over food.

4.5 SUMMARY

The implemented system connects customer preorders with menu recipes, ingredient batches, packaging and owner records. Client feedback simplified account creation, morning handover and order receipt while retaining stock checks and payment review. Chapter 5 presents the functional testing and explains how client and student evaluation will be used to assess the completed system.

CHAPTER 5
TESTING AND RESULTS

5.1 INTRODUCTION

This chapter presents the testing carried out on the preorder and freshness management system. Testing checked whether customer orders, inventory deductions, payment records and reports followed the intended rules. It also considered invalid inputs and actions that should be refused, such as placing an order without enough stock.

Software testing and user evaluation serve different purposes. Functional tests check what the system does, while client and student evaluation examines whether people understand it and find it useful. The functional results are available. Formal client acceptance and student survey results have not yet been collected.

5.2 TESTING APPROACH

Automated functional tests submitted valid and invalid inputs and checked the resulting records. For example, an order test compared stock before and after placement, while a date test checked times inside and outside the delivery window. Repeating these checks helps detect whether a later change breaks an earlier function.

Automated browser checks covered the customer and staff journeys, including guest basket review, passkey registration and login, recommendations, checkout, cancellation, inventory entry and owner-photo uploads. Batch printing and recovery from selected failed requests were also checked at laptop and phone widths. Passkey checks used a browser test device, so they do not establish that every customer's physical device will work.

The checks ran locally using a separate test database and example records. They did not change the live Supabase records, make a bank transfer, send a real reminder or book a courier. Saved test logs provide the detailed results in the digital evidence package. No independent tester or participant is credited with these automated checks.

On 9 November 2026, the complete PostgreSQL run passed all 1,049 backend tests with no skips. Frontend tests, deployment and runtime checks, code-quality checks and the production build also passed. Browser checks recorded no page errors, and the saved forecast model ran without retraining. These results describe the local test conditions and do not replace user evaluation or live deployment verification.

After the checkout, order-navigation and Settings refinements, the affected frontend regression groups, code-quality check and production build passed again. A separate browser run checked receipt upload and the changed layouts at laptop, tablet and phone widths. It found no horizontal overflow or page errors in those scenarios. The refinements did not change stock, payment or forecasting calculations, so this follow-up is reported separately from the complete backend run.

5.3 FUNCTIONAL TEST RESULTS

The following tables summarise representative tests in terms of actions and outcomes. Pass means the stated automated check produced the expected result. It does not mean that a survey participant completed the task or that the client formally accepted it. Full test names and logs are retained separately so the report can remain readable.

5.3.1 ACCOUNTS AND CHECKOUT

Account and checkout tests covered guest basket review, registration, invalid verification attempts, morning delivery times, area charges and payment choices. They also checked cancellation updates, delivery-date limits and the distinction between an uploaded receipt and a verified payment.

Table 5.1: Customer account and checkout checks (automated)

Case	Action tested	Expected result	Observed result	Status
FT-01	Create a customer account; try an incomplete phone number and then a full number.	Valid name, full phone and passkey create an account. The basket is retained; an incomplete number is rejected.	The valid account retained its basket. The phone prefix alone was rejected.	Pass
FT-02	Try to reuse an account-verification challenge or use it from the wrong website origin.	Invalid verification attempts do not create or grant access to a customer account.	Invalid challenges were rejected.	Pass
FT-03	Check times at and just outside the morning delivery window.	07:30 and 11:00 are within the configured bounds; 07:29 and 11:01 are outside them. Other checks still apply.	Availability checks agreed with the configured minute boundaries.	Pass
FT-04	Choose a delivery area and a qualifying bulk quantity; submit an outdated total.	The saved area fee and food discount determine the total. An outdated total cannot place the order.	The calculated, receipt-upload and accepted totals agreed. The outdated total was rejected.	Pass

Case	Action tested	Expected result	Observed result	Status
FT-05	Try delivery cash and a collection setup without a receiving QR image.	Delivery cash remains unpaid until confirmed. Bank instructions alone cannot enable QR-only collection.	Cash delivery saved the order details. The missing collection QR requirement was enforced.	Pass
FT-06	Upload an example receipt and place an eligible local preorder.	The receipt attaches to the order; payment is not automatically confirmed.	The uploaded receipt appeared in staff review. The placed order remained unpaid.	Pass
FT-07	Review a guest basket, then select Continue to checkout.	Basket viewing stays open; checkout opens Login. Registration keeps the basket.	The modal opened only from checkout and registration retained the food.	Pass
FT-08	Cancel a pending order; also simulate a rejected cancellation.	A successful cancellation updates the card and counts immediately. A rejected request leaves the order unchanged.	Status and counts updated without refresh; the failed request kept the pending status.	Pass
FT-09	Request a date later than the allocated stock permits.	The change is rejected without changing the agreed time or allocations.	The deadline check rejected the later date and preserved the order.	Pass

5.3.2 INVENTORY AND PACKAGING

Stock tests checked the connection between accepted orders, ingredient recipes and packaging requirements. Date tests checked the difference between manufacture-based and receipt-based life, earlier deadlines and held batches.

Table 5.2: Inventory and packaging checks (automated)

Case	Action tested	Expected result	Observed result	Status
FT-10	Place the client-menu basket, then repeat the accepted request.	Ingredients and packaging are deducted once using the recipes and packaging requirements.	The expected deductions were recorded once; repeating the request did not repeat them.	Pass
FT-11	Place an order with insufficient assigned packaging.	No partial order or stock deduction is saved. The basket is kept.	The shortage prevented placement and preserved the basket.	Pass
FT-12	Compare manufacture-based and receipt-based life, then add an earlier printed or opened limit.	Each duration starts from the correct date. The earliest applicable deadline controls the batch.	Date calculations used the correct start and earlier limits.	Pass
FT-13	Record uncertain handling for a batch with a future deadline.	The batch is held and excluded from usable stock.	The future date did not remove the hold.	Pass

Case	Action tested	Expected result	Observed result	Status
FT-14	Check a target across menus that share ingredients and packaging.	Requirements are added across menus; the check changes no stock or orders.	Shared requirements and shortages were calculated without stock changes.	Pass

The results show that date-based rules were applied consistently in the tested records. They do not establish that an ingredient was physically safe or that its storage history was entered correctly. Those matters still depend on actual handling and accurate owner records.

5.3.3 ORDER RECORDS AND REPORTING

Reporting tests compared daily order totals with sales totals, checked incomplete costs and confirmed that later packaging edits did not rewrite accepted orders. Browser checks also printed three orders through one action and checked that owner-photo changes were published only after saving.

Table 5.3: Order records and reporting checks (automated)

Case	Action tested	Expected result	Observed result	Status
FT-15	Edit a menu packaging assignment after an order has been accepted.	The accepted order keeps its earlier packaging quantities and costs.	The earlier accepted records were unchanged.	Pass
FT-16	Compare daily operational totals with sales using open, cancelled and generated records.	Each report applies its stated rules. Generated or test-payment records do not become sales.	The daily and sales reports included and excluded the expected records.	Pass
FT-17	Leave the cost of assigned packaging unknown in a price comparison.	The contribution estimate is unavailable rather than treating the missing cost as zero.	Missing packaging cost prevented a misleading estimate.	Pass
FT-18	Preview a collection slip and export a report containing spreadsheet-sensitive text.	The slip uses the accepted collection contact details. The export protects report access and handles formula-like text safely.	The slip used the accepted phone and collection details. The authorised export escaped formula-like values.	Pass
FT-19	Print three matching daily orders in one action.	All three slips are included; one Print action produces one page per order.	One action prepared three slips and the PDF contained three pages.	Pass
FT-20	Upload and replace an owner photo; try invalid and failed uploads.	A change is published only after saving. Failure preserves the existing image.	Saving, replacement and removal worked; rejected uploads kept the photo.	Pass

5.3.4 MENU RECOMMENDATIONS

Recommendation tests varied the maximum budget, item count and menu choice while checking the complete-order total. They included an exact-budget match, a budget too low for the basket, repeated menu units and alternative one-menu baskets. A correct calculation is separate from evidence that customers find choosing easier.

Table 5.4: Menu recommendation checks (automated)

Case	Action tested	Expected result	Observed result	Status
FT-21	Search with a maximum budget equal to the complete order total.	The exact total is allowed; discount and delivery are included.	The exact-match result used the complete total and stayed within budget.	Pass
FT-22	Keep three items and the same area; change the example budget from RM43 to RM44.	No basket is offered below the cheapest feasible total; increasing the budget can make one available.	No option was returned at RM43. One valid three-item option was returned at RM44.	Pass
FT-23	Request three items using One menu only, with several eligible menus.	Each alternative contains three units of one menu; at most three alternatives are shown.	Quantity and repeated-menu rules passed in the automated checks.	Pass

5.3.5 CORRECTIONS AND RETESTING

Testing also helped identify interface problems. Table 5.5 gives selected examples of the changes made and the observed retest outcome. These were development checks rather than client evaluation results.

Table 5.5: Selected problems, corrections and retest outcomes

Problem	Change made	Retest outcome
The page could remain unable to scroll after sign-in dialogs closed.	Corrected the way dialogs restore page scrolling.	Scrolling returned after the checked dialog sequences.
An invalid ordering day could leave an earlier availability result visible.	Cleared the old result and explained the basket's offered days.	The old result could not be used to submit the invalid date.
Cancelling the receipt file chooser could discard a valid upload.	Kept the existing upload when no replacement file was selected.	The uploaded evidence remained available; invalid replacements still showed an error.
The stock-check action was misaligned and its target was unclear.	Aligned the controls and explained units per offered menu.	The controls adapted to the checked screen sizes, and the total changed correctly with the target.

Problem	Change made	Retest outcome
My orders could still show the earlier status after cancellation.	Applied the server-confirmed order to the visible card and counts.	Cancellation appeared without a reload; a rejected request left the order unchanged.
The date finder still showed an unnecessary budget comparison.	Kept one optional maximum-budget field.	Budget-boundary and item-count checks returned the expected baskets.
Checkout links and order navigation were visually inconsistent; delivery-hour fields were crowded.	Gave payment links clear controls, reset the order navigation style and separated delivery hours from area fees.	Receipt upload and the changed layouts passed the affected regression and browser checks.

5.3.6 SECURITY AND CONCURRENT ORDER CHECKS

The additional cases in Table 5.6 check who can change records and what happens when requests overlap. They use separate test accounts and a local PostgreSQL database. They complement the ordinary customer journeys because a successful checkout alone does not demonstrate privacy, request protection or safe stock allocation.

Table 5.6: Selected security and concurrent-order checks

Case	Action tested	Expected result	Observed result	Status
FT-24	Try to read one customer's uploaded receipt from another customer account.	Only the uploader and authorised kitchen staff can read the private image.	The other customer received no receipt; the uploader and staff could view it.	Pass
FT-25	Submit customer cancellation without a valid session or request-protection token.	The request is rejected without changing the order.	The session and request-protection checks rejected the writes.	Pass
FT-26	Submit two orders for different menus sharing 500 g of one ingredient; each needs 300 g.	One order is accepted and one is rejected; stock never becomes negative.	One order was saved, one was rejected and 200 g remained.	Pass
FT-27	Send two simultaneous placement requests with the same customer request reference.	Both responses identify one order, with one ingredient deduction.	One order and one deduction were saved; the retry returned the same order.	Pass

These results provide evidence for the stated access and concurrency rules. They are not an independent penetration test or a high-volume performance test. A separate simultaneous-receipt check also confirmed that one uploaded receipt image could not

fund two orders. The result concerns application evidence reuse; it does not establish whether a transfer occurred in a bank account.

5.3.7 LIMITS OF THE TESTING

Access-control and invalid-input checks covered selected security risks, including invalid account verification, payment-evidence access and unauthorised management actions. They are not an independent security audit. Likewise, successful builds and adaptable layouts do not establish performance under a large number of simultaneous customers.

The local PostgreSQL run verified selected concurrent-placement and retry scenarios, including the cases above. It did not reproduce every production load, network interruption or device. Browser screenshots also do not prove that the live deployment, email schedule or banking service is working. These limits remain separate from the recorded functional passes.

5.4 USER EVALUATION

5.4.1 CLIENT FEEDBACK AND ACCEPTANCE

The client meetings provided requirements feedback. Requested changes included packaging stock, immediate stock deduction, simpler customer details, area-based fees, morning delivery, receipt uploads, order printing and a simpler food-received status. These changes helped adapt the system to Wrap & Bowl Kuch.

The meetings were informal, and no completed interview transcript or formal acceptance form has been supplied. The implemented changes therefore demonstrate a response to the client's requests, but are not presented as a scored acceptance result.

A short follow-up is planned after the client tries the revised owner pages. It covers menu and delivery settings, ingredient dates, packaging, order and payment review, daily summaries and sales information. The record distinguishes tasks completed independently, completed with help, unsuccessful and not tried. The client can respond using a paper form or a Google Form; a separate formal interview is not required merely to document those answers.

Formal client evaluation results are pending. This subsection will report the actual tasks tried, comments received, requested changes and any follow-up checks after collection. Appendix E contains the blank follow-up form.

5.4.2 STUDENT SURVEY RESULTS

Students are intended to evaluate the customer journey through a short Google Forms questionnaire. They will try menu browsing, a date-based recommendation search, basket review and the available checkout guidance. Questions concern business identity, ordering dates, total price, account instructions and recommendation clarity. Students will not be asked to verify hidden inventory calculations or accounting rules.

The questionnaire includes a five-point agreement scale, a Not tried option and short comments. A simple comparison asks participants to try ordinary browsing and recommendations, each starting with an empty basket, under comparable date, quantity and budget conditions, then report the effort involved. Where both tasks are observed, their order can be alternated between participants to reduce the advantage of doing one task first.

Participation is voluntary. The study will use assigned example account details where needed and will stop before a real payment or live order placement. The questionnaire will not request personal bank receipts, phone numbers or email addresses. If an account step cannot be tried on a participant's device, that limitation will be recorded.

No student responses have been collected for this revision, so no satisfaction percentage or survey chart is reported. Once available, the results will state the actual number of responses, show rating counts and percentages with the number of valid answers, summarise comments, and explain the resulting changes. Not tried responses will be reported separately rather than treated as agreement. Appendix F contains the questionnaire.

The comparison can provide evidence about reported effort and task completion. It does not measure real paid purchase conversion. Purchase intention, if discussed in feedback, will be described as intention rather than a completed sale.

5.5 DISCUSSION AGAINST THE OBJECTIVES

5.5.1 IMPLEMENTATION AND USER EVIDENCE

Table 5.7 links the implemented functions and testing to the five objectives in Section 1.3. The final column identifies where actual user feedback is still needed.

Table 5.7: Evidence related to the project objectives

Objective	Implemented and tested functions	Evaluation still needed
1: Date-based preordering and scheduling	Offered dates, morning time selection, area fees and order acceptance checks.	Whether the client and customers can use the ordering and handover process easily.
2: Ingredient shelf life and storage duration	Batch-date calculations, earlier limits, holds and exclusion of unusable recorded stock.	Whether the owner understands the date methods and can maintain accurate records.
3: Personalised single-vendor storefront	Client identity and story, own menu categories, images and business settings.	Whether customers recognise the business and the client finds the branding suitable.
4: Sales analytics for decisions	Daily summaries, sales rules, printing and cost comparisons.	Whether the owner understands the reports and finds them useful for decisions.
5: Menu recommendations	Personalised browsing suggestions and date/quantity/budget alternatives.	Whether users report less effort when choosing; real conversion improvement has not been measured.

The results address the specific workflow gaps discussed in Chapters 1 and 2: preorder records, ingredient dates, storefront identity and decision support operate within one application. Local tests support those internal connections. External banking, messaging and courier services remain separate, and the case study does not establish that every home-based business has the same requirements.

The food-poisoning reports discussed in Chapter 1 and the monitoring literature in Section 2.2.2 motivate attention to ingredient management. The implemented approach records shelf life and storage duration without requiring sensors. It makes the dates used for stock decisions visible, but cannot detect contamination or replace proper handling.

Client feedback also showed that more functionality is not always more suitable. Detailed preparation planning was removed from the normal workflow because it did not fit the client's needs. Date-based delivery and collection were retained. Passkeys similarly reduce the need to remember a website password, but their ease of use still needs feedback from the intended users.

5.5.2 FORECAST RESULTS AND THEIR SCOPE

FreshCast retains an earlier evaluation using the external Genpact dataset. The recorded weighted absolute percentage error was approximately 28.23%, compared with

34.81% for its reference method; lower error is better. This is an earlier external-data result, not a new test using Wrap & Bowl Kuch's actual sales.

The example orders used in development support demonstrations of charts and calculations, but cannot establish genuine demand accuracy, profit or food-waste reduction for the client. The model was not retrained for this report revision. Genuine operating data would be needed to evaluate those business outcomes.

The latest automated run also loaded the saved model and checked forecast and historical-replay responses. This confirmed that the retained model could run in the test environment. It did not provide a new accuracy result for the client's orders.

5.5.3 DISCUSSION IN RELATION TO THE LITERATURE

The reviewed systems in Section 2.2.5 motivated structured date selection, ingredient records and an independently branded storefront. The implemented ordering process addresses those specific gaps by checking offered dates, recording the accepted basket and applying recorded-stock eligibility. It does not reproduce the delivery networks of GrabFood or FoodPanda, or the conversations provided by WhatsApp. Its practical contribution is linking the kitchen’s ordering, stock and reporting records for the client’s workflow.

The date-based stock rules provide a lightweight response to the monitoring need discussed by Pallianysamy et al. (2025). They make recorded expiry dates and handling limits available to ordering decisions. By comparison, the system examined by Tsang et al. (2019) uses IoT-based shelf-life management. This project does not measure temperature or product deterioration, so its successful date calculations cannot be interpreted as equivalent physical monitoring or evidence of reduced food poisoning.

The implemented stock checks and FreshCast estimates also have a narrower purpose than the inventory-routing model discussed by Violi et al. (2024). This website does not optimise delivery routes or claim that its example orders establish an optimal purchasing policy. It shows which recorded stock can support a date and where forecast demand and expiry may conflict. The external forecasting benchmark supports its reported error comparison, while genuine kitchen records are still needed to assess local forecast accuracy.

Visible order feedback, consistent controls and recovery after a failed request are consistent with the usability principles described by Nielsen (1994). Local browser checks show that the tested actions and layouts work. Whether customers understand

those actions more easily, or spend less effort choosing food, requires the objective-linked user tasks and questionnaire in Section 5.4. The contribution established here is an implemented and technically tested case-study system; wider usability and business effects remain to be evaluated.

5.6 SUMMARY

The functional checks support the tested account, ordering, date, stock, packaging, payment-evidence and reporting rules. Browser checks also identified interface issues that were corrected and retested. Client meetings informed the implemented changes, while formal client acceptance and student survey results remain pending. The following chapter discusses the project's achievements, limitations and possible improvements.

CHAPTER 6
CONCLUSION AND SUGGESTIONS

6.1 INTRODUCTION

This project developed a web-based preorder and freshness management system for home-based food entrepreneurs, using Wrap & Bowl Kuch as its case study. The system connects customer ordering with ingredient batches, packaging and owner reports. Client feedback helped simplify the original design to suit a morning preorder business. This chapter reviews the five objectives, the main limitations and possible improvements.

6.2 REVIEW OF THE PROJECT OBJECTIVES

6.2.1 DATE-BASED PREORDERING AND SCHEDULING

Customers can select a future date and time that match the menus in their basket and the kitchen's delivery or collection hours. Delivery fees depend on the selected area. Checkout checks the accepted price and ingredient and packaging stock usable on that date. Eligible orders can be cancelled or have their time changed, subject to the accepted order and stock limits.

Scheduling in the revised workflow concerns delivery and collection. Detailed preparation planning is no longer required because the client asked for a simpler process. Functional tests support the implemented date and ordering rules, while actual delivery still depends on kitchen preparation, rider availability and customer coordination.

6.2.2 INGREDIENT SHELF LIFE AND STORAGE DURATION

Inventory batches record received dates, quantities, storage locations and expiry evidence. The owner can use a printed date or calculate a deadline from a manufacture or receipt date and a documented duration. Earlier opening or thawing limits and held batches are considered when determining usable stock.

These functions address the second objective by making recorded shelf life and storage duration visible and linking them to stock allocation. They do not detect contamination or confirm physical food safety. Accurate records and proper storage remain necessary.

6.2.3 PERSONALISED SINGLE-VENDOR STOREFRONT

The website presents the client's identity, story, menu, images and contact information without competitor listings. The owner can update business information, categories, food photos, an owner portrait and selling details. Uploads can be previewed before the business story is saved. Customer recognition and the client's satisfaction with the presentation still require user evaluation.

6.2.4 SALES ANALYTICS FOR BUSINESS DECISIONS

The owner can review daily menu quantities, print all matching order slips together and inspect sales for a selected period. Cost estimates and price comparisons support questions about food prices and bulk offers. Missing costs are identified rather than treated as zero.

The reporting functions provide information for decisions, but useful financial conclusions depend on correct purchase costs, payment records and operating expenses. Food contribution is not the same as net profit. Feedback from the client is still needed to establish how useful these reports are in the business routine.

6.2.5 MENU RECOMMENDATIONS

The website offers browsing suggestions and a date-based finder using item count, category and an optional maximum whole-order budget. Customers can compare mixed baskets, including repeated units, or alternatives containing one menu. Tests checked the pricing, quantity and stock-selection rules.

The recommendation feature is implemented, but a successful calculation does not prove that decision fatigue has decreased or purchase conversion has increased. The planned student comparison can examine reported effort and task completion. A study of actual purchases would be needed to establish a change in paid conversion.

6.3 LIMITATIONS

The development records are examples rather than genuine client sales or physical stock counts. Recipes and packaging assignments initially use editable estimates. The client must confirm these values, actual received quantities and purchase costs before relying on the system for live operations.

Inventory monitoring depends on the dates and storage evidence entered by the owner. The application has no temperature sensor or other means of detecting

deterioration. A recorded deadline therefore supports stock planning without guaranteeing freshness or safety.

Future orders are also limited by recorded stock. A meal can be available now but unavailable for a later delivery date because the current batches will have expired. A longer-term enquiry can be discussed with the kitchen, but a WhatsApp conversation does not itself create a website order or sales record. Suitable stock must be entered before the customer places the online order.

Bank transfers, WhatsApp conversations and courier bookings take place outside the website. The kitchen chooses its own driver or an external rider; checkout does not offer courier-provider selection. Uploaded receipts need owner verification, and food receipt needs actual handover confirmation. If a rider cannot collect cash, a COD order requires an agreed alternative payment arrangement with the customer. Tracking does not automatically change the recorded payment method or delivery status.

Passkey access depends on a compatible device and continued access to a saved credential. Testing with a browser test device does not cover every customer's phone or recovery situation. Email reminder reliability also depends on the configured service and schedule.

The local checks do not replace testing under heavy simultaneous use, an independent security review or a study with the intended users. Formal client acceptance and student survey results remain pending. The external forecast result discussed in Chapter 5 is not a measurement of demand accuracy for Wrap & Bowl Kuch.

6.4 SUGGESTIONS FOR FUTURE IMPROVEMENT

The immediate priority is to collect client and student feedback using the prepared forms. The client can assess whether the owner tasks fit the business routine. Students can identify confusing steps in the customer journey and compare browsing with recommendations. The findings should guide further changes and follow-up checks.

The owner should confirm recipe quantities, packaging requirements, area fees and actual stock records. Comparing physical counts with system balances would help maintain reliable information. Sufficient genuine sales and stock history would also make later forecasting evaluation more meaningful.

Further testing could cover a wider range of physical phones, passkey recovery, keyboard access and measured loading performance. The completed PostgreSQL checks include simultaneous stock operations and logical backup restoration, but they do not replace sustained production-load testing or an independent review of operational reliability.

Bank, courier or messaging integrations could be considered if suitable access and a clear business need become available. They may reduce manual work, but would introduce provider requirements, costs and maintenance. The current workflow can continue using manual bank verification and externally arranged delivery.

6.5 CONCLUSION

The project demonstrates an integrated approach to date-based preordering, recorded ingredient monitoring, a single-vendor storefront, sales information and menu recommendations. Client feedback led to a simpler ordering process while preserving the five original objectives. Testing supports the implemented functions under the recorded conditions. Further client and student evaluation is needed to judge ease of use and the benefits in actual business operation.

REFERENCES

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. Agilemanifesto.org

Bernama. (2022, April 7). Malaysia's e-commerce income can breach RM1 trillion again this year - MDEC. https://www.bernama.com/en/business/news.php?id=2069812

Department of Statistics Malaysia. (2025, November 26). Malaysia digital economy (#276): Contribution of ICT to GDP in 2024. https://www.dosm.gov.my/uploads/release-content/file_20260327194302.pdf

Django Software Foundation. (2026). Meet Django. Djangoproject.com

Django Software Foundation. (n.d.). Database transactions. Django documentation. https://docs.djangoproject.com/en/5.2/topics/db/transactions/

Django Software Foundation. (n.d.). User authentication in Django. Django documentation. https://docs.djangoproject.com/en/5.2/topics/auth/

Eu, E. Z. R., & Sameeha, M. J. (2021). Consumers' perceptions of healthy food availability in online food delivery applications and its association with food choices among public university students in Malaysia. Frontiers in Nutrition, 8, 674427. https://doi.org/10.3389/fnut.2021.674427

Food poisoning turns fatal. (2024, June 12). The Star. https://www.thestar.com.my/news/nation/2024/06/12/food-poisoning-turns-fatal

foodpanda. (2012). Backgrounder: foodpanda APAC. https://www.foodpanda.com/wp-content/uploads/2021/01/EN-Backgrounder-foodpanda-APAC.pdf

FoodSafety.gov. (n.d.). Cold food storage chart. https://www.foodsafety.gov/food-safety-charts/cold-food-storage-charts

Gogliettino, M., Balestrieri, M., Ambrosio, R. L., Anastasio, A., Smaldone, G., Proroga, Y. T. R., Moretta, R., Rea, I., De Stefano, L., Agrillo, B., & Palmieri, G. (2020). Extending the shelf- life of meat and dairy products via PET-modified packaging activated with the antimicrobial peptide MTP1. Frontiers in microbiology, 10, 2963. https://www.frontiersin.org/journals/microbiology/articles/10.3389/fmicb.2019.02963/full

Google. (2026). Firebase documentation. Google Cloud. https://firebase.google.com/docs

Gou, Z., Gamage, W., Lau, S. S. Y., & Lau, S. S. Y. (2018). An investigation of thermal comfort and adaptive behaviors in naturally ventilated residential buildings In

tropical climates: A pilot study. Buildings, 8(1), 1-17. https://doi.org/10.3390/buildings8010005

Grab. (2018, May 28). Grab launches GrabFood beta and unveils vision to be an everyday app. Grab Press Centre. https://www.grab.com/sg/press/business/grab-launches-grabfood-beta-and-unveils-vision-to-be-an-everyday-app/

Hansen, C. (2026). Preorder and freshness: Wrap & Bowl Kuch [Computer software]. GitHub. https://github.com/CHplus2/preorder-freshness

i-CATS University College. (2026). IT Project 2 briefing on 17 September [Annotated briefing supplied by the project author].

Islam, A. K. M. Z., & Ferworn, A. (2020). A comparison between agile and traditional software development methodologies. Global Journal of Computer Science and Technology, 20(2), 7-42. https://globaljournals.org/GJCST_Volume20/2-A-Comparison-between-Agile.pdf

Kandoli, L. (2022). Effect of food shelf life on food quality. Journal of Advances in Humanities and Social Sciences, 8(1), 38-42. http://www.tafpublications.com/gip_content/paper/jahss-8.1.5.pdf

Leau, Y. B., Loo, W. K., Tham, W. Y., & Tan, S. F. (2012). Software development life cycle AGILE vs traditional approaches. In International Conference on Information and Network Technology (Vol. 37, No. 1, pp. 162-167). https://ku-fpg.github.io/files/agile-traditional.pdf

Lok, Y. H., Liou, Z. Y., Teoh, K. B., Lim, J. T., Lim, K. E., Lim, P. E., & Kee, D. M. H. (2024). Influencing Factors on Customer Behavioral Intentions to Use a Food Delivery App: A Study of GrabFood in Malaysia. Advances in Global Economics and Business Journal, 5(1), 1-14. https://www.agebj.org/index.php/agebj/article/view/92

Nielsen, J. (1994, April 24). 10 usability heuristics for user interface design. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

OpenJS Foundation. (2026). Express documentation (Version 4.21). https://expressjs.com/

OWASP Foundation. (n.d.). File upload cheat sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Pallianysamy, S., Abdul-Mutalib, N. A., Abidin, U. F. U. Z., Rashid, N. K. M., & Ahmad, N. H. (2025). Home-based Foods in Malaysia: A Food Safety Perspective. Pertanika Journal of Tropical Agricultural Science, 48(3). http://pertanika2.upm.edu.my/resources/files/Pertanika%20PAPERS/JTAS%20Vol.%204 8%20(3)%20May.%202025/01%20JTAS-3151-2024.pdf

PayPal. (n.d.). Orders API v2. PayPal Developer. https://developer.paypal.com/docs/api/orders/v2/

Peres, F. A. P., Bondarczuk, B. A., Gomes, L. D. C., Jardim, L. D. C., Corrêa, R. G. D. F., & Baierle, I. C. (2025). Advances in food quality management driven by Industry 4.0: A systematic review-based framework. Foods, 14(14), 2429. https://www.mdpi.com/2304-8158/14/14/2429

PostgreSQL Global Development Group. (2026). PostgreSQL 18.4 documentation. https://www.postgresql.org/docs/

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. Advances in Neural Information Processing Systems, 31. https://arxiv.org/abs/1706.09516

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26(August), 1-9.

Sixty-five Sungai Petani primary school pupils suffer food poisoning. (2024, May 21). The Star. https://www.thestar.com.my/news/nation/2024/05/21/65-sungai-petani-primary-school-pupils-suffer-food-poisoning

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

Statista. (2024). Online food delivery-Malaysia. https://www.statista.com/outlook/emo/online-food-delivery/malaysia

Susanto, D. A., Suef, M., Karningsih, P. D., & Prasetya, B. (2026). Quality Management System Model for Food SMEs. Sustainability, 18(2), 890. https://www.mdpi.com/2071-1050/18/2/890

Tan, H., & Kim, V. W. E. (2021). Examining the factors that influence consumer satisfaction with online food delivery in Klang Valley, Malaysia. Journal of Management Theory and Practice, 2(2). https://doi.org/10.37231/jmtp.2021.2.2.115

Teo, S. C., Liew, T. W., & Lim, H. Y. (2024). Factors influencing consumers' continuance purchase intention of local food via online food delivery services. Cogent Business & Management, 11(1). https://doi.org/10.1080/23311975.2024.2316919

Tsang, Y. P., Choy, K. L., Wu, C. H., Ho, G. T. S., & Lam, H. Y. (2019, August). An internet of things (IoT)-based shelf life management system in perishable food e-commerce businesses. In 2019 Portland International Conference on Management of Engineering and Technology (PICMET) (pp. 1-8). IEEE. https://ieeexplore.ieee.org/abstract/document/8893684

Violi, A., De Maio, A., & Fattoruso, G. (2024). Inventory management and delivery of perishable products with stochastic demands and risks consideration. Procedia Computer Science, 232, 2941-2949. https://www.sciencedirect.com/science/article/pii/S1877050924002886

World Wide Web Consortium (W3C). (2021, April 8). Web Authentication: An API for accessing public key credentials, Level 2. https://www.w3.org/TR/webauthn-2/

World Wide Web Consortium (W3C). (2023, October 5). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

Wrap & Bowl Kuch. (n.d.). Menu poster and founder introduction [Client materials supplied by the project author].

Zhou, X., Tang, J., Jacobs, T. L., & Saguy, I. S. (2025). Transforming food supply chains through digital tracking and monitoring technologies. Trends in Food Science & Technology, 163, 105142. https://doi.org/10.1016/j.tifs.2025.105142

APPENDIX A — SUPERVISOR CONSULTATION RECORD

Consultation record: awaiting inclusion of the completed institutional log.

APPENDIX B — ORIGINALITY / SIMILARITY RECORD

Similarity summary: awaiting the institutional originality check and its summary record.

APPENDIX C — DECLARATION OF INTEGRITY

Declaration for completion by the author before submission.

I hereby declare that the submitted work together with this form is my original work. The following are further details of this declaration which I approve.

All references and sources relevant to this submission (from prints or the Internet) are inspirations to my written idea and creativity as well as my understanding of used references and sources.

I did not copy any other people's work, be it from amongst the people I know in real life or from random sources on the Internet. Similarities are expected for common facts but I understand that my worded works should not be too similar to other people's works at an acceptable amount set by the college.

If I fail to satisfy my pledge, I am responsible for any errors or negligence that might occur in this submission and I accept the final decision made by the lecturer and the university college. I understand the risk of non-compliance to this pledge may result in a reduction of marks or failure.

APPENDIX D — APPLICATION DATA DICTIONARY

The following tables describe the core application records. Product represents the current menu, while OrderItem preserves the accepted order name and price. Nullable fields can be left without a database value, although form rules may still require them. The remaining supporting and retained entities are listed after these tables. Complete field metadata is supplied in the digital evidence.

Category — myapp_category

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
name	CharField	Unique; Not null; Max 100
created_at	DateTimeField	Not null

Product — myapp_product

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
selling_details	JSONField	Not null
delivery_weekdays	JSONField	Not null
delivery_dates	JSONField	Not null
order_cutoff_time	TimeField	Nullable
order_cutoff_days_before	PositiveSmallIntegerField	Not null
selling_status	CharField	Not null; Max 10
packaging_cost	DecimalField	Nullable; 2 decimal places
lead_hours	PositiveIntegerField	Not null
preparation_minutes	PositiveIntegerField	Not null
preparation_tasks	JSONField	Not null
max_preparation_days	PositiveIntegerField	Not null
batch_size	PositiveIntegerField	Not null
additional_batch_minutes	PositiveIntegerField	Not null

Column	Django type	Storage / relationship properties
packing_minutes_per_portion	PositiveIntegerField	Not null
max_early_minutes	PositiveIntegerField	Not null
daily_capacity	PositiveIntegerField	Not null
social_url	CharField	Not null; Max 200
name	CharField	Not null; Max 200
description	TextField	Not null
ai_summary	TextField	Not null
category_id	ForeignKey	FK → Category; SET_NULL; Nullable
price	DecimalField	Not null; 2 decimal places
image_url	CharField	Not null; Max 200
created_at	DateTimeField	Not null
updated_at	DateTimeField	Not null

RawMaterial — myapp_rawmaterial

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
estimated_unit_cost	DecimalField	Nullable; 6 decimal places
storage_guidance	JSONField	Not null
name	CharField	Unique; Not null; Max 100
unit	CharField	Not null; Max 10
created_at	DateTimeField	Not null

ProductIngredient — myapp_productingredient

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
product_id	ForeignKey	FK → Product; CASCADE; Not null
raw_material_id	ForeignKey	FK → RawMaterial; CASCADE; Not null

Column	Django type	Storage / relationship properties
quantity_required	DecimalField	Not null; 3 decimal places

InventoryItem — myapp_inventoryitem

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
supplier	CharField	Not null; Max 160
source_type	CharField	Not null; Max 20
label_date_type	CharField	Not null; Max 20
original_expiry_date	DateField	Nullable
opened_date	DateField	Nullable
after_open_days	PositiveIntegerField	Nullable
thawed_date	DateField	Nullable
after_thaw_days	PositiveIntegerField	Nullable
handling_history	CharField	Not null; Max 20
handling_note	CharField	Not null; Max 500
unit_cost	DecimalField	Nullable; 6 decimal places
expiry_basis	CharField	Not null; Max 20
manufactured_date	DateField	Nullable
shelf_life_days	PositiveIntegerField	Nullable
storage_type	CharField	Not null; Max 20
guidance_note	CharField	Not null; Max 300
quarantined	BooleanField	Not null
raw_material_id	ForeignKey	FK → RawMaterial; PROTECT; Not null
quantity	DecimalField	Not null; 3 decimal places
batch_code	CharField	Not null; Max 100
storage_location	CharField	Not null; Max 200
received_date	DateField	Not null
expiry_date	DateField	Not null

Column	Django type	Storage / relationship properties
created_at	DateTimeField	Not null
updated_at	DateTimeField	Not null

Order — myapp_order

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
ordering_mode	CharField	Not null; Max 12
stock_returned_at	DateTimeField	Nullable
fulfilment_snapshot	JSONField	Not null
checkout_request_id	UUIDField	Unique; Nullable; Max 32
payment_method	CharField	Not null; Max 20
payment_instructions	JSONField	Not null
preparation_end_at	DateTimeField	Nullable
preparation_plan	JSONField	Not null
delivery_address	JSONField	Not null
delivery_at	DateTimeField	Nullable
preparation_at	DateTimeField	Nullable
delivery_method	CharField	Not null; Max 20
inventory_deducted	BooleanField	Not null
user_id	ForeignKey	FK → User; PROTECT; Not null
address_id	ForeignKey	FK → Address; SET_NULL; Nullable
total_amount	DecimalField	Not null; 2 decimal places
discount_amount	DecimalField	Not null; 2 decimal places
shipping_fee	DecimalField	Not null; 2 decimal places
status	CharField	Not null; Max 20
payment_status	CharField	Not null; Max 20
created_at	DateTimeField	Not null
updated_at	DateTimeField	Not null

OrderItem — myapp_orderitem

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
recipe_source	CharField	Not null; Max 30
preparation_snapshot	JSONField	Not null
packaging_unit_cost	DecimalField	Nullable; 6 decimal places
order_id	ForeignKey	FK → Order; CASCADE; Not null
product_id	ForeignKey	FK → Product; SET_NULL; Nullable
product_name	CharField	Not null; Max 200
unit_price	DecimalField	Not null; 2 decimal places
quantity	PositiveIntegerField	Not null
subtotal	DecimalField	Not null; 2 decimal places

InventoryLog — myapp_inventorylog

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
inventory_item_id	ForeignKey	FK → InventoryItem; SET_NULL; Nullable
change	DecimalField	Not null; 3 decimal places
reason	CharField	Not null; Max 255
reference	CharField	Not null; Max 255
admin_id	ForeignKey	FK → User; SET_NULL; Nullable
created_at	DateTimeField	Not null

Storefront — myapp_storefront

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
ordering_mode	CharField	Not null; Max 12

Column	Django type	Storage / relationship properties
delivery_start_time	TimeField	Not null
delivery_end_time	TimeField	Not null
pickup_start_time	TimeField	Not null
pickup_end_time	TimeField	Not null
delivery_area_fees	JSONField	Not null
delivery_enabled	BooleanField	Not null
pickup_enabled	BooleanField	Not null
pickup_location	CharField	Not null; Max 250
pickup_instructions	TextField	Not null; Max 1000
delivery_open_hour	PositiveSmallIntegerField	Not null
delivery_close_hour	PositiveSmallIntegerField	Not null
pickup_open_hour	PositiveSmallIntegerField	Not null
pickup_close_hour	PositiveSmallIntegerField	Not null
delivery_areas	JSONField	Not null
delivery_fee_myr	DecimalField	Not null; 2 decimal places
free_delivery_minimum_myr	DecimalField	Nullable; 2 decimal places
manual_payment_enabled	BooleanField	Not null
bank_transfer_instructions	TextField	Not null; Max 1000
duitnow_qr_url	CharField	Not null; Max 200
kitchen_open_hour	PositiveSmallIntegerField	Not null
kitchen_close_hour	PositiveSmallIntegerField	Not null
founder_name	CharField	Not null; Max 100
founder_intro	CharField	Not null; Max 250
founding_story	TextField	Not null
food_philosophy	TextField	Not null

Column	Django type	Storage / relationship properties
signature_food	CharField	Not null; Max 200
service_area	CharField	Not null; Max 200
whatsapp_number	CharField	Not null; Max 20
founder_image_url	CharField	Not null; Max 200
name	CharField	Not null; Max 100
tagline	CharField	Not null; Max 200
story	TextField	Not null
social_url	CharField	Not null; Max 200
contact_email	CharField	Not null; Max 254
delivery_buffer_minutes	PositiveIntegerField	Not null
bulk_minimum	PositiveIntegerField	Not null
bulk_discount_percent	PositiveIntegerField	Not null

OperatingExpense — myapp_operatingexpense

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
request_id	UUIDField	Unique; Not null; Max 32
voided	BooleanField	Not null
date	DateField	Not null
category	CharField	Not null; Max 20
amount	DecimalField	Not null; 2 decimal places
note	CharField	Not null; Max 300
created_at	DateTimeField	Not null

WasteRecord — myapp_wasterecord

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null

Column	Django type	Storage / relationship properties
request_id	UUIDField	Unique; Not null; Max 32
inventory_item_id	ForeignKey	FK → InventoryItem; PROTECT; Not null
quantity	DecimalField	Not null; 3 decimal places
estimated_cost	DecimalField	Nullable; 2 decimal places
reason	CharField	Not null; Max 300
created_at	DateTimeField	Not null

OrderIngredient — myapp_orderingredient

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
order_item_id	ForeignKey	FK → OrderItem; CASCADE; Not null
raw_material_id	ForeignKey	FK → RawMaterial; PROTECT; Not null
material_name	CharField	Not null; Max 100
unit	CharField	Not null; Max 10
quantity_per_portion	DecimalField	Not null; 3 decimal places

IngredientConsumption — myapp_ingredientconsumption

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
allocation_stage	CharField	Not null; Max 12
returned_at	DateTimeField	Nullable
order_item_id	ForeignKey	FK → OrderItem; PROTECT; Not null
inventory_item_id	ForeignKey	FK → InventoryItem; PROTECT; Not null
material_name	CharField	Not null; Max 100
batch_code	CharField	Not null; Max 100
unit	CharField	Not null; Max 10
quantity	DecimalField	Not null; 3 decimal places

Column	Django type	Storage / relationship properties
unit_cost	DecimalField	Nullable; 6 decimal places
recorded_expiry	DateField	Not null
created_at	DateTimeField	Not null

PaymentEvent — myapp_paymentevent

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
order_id	ForeignKey	FK → Order; PROTECT; Not null
request_id	UUIDField	Unique; Not null; Max 32
kind	CharField	Not null; Max 10
outcome	CharField	Not null; Max 12
amount	DecimalField	Not null; 2 decimal places
method	CharField	Not null; Max 20
reference	CharField	Not null; Max 200
note	CharField	Not null; Max 500
source	CharField	Not null; Max 30
resolves_id	OneToOneField	FK → PaymentEvent; PROTECT; Unique; Nullable
actor_id	ForeignKey	FK → User; SET_NULL; Nullable
created_at	DateTimeField	Not null

OrderAmendment — myapp_orderamendment

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
order_id	ForeignKey	FK → Order; PROTECT; Not null
actor_id	ForeignKey	FK → User; SET_NULL; Nullable
before	JSONField	Not null
after	JSONField	Not null

Column	Django type	Storage / relationship properties
created_at	DateTimeField	Not null

PackagingItem — myapp_packagingitem

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
name	CharField	Unique; Not null; Max 100
quantity	PositiveIntegerField	Not null
reorder_level	PositiveIntegerField	Not null
unit_cost	DecimalField	Nullable; 6 decimal places
is_active	BooleanField	Not null
created_at	DateTimeField	Not null
updated_at	DateTimeField	Not null

ProductPackaging — myapp_productpackaging

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
product_id	ForeignKey	FK → Product; CASCADE; Not null
packaging_item_id	ForeignKey	FK → PackagingItem; PROTECT; Not null
quantity_per_item	PositiveIntegerField	Not null

OrderPackaging — myapp_orderpackaging

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
order_item_id	ForeignKey	FK → OrderItem; PROTECT; Not null
packaging_item_id	ForeignKey	FK → PackagingItem; PROTECT; Not null
packaging_name	CharField	Not null; Max 100
quantity	PositiveIntegerField	Not null

Column	Django type	Storage / relationship properties
unit_cost	DecimalField	Nullable; 6 decimal places
allocated_at	DateTimeField	Nullable
released_at	DateTimeField	Nullable

PackagingMovement — myapp_packagingmovement

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
packaging_item_id	ForeignKey	FK → PackagingItem; PROTECT; Not null
order_packaging_id	ForeignKey	FK → OrderPackaging; PROTECT; Nullable
request_id	UUIDField	Unique; Nullable; Max 32
kind	CharField	Not null; Max 10
change	IntegerField	Not null
balance_after	PositiveIntegerField	Not null
reason	CharField	Not null; Max 200
actor_id	ForeignKey	FK → User; SET_NULL; Nullable
created_at	DateTimeField	Not null

CheckoutProof — myapp_checkoutproof

Column	Django type	Storage / relationship properties
id	UUIDField	PK; Not null; Max 32
user_id	ForeignKey	FK → User; CASCADE; Not null
order_id	OneToOneField	FK → Order; PROTECT; Unique; Nullable
content	BinaryField	Not null
digest	CharField	Not null; Max 64
scope_digest	CharField	Not null; Max 64
amount	DecimalField	Not null; 2 decimal places
created_at	DateTimeField	Not null

CustomerAccount — myapp_customeraccount

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
user_id	OneToOneField	FK → User; CASCADE; Unique; Not null
user_handle	UUIDField	Unique; Not null; Max 32
name	CharField	Not null; Max 150
phone	CharField	Not null; Max 30
created_at	DateTimeField	Not null
updated_at	DateTimeField	Not null

CustomerPasskey — myapp_customerpasskey

Column	Django type	Storage / relationship properties
id	BigAutoField	PK; Not null
account_id	ForeignKey	FK → CustomerAccount; CASCADE; Not null
credential_id	CharField	Unique; Not null; Max 1400
public_key	BinaryField	Not null
sign_count	PositiveBigIntegerField	Not null
label	CharField	Not null; Max 80
device_type	CharField	Not null; Max 32
backed_up	BooleanField	Not null
created_at	DateTimeField	Not null
last_used_at	DateTimeField	Nullable

Supporting and retained entities

Entity	Purpose
MenuPhoto	Uploaded menu, owner-profile and receiving-image storage.
Wallet	Retained earlier wallet records.
WalletTransaction	Retained earlier wallet movements.

Entity	Purpose
Address	Supporting saved address records; accepted orders keep their own details.
CartItem	Customer basket quantities.
Review	Retained review records.
OrderReminder	Saved reminder receipts.
KitchenBlock	Retained preparation-planning blocks.
AuthAttempt	Sign-in attempt tracking.
RecommendationSession	Saved recommendation search state.
RecommendationEvent	Recommendation interaction records.
GuestCustomer	Browsing-session identity.
PasskeyCeremony	Temporary account-verification challenges.

APPENDIX E — CLIENT FOLLOW-UP AND ACCEPTANCE FORM

Complete this follow-up after trying the current system. It can be recorded on paper or in Google Forms. Use example records in the supervised test setup for actions that change stock, orders or payments.

Item	Response
Business	Wrap & Bowl Kuch
Client / date	_____ / _____
Permission to record feedback	[ ] Agreed [ ] Not agreed

For each task: I = completed independently; H = completed with help; F = could not complete; N = not tried. Use N when the task was only described and was not attempted.

Task / objective	Task	Outcome / comment
C1 / O3	Review the kitchen profile and one menu, including price, selling unit, offered days and recipe.	I / H / F / N _____
C2 / O1	Check delivery areas and fees, morning hours and the collection option.	I / H / F / N _____
C3 / O2	Try an ingredient batch in the test setup; explain its date method, deadline and hold status.	I / H / F / N _____
C4 / O1, O2	Review packaging stock and the pieces assigned to one menu.	I / H / F / N _____
C5 / O1, O2	Check a stock target and compare the ingredient and packaging deductions for an example order.	I / H / F / N _____
C6 / O1	Review payment evidence and food-received status; explain the difference.	I / H / F / N _____
C7 / O1, O4	Open a daily order summary and preview a slip containing the customer name, address and phone.	I / H / F / N _____
C8 / O4	Read a sales period and compare a price scenario; explain what the cost estimate includes.	I / H / F / N _____

1. Which parts fit your business routine, and which are difficult or unnecessary?

Response: _____

2. Are name, phone and passkey accounts practical for your customers? What concern remains?

Response: _____

3. Which menu, recipe, packaging, delivery-fee or other details still need correcting?

Response: _____

4. What must change before you would use this system for actual orders?

Response: _____

Overall assessment: [ ] Suitable for the tasks tried [ ] Suitable after changes [ ] Not yet suitable [ ] Not enough tried to decide.

Changes agreed: _____

Client name / optional signature: _____ Date: _____

APPENDIX F — STUDENT CUSTOMER-USABILITY QUESTIONNAIRE

This questionnaire asks about the customer experience of Wrap & Bowl Kuch. Participation is voluntary. Use the agreed test setup and example details; stop before a real payment or live order placement. Do not enter a personal phone number, email address or bank receipt in the questionnaire.

Item	Response
Consent	[ ] I agree to take part. [ ] I do not agree (stop).
Participant code / date	_____ / _____
Device	[ ] Phone [ ] Tablet [ ] Laptop or desktop

1. Browse the menu and open Ingredient storage records. Review what is recorded and the explanation of date-based status.

2. Task A: start with an empty basket and use ordinary browsing to choose three selling units for an assigned offered date and area, within the agreed budget. Stop at checkout review.

3. Task B: start again with an empty basket and use Suggestions for a date for a comparable date, area, quantity and budget. Review an option and checkout guidance. Stop before payment or order placement.

The evaluator assigns feasible dates, areas and budgets. Empty the basket before each comparison task. Task order: [ ] A then B [ ] B then A [ ] Only A [ ] Only B [ ] Neither. Do not rate a step you did not try.

Scale: 1 = strongly disagree, 2 = disagree, 3 = neither agree nor disagree, 4 = agree, 5 = strongly agree; N/A = not tried or not applicable.

Item / objective	Statement	Rating
S1 / O3	I could identify the business I was ordering from.	1 2 3 4 5 N/A
S2 / O3	The price and contents of a selling unit were clear.	1 2 3 4 5 N/A
S3 / O1	I could find an offered date and an allowed delivery or collection time.	1 2 3 4 5 N/A

Item / objective	Statement	Rating
S4 / O1	I could see the delivery fee and complete order total.	1 2 3 4 5 N/A
S5 / O1	The account and passkey instructions were understandable.	1 2 3 4 5 N/A
S6 / O1	I understood that an uploaded receipt still needs payment verification.	1 2 3 4 5 N/A
S7 / O2	The ingredient storage summary was understandable.	1 2 3 4 5 N/A
S8 / O5	The suggestions matched my date, quantity and budget choices.	1 2 3 4 5 N/A
S9 / O5	The difference between Allow a mix and One menu only was clear.	1 2 3 4 5 N/A

S10 / O5: Compared with ordinary browsing, how much effort did choosing with suggestions require?

[ ] Much less effort [ ] Less effort [ ] About the same effort [ ] More effort [ ] Much more effort [ ] I did not try both

S11: Which step was most confusing, and what would you change?

Response: _____

_____

Thank you. These responses describe customer experience; they do not certify food safety or verify the owner's accounting calculations.

APPENDIX G — SUPPORTING TEST EVIDENCE

The digital report package retains the detailed evidence behind the functional test tables. Automated results and browser checks are separate from the client and student evaluation forms. The package does not contain completed survey responses or a signed client acceptance record.

Evidence	Purpose
Functional-test mapping and logs	Connects the readable cases in Chapter 5 to the executed automated checks and their recorded outcomes.
Verification summary	Lists the test environment, command outcomes and skipped checks.
Browser capture record	Identifies the current local screenshots, their screen sizes and the example-record origin.
Schema metadata and editable diagrams	Supports review of the model relationships and diagram content.

Technical filenames, full test names and source identifiers are retained in the digital evidence rather than reproduced as long listings in this report.

APPENDIX H — SHORT OWNER OPERATING GUIDE

H.1 Prepare the kitchen profile

Sign in through Kitchen staff sign in. Review the business name, introduction, approved public contact and service description. In Your business story, upload and preview the owner photo, then save that section to publish it. Save each settings section separately.

H.2 Confirm delivery and collection

Use short area names with their own fees; keep descriptive sentences in the service description. Review Delivery from and Delivery until above Delivery areas & fees, then save the delivery settings. Enable collection explicitly and provide a genuine collection location. Test a new quote after saving.

H.3 Configure receiving payment

Use business-approved QR/bank receiving details. Collection needs a QR image. Verify the displayed recipient and test only with authorised evidence; uploading an image does not verify a transfer.

H.4 Review menus and packaging

Check prices, offered weekdays/dates, selling units and contents. Confirm estimated recipes and packaging pieces with the actual kitchen. Enter known costs or leave them unknown; do not use zero merely to remove a warning.

H.5 Receive ingredients

Select the ingredient and its inventory unit, actual received date and storage. Choose Printed expiry date, Received date + shelf life, or Manufacture date + shelf life, with the relevant evidence. Review the earliest deadline and hold uncertain batches. Use Save and add another to enter the next batch.

H.6 Receive packaging

Record actual received whole pieces and reason. Map the pieces used by one selling unit of each menu. Review removals and movement history; do not duplicate an automatic order allocation.

H.7 Check a stock target

Choose a date and units per offered menu. Read the total units and included-menu list. Replenish only quantities actually obtained; the check and worksheet do not create inventory.

H.8 Accept and review orders

Use the order queue and date filters. Placement already allocates ingredients and packaging. Review shortages and bank/cash evidence without repeating deductions. A customer cancellation appears in the cancelled view; discuss any payment separately and return unused stock only after checking it.

H.9 Record payment and receipt separately

Compare a transfer with actual bank records before confirming money received. Confirm food received only after handover/collection. Review the exact selected set before bulk confirmation. Use corrections/cancellation with reasons rather than rewriting history.

H.10 Print and interpret reports

Open Daily order summary for packing quantities. Choose Print all for matching orders, review the slips and press Print once. Individual previews remain available. Use Sales for paid-and-received periods. Contribution needs known costs and is not net profit; a price scenario changes no live price.

H.11 Review reminders and contact

Refresh status to inspect recent email activity; it does not send a message. Use the test-email button for its stated check. Phone/passkey accounts are not automatically subscribed to email, SMS or WhatsApp. For an external rider, copy recipient details and save any tracking reference from the order. Agree cash collection or an alternative payment directly with the customer before dispatch. Tracking does not change payment or food-receipt status.

H.12 Recover and maintain

Keep database backups separate from source and model files. Use a backup or synchronised customer passkey. Record the page, action and error when reporting a problem. Review deployment and migration checks before using an updated system.